

Concepts from Zero

Every machine-learning and deep-learning idea in the Waste & Trash Detection study guide, explained for a beginner, worked through with real numbers, and linked back to the guide page by page.

Prepared for Sreyanshu and team · 27 September 2026 · Evaluation on 29–30 September

At a glance

What it is	85 concepts in 10 parts, from "what is a model?" to "why does SSD300 miss cigarette butts?". Read it alongside the study guide: the guide says what to do, this companion explains why it works.
Each concept	A one-line summary, a plain explanation with an everyday comparison where it helps, a worked example with numbers (most of them from this project), and an "In your project" box that points to the notebook code and the viva questions.
Links to the guide	§4.3 (p. 12) is section 4.3 on page 12 of the study guide. Q24 is viva question 24 (guide chapter 12), D3 is decision 3 (chapter 2), and Step 5 is notebook step 5 (chapter 10). "Concept 4.3" always means this companion.
Finding things	The reading map (p. 4) goes from each part of the guide to the concepts behind it. Appendix A (p. 74) maps all 50 viva questions to concepts, and Appendix B (p. 76) is an index of terms. In the PDF, concept numbers are clickable.
Assumes	Python and school maths. Nothing else.

How to use this companion

Each concept starts with the places it appears in the guide, such as §4.3 p. 12 , so you can read the two side by side. Then comes the idea in one line, the explanation, a worked example and the link to your project. Concepts build on earlier ones, and when one relies on another it names it ("concept 1.4"), so you can always step back.

If you have...	Read
One evening, before the viva	Part 4 (detection building blocks), Part 5 (the three models), concepts 8.1–8.5 (metrics) and 9.2–9.3 (error types). Together they cover more than half of the 50 viva questions. Then test yourself with Appendix A.
Two days	Day 1: Parts 1–3, the foundations (how learning works, neural networks, CNNs), about 4 hours. Day 2: Parts 4, 5, 8 and 9, about 5 hours.
While building or writing	Part 6 next to Steps 2–5, Part 7 next to Step 6, Part 10 whenever a tool or an error puzzles you, and concepts 7.5, 8.9 and 9.5 while writing the report.

Contents

Reading map: from the guide to the concepts	4	4.3	Yes/no scores: sigmoid and binary cross-entropy	26
Part 1 • How machines learn from examples	5	4.4	Bounding boxes and the three label formats	27
1.1 Machine learning and supervised learning	5	4.5	Intersection over Union (IoU)	28
1.2 Models, parameters and architecture	5	4.6	Anchors: a fixed set of candidate boxes	29
1.3 Loss: a score for how wrong the model is	6	4.7	Giving every anchor a target: positive, negative or ignored	30
1.4 Gradient descent: how the weights get adjusted	6	4.8	Box regression: learning corrections to anchors	31
1.5 Backpropagation: finding every weight's gradient	7	4.9	Box losses: L1, smooth L1 and GIoU	32
1.6 Batch, iteration and epoch	8	4.10	The imbalance between objects and background	33
1.7 Optimizers: SGD, momentum, Adam and AdamW	8	4.11	Post-processing: score threshold, top-k and NMS	34
1.8 Learning-rate schedules: warm-up and cosine decay	9	4.12	The Feature Pyramid Network (FPN)	35
1.9 Generalisation, overfitting and underfitting	9	4.13	Families of detectors: two-stage, one-stage and beyond	36
1.10 Training, validation and test sets	10			
1.11 Data leakage	11	Part 5 • The three models, stage by stage	38	
1.12 Hyperparameters	11	5.1	Faster R-CNN: propose, then inspect	38
1.13 Randomness, seeds and reproducibility	12	5.2	RetinaNet and focal loss	40
Part 2 • Neural networks	13	5.3	SSD300: one quick look at six scales	42
2.1 The artificial neuron	13	5.4	Comparing the three designs	43
2.2 Activation functions: ReLU, sigmoid and softmax	13	Part 6 • Data: from files to training examples	45	
2.3 Layers, depth and fully connected layers	14	6.1	Labelled datasets and what's in this one	45
2.4 Tensors: how PyTorch stores numbers	15	6.2	Label files: XML, JSON and TXT	45
2.5 Normalisation layers: BatchNorm and GroupNorm	15	6.3	EXIF orientation: the silent rotation bug	46
2.6 Initialisation: where the weights start	16	6.4	Cleaning the labels	47
2.7 GPUs, CUDA and GPU memory	16	6.5	Exploring the data before training	48
2.8 Mixed precision (AMP)	17	6.6	Class imbalance between the waste classes	48
2.9 Exploding gradients, clipping and nan	17	6.7	Splitting by photo, and the seed search	49
Part 3 • Seeing images: convolutional networks	19	6.8	Augmentation for detection	49
3.1 Digital images, pixels and coordinates	19	Part 7 • Training in practice	51	
3.2 Convolution: sliding pattern detectors	19	7.1	One training iteration, line by line	51
3.3 Stride, padding, pooling and downsampling	20	7.2	Training mode and evaluation mode	52
3.4 Feature maps, feature vectors and receptive fields	21	7.3	Checkpoints and keeping the best epoch	52
3.5 1 × 1 and 3 × 3 convolutions as building blocks	21	7.4	Reading the training curves	53
3.6 Backbones: VGG-16 and ResNet-50	22	7.5	A fair experiment: change only the model	54
3.7 Backbone, neck and head	22	Part 8 • Measuring detection quality	55	
3.8 Transfer learning, fine-tuning and freezing	23	8.1	Correct, false and missed: matching predictions to objects	55
3.9 Upsampling and bilinear interpolation	23	8.2	Precision, recall and F1	56
Part 4 • From classification to object detection	25	8.3	The confidence threshold, and choosing it fairly	56
4.1 What a detector outputs	25	8.4	The precision-recall curve and Average Precision (AP)	57
4.2 Cross-entropy: the loss for choosing a class	25			

8.5	The mAP family	59	Part 10 • The tools: Colab, PyTorch and friends	67
8.6	The confusion matrix for detection	60	10.1 Colab, notebooks and runtimes	67
8.7	COCO evaluation with torchmetrics, and why we also count by hand	60	10.2 Files, paths and Google Drive	67
8.8	Speed and size	61	10.3 PyTorch, torchvision and the other libraries	68
8.9	How much to trust the numbers	62	10.4 Dataset, DataLoader and collate	69
Part 9 • Error analysis	63		10.5 Using torchvision's detection models	69
9.1	From a score to an explanation	63	10.6 transforms.v2 and tv_tensors: how boxes move with pixels	70
9.2	False detections: four reasons	64	10.7 The result files, and where each one goes	71
9.3	Missed objects: five reasons	64	10.8 Saving and loading weights	71
9.4	Recall by object size, and from cause to remedy	65	10.9 Running the models on a laptop	72
9.5	Your own photos, domain shift, and writing observations	66	10.10 Debugging: reading errors and the sanity check	73
			Appendix A • Viva questions and where they're explained	74
			Appendix B • Index	76

Reading map: from the guide to the concepts

Each row is a part of the study guide. The numbers on the right are the companion concepts that explain the ideas used there. Read the guide section, and whenever something isn't clear, follow the arrow.

Study guide	Concepts in this companion
Ch. 1 The 5-day plan p. 4	Sharing files through Drive 10.2 · a fair split of the work 7.5 · Colab and GPU quota 10.1
Ch. 2 The thought process p. 6	§2.1 the brief → 4.1 , 9.1 · D1 → 3.8 · D2 → 4.10 , 4.13 , 5.4 · D3 → 1.10 , 1.11 , 6.7 · D4 → 6.8 · D5 → 7.5 · D6 → 1.10 , 8.3 · D7 → 8.8 · §2.3 → 1.12 , 7.5 · §2.5 hypotheses → 5.4 , 8.9 · §2.6 pitfalls → 6.3 , 1.11 , 4.4 , 7.4 , 8.3 , 8.9 , 10.2
Ch. 3 Deep-learning refresher p. 10	§3.1 → 3.1-3.4 · §3.2 → 3.6 , 3.7 · §3.3 → 3.8 · §3.4 → 1.4-1.8 , 1.13 , 2.8 , 2.9 · §3.5 → 1.9 · §3.6 → 1.10 . Background for the whole chapter: Parts 1-3.
Ch. 4 How object detectors work p. 12	§4.1 → 4.1 · §4.2 → 4.4 · §4.3 → 4.5 · §4.4 → 4.6 , 4.7 · §4.5 → 4.8 , 4.9 · §4.6 → 4.10 · §4.7 → 4.11 · §4.8 → 4.12 · §4.9 → 4.13
Ch. 5 The three models p. 16	§5.1 → 5.1 , with 4.2 , 4.3 for its losses · §5.2 → 5.2 , with 2.5 , 4.9 · §5.3 → 5.3 · §5.4 → 5.4 , 8.8
Ch. 6 Data p. 21	§6.1 → 6.1 , 1.11 · §6.2 → 6.2 , 4.4 · §6.3 → 6.3 , 6.4 · §6.4 → 6.5 · §6.5 → 6.7 · §6.6 → 6.8 , 10.6 · §6.7 → 6.6
Ch. 7 Training p. 24	§7.1 → 1.7 , 1.8 , 1.12 , 7.5 · §7.2 → 7.1 , 7.2 , 1.5 · §7.3 → 7.4 , 1.9 · §7.4 → 2.7 , 2.8
Ch. 8 Evaluation metrics p. 26	§8.1 → 8.1 · §8.2 → 8.2 · §8.3 → 8.3 · §8.4 → 8.4 , 8.7 · §8.5 → 8.5 · §8.6 → 8.6 · §8.7 → 8.8 · §8.8 → 8.9
Ch. 9 Error analysis p. 29	§9.1 → 9.1-9.3 · §9.2 → 9.4 · §9.3 → 9.4 · §9.4 and §9.5 → 9.5
Ch. 10 Build guide p. 31	Steps 0-1 → 10.1 , 10.2 · Step 2 → 6.2-6.4 · Step 3 → 6.5 · Step 4 → 6.7 · Step 5 → 6.8 , 10.4-10.6 , 3.8 and Part 5 · Step 6 → 7.1-7.4 · Step 7 → 8.1-8.8 , 10.8 · Step 8 → 8.6 , 9.1-9.4 · Step 9 → 9.5 , 10.9
Ch. 11 Writing the report p. 39	7.5 fairness statement · 8.9 limitations · 9.5 observations · 10.7 which file feeds which table
Ch. 12 Viva preparation p. 41	Appendix A lists, for every question, the concepts that explain the answer.
Ch. 13 Troubleshooting p. 45	10.10 debugging and the sanity check · 2.7-2.9 memory, AMP and <code>nan</code> · 6.3 , 6.4 label problems
App. A Formula sheet p. 46	IoU 4.5 · box formats 4.4 · box encoding 4.8 · P, R, F1 8.2 · AP 8.4 · mAP 8.5 · focal loss and prior 5.2 · SSD boxes 5.3 · schedule 1.8 · iterations 1.6 · object size 6.5
App. B Glossary p. 47	Every glossary term has a full explanation here: use the index (Appendix B of this companion).

How machines learn from examples

The guide takes a few ideas for granted: models, weights, loss, learning rate, epochs, overfitting, validation. This part builds each of them from nothing, using school algebra and small numbers. Everything later in this companion stands on these thirteen ideas.

1.1 Machine learning and supervised learning

IN THE GUIDE §2.1 p. 6 §4.1 p. 12 §6.1 p. 21

In one line. Instead of writing the rules yourself, you show the computer many examples together with the right answers, and it works out the rules on its own.

Think about how you would write a normal program to recognise a can: "if the object is shiny, cylindrical, about 12 cm tall...". It falls apart immediately. Cans are crushed, photographed from above, half hidden under leaves, lit by street lamps, printed with every colour. Nobody can write rules for every case.

Machine learning turns the problem around. You collect **examples**, each with its **input** (a photo) and the **correct answer** for it. A learning algorithm then adjusts a model (concept 1.2) until its answers match the correct ones as closely as possible. If it has learned the general pattern rather than memorised the examples, it can also answer for photos it has never seen.

When every example comes with its correct answer, this is called **supervised learning**: the answers "supervise" the learning like a teacher marking homework. The correct answer is called the **label**, **target** or **ground truth**, and the model's answer is its **prediction**. (There are other kinds of machine learning, such as learning without labels or learning from rewards, but this project only uses supervised learning.)

EVERYDAY COMPARISON

A child learns what a cup is by being shown many cups and non-cups and gently corrected, not by memorising a dictionary definition. After enough examples they recognise a cup they've never seen before. That last step, handling new cases, is the whole point.

IN YOUR PROJECT

The inputs are the 163 street-trash photos. The labels are the boxes and class names stored in the Pascal VOC .xml files: the "ground truth" drawn by the dataset's author. The trained detectors are then judged on photos they never learned from: the 25 test photos, and your own phone photos in Step 9.

1.2 Models, parameters and architecture

IN THE GUIDE §5.4 p. 19 Step 7 p. 36 Step 9 p. 38

In one line. A model is a mathematical function with adjustable numbers inside it, called **parameters** or **weights**. Training means finding good values for them.

The smallest possible model is a straight line: $prediction = w \times input + b$. The two numbers w and b are its parameters. With $w = 2$ and $b = 1$, an input of 3 gives 7; change w to 3 and the same input gives 10. The formula stays the same; the parameters decide how it behaves.

A detector is the same idea on a gigantic scale: tens of millions of parameters arranged in layers (Parts 2 and 3). Two words keep this clear:

- **Architecture:** the design of the function, meaning which layers exist and how they connect. "Faster R-CNN with a ResNet-50 backbone" is an architecture.
- **Weights:** the learned numbers that fill that design. The same architecture with different weights behaves completely differently, just as the same piano plays different music depending on the keys pressed.

A **checkpoint** (a `.pt` file) stores only the weights. To use it you must first build the identical architecture, then load the numbers into it.

IN YOUR PROJECT

With our 9-class heads, Faster R-CNN has about 43.3 million parameters, RetinaNet 36.5 million and SSD300 24.7 million (\$5.4, p. 19). That's the "Params (M)" column in your results table. `load_trained` (Step 7) and `build()` in `predict.py` both rebuild the architecture with `weights=None`, then call `load_state_dict` to fill it with your trained numbers.

1.3 Loss: a score for how wrong the model is

IN THE GUIDE §5.1 p. 16 §5.2 p. 17 §5.3 p. 19 §7.3 p. 25

In one line. The loss is one number that measures how far the model's predictions are from the labels. Training tries to make it as small as possible.

For each training example we compare the prediction with the label and turn the difference into an error score. Averaging these over a group of examples gives the **loss**. A loss of 0 would mean every prediction was perfect. The formula that produces it is the **loss function**, and different tasks use different ones:

- for numbers, e.g. squared error: label 10, prediction 7 gives $(10 - 7)^2 = 9$, while prediction 9.5 gives only 0.25;
- for choosing a class: cross-entropy (concept 4.2);
- for box positions: smooth L1 or GIoU (concept 4.9).

A detector must get two things right at once, *what* and *where*, so its total loss is a sum of parts: a classification part plus a box part (plus two more for Faster R-CNN's first stage).

Don't compare losses between models. Each model adds up different parts with different formulas, so RetinaNet's 0.4 and SSD's 3.0 aren't on the same scale. That's why the guide's training chart divides each model's loss by its own first-epoch value (§7.3, p. 25).

IN YOUR PROJECT

Each epoch, the training log prints `loss`. It is the sum of the dictionary torchvision returns, e.g. `loss_classifier + loss_box_reg + loss_objectness + loss_rpn_box_reg` for Faster R-CNN. The guide's chapter 5 explains each part per model.

1.4 Gradient descent: how the weights get adjusted

IN THE GUIDE §3.4 p. 10 §7.1 p. 24 §7.3 p. 25 ch. 13 p. 45

In one line. Over and over, nudge every weight a little in the direction that makes the loss smaller.

Picture the loss as a hilly landscape: every possible setting of the weights is a spot on the ground, and the height is the loss there. Training means finding a deep valley. You are standing in fog and can't see the valley, but you can feel the slope under your feet. So you take a small step downhill, feel the slope again, and repeat.

The slope is the **gradient**. For each weight it says how much the loss would change if that weight went up a tiny bit. A positive gradient means "increasing this weight increases the loss", so we decrease it, and vice versa. The update rule is:

$$w_{\text{new}} = w - \text{learning rate} \times \text{gradient}$$

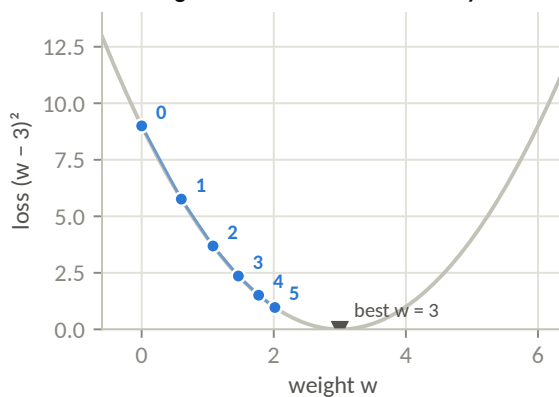
The **learning rate** is the step size, and it matters enormously:

WORKED EXAMPLE

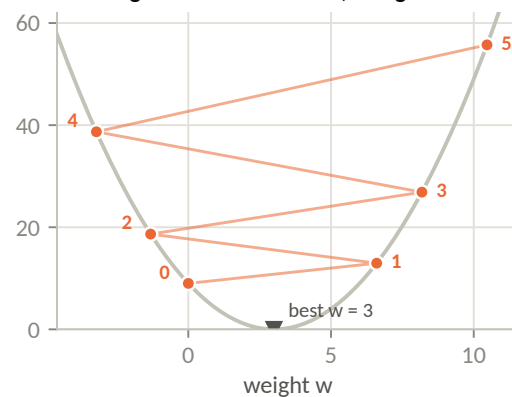
Let the loss be $(w - 3)^2$, so the best weight is 3. Its gradient is $2(w - 3)$. Start at $w = 0$:

- **Learning rate 0.1:** $0 \rightarrow 0.6 \rightarrow 1.08 \rightarrow 1.46 \rightarrow 1.77 \rightarrow 2.02 \dots$ steadily approaching 3.
- **Learning rate 1.1 (too big):** $0 \rightarrow 6.6 \rightarrow -1.32 \rightarrow 8.18 \rightarrow -3.22 \rightarrow 10.47 \dots$ every step overshoots further. In a real network this is the loss exploding to `nan`.
- **Learning rate 0.01 (too small):** $0 \rightarrow 0.06 \rightarrow 0.12 \rightarrow 0.18 \dots$ right direction, but hundreds of steps needed.

Learning rate 0.1: walks into the valley



Learning rate 1.1: overshoots, and gets worse



The first two cases above. A sensible step size walks into the valley; a step size that's too big bounces out of it.

Computing the exact slope over *all* training photos for every step would be slow. So in practice each step uses a small group of photos (a batch, concept 1.6). This is **mini-batch** or **stochastic gradient descent**: each step's slope is a slightly noisy estimate, but steps are cheap and it works very well.

IN YOUR PROJECT

Every iteration of `train_model` takes one such step for about 40 million weights at once (`opt.step()`). The "unstable" pattern in §7.3 (p. 25) and "Loss becomes nan" in ch. 13 (p. 45) are the too-big-step case, and the fix given there is a smaller learning rate (`LR = 5e-5`).

1.5 Backpropagation: finding every weight's gradient

IN THE GUIDE §7.2 p. 24 Step 6 p. 35

In one line. Backpropagation computes the gradient of every weight efficiently by starting at the loss and passing "blame" backwards through the layers.

A network is a long chain: photo $\rightarrow$ layer 1 $\rightarrow$ layer 2 $\rightarrow \dots \rightarrow$ prediction $\rightarrow$ loss. Running a photo through the chain is the **forward pass**. To know how an early weight affects the loss, you multiply how much each link in the chain passes a change along (the **chain rule** from calculus). Doing this separately for 40 million weights would take forever.

Backpropagation does it in one sweep from the end back to the start, reusing each layer's result for the layers before it. This **backward pass** costs roughly as much as a forward pass.

EVERYDAY COMPARISON

A factory produces a faulty product. The final inspector finds the fault, and each station passes back to the station before it "how much of this was your doing". Every station learns its share of the blame in one walk back down the line.

PyTorch does all of this automatically (a feature called **autograd**). During the forward pass it records every operation, and `loss.backward()` then fills in a `.grad` for every weight. Two details show up in the code:

- Gradients *add up* across calls, so the old ones must be cleared before each new step: `opt.zero_grad()`.
- Weights with `requires_grad = False` (frozen layers, concept 3.8) get no gradient and are never changed.

IN YOUR PROJECT

These lines of Step 6 are one full cycle: `losses = model(imgs, tgts) (forward)`, `opt.zero_grad(...)`, `scaler.scale(loss).backward()` (backward), then `scaler.step(opt)` (the gradient-descent step).

1.6 Batch, iteration and epoch

IN THE GUIDE §3.4 p. 10 §7.1 p. 24 ch. 13 p. 45

In one line. A **batch** is a handful of photos processed together, an **iteration** is one weight update from one batch, and an **epoch** is one pass through every training photo.

Why batches rather than one photo at a time? The GPU can process several photos in parallel almost as fast as one. The gradient averaged over several photos is less noisy. And GPU memory limits how many fit at once. Before each epoch the photos are **shuffled**, so the batches differ every time.

WORKED EXAMPLE: YOUR NUMBERS

114 training photos ÷ batch size 4 = 28.5, so each epoch has **29 iterations** (28 full batches and a last batch of 2). Thirty epochs make $30 \times 29 = 870$ **iterations**: the weights are updated 870 times in total. Each photo is seen 30 times, each time with a different random augmentation (concept 6.8).

IN YOUR PROJECT

`BATCH_SIZE = 4` and `EPOCHS = 30` in the settings cell; the training log prints one line per epoch. A bigger batch needs more GPU memory, which is why the fix for "CUDA out of memory" is `BATCH_SIZE = 2` (ch. 13, p. 45).

1.7 Optimizers: SGD, momentum, Adam and AdamW

IN THE GUIDE §3.4 p. 10 §7.1 p. 24 Q24 p. 42

In one line. The optimizer is the rule that turns gradients into weight changes. AdamW, which we use, gives every weight its own sensible step size and gently shrinks the weights.

- **SGD** (stochastic gradient descent) is exactly the rule of concept 1.4: weight minus learning rate times gradient.
- **Momentum** keeps a running average of recent gradients and moves along that average, like a heavy ball rolling downhill that isn't knocked sideways by every small bump. Progress along consistent directions speeds up.
- **Adam** adds a second idea: it divides each weight's step by the recent typical size of that weight's gradients. Weights with big, jumpy gradients take careful steps, and weights with tiny gradients still make progress. This makes Adam much less sensitive to the exact learning rate.

- **Weight decay** shrinks every weight slightly towards zero at every step. It discourages the model from relying on a few extreme weights and is a mild defence against overfitting (concept 1.9).
- **AdamW** is Adam with the weight decay applied as a separate, direct shrink ("decoupled"), not mixed into the gradient. It is a standard choice for fine-tuning pretrained networks.

IN YOUR PROJECT

`torch.optim.AdamW(params, lr=1e-4, weight_decay=1e-4)`, where `1e-4` is scientific notation for 0.0001. All three models use exactly this setting. §7.1 (p. 24) explains why one setting is fair, and Q24 (p. 42) is the matching viva answer.

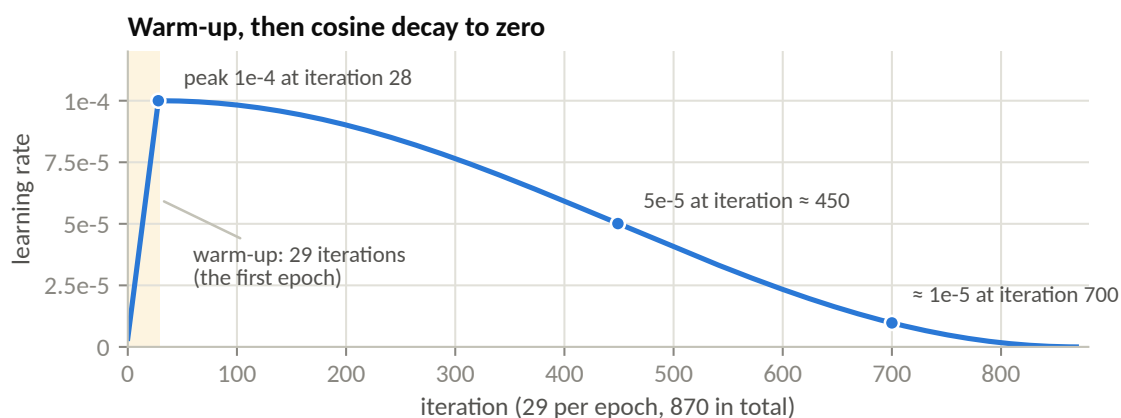
1.8 Learning-rate schedules: warm-up and cosine decay

IN THE GUIDE §3.4 p. 10 §7.1 p. 24 Step 6 p. 35 App. A p. 46 Q25 p. 43

In one line. The learning rate changes during training: it ramps up gently at the start, then glides down to zero by the end.

Warm-up (the first epoch, 29 iterations). The learning rate climbs in a straight line from $0.0001 \div 29 \approx 0.0000034$ up to 0.0001. The reason: the new final layer starts with random weights (concept 2.6), so its first gradients are large and meaningless. Full-size steps at that moment could scramble the carefully pretrained layers behind it.

Cosine decay (iterations 29 to 870). The learning rate then follows half of a cosine wave from 0.0001 down to 0. It's 0.00005 halfway through (iteration ≈ 450) and about 0.00001 at iteration 700. Big steps early make fast progress; tiny steps at the end let the weights settle precisely into a good valley instead of bouncing around it.



The exact schedule used for every model: 29 warm-up iterations, then cosine decay to zero at iteration 870.

IN YOUR PROJECT

In `train_model`, the function `lr_factor(s)` returns the multiplier for iteration `s`, `LambdaLR` applies it to the base rate, and `sched.step()` moves to the next iteration after every update. The formula is in App. A (p. 46), and "why warm-up?" is viva Q25 (p. 43).

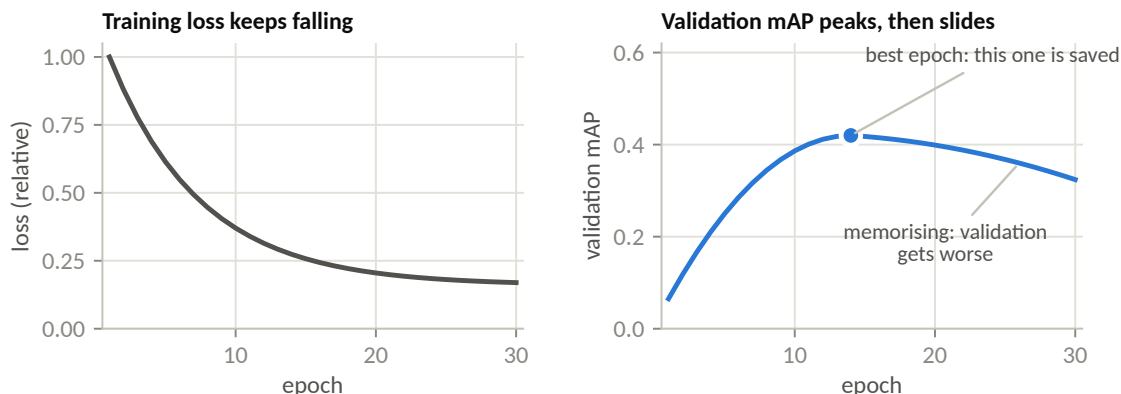
1.9 Generalisation, overfitting and underfitting

IN THE GUIDE §3.5 p. 11 §7.3 p. 25 Q27 p. 43

In one line. A good model does well on new photos (it generalises). An overfit model has memorised its training photos, and an underfit one hasn't learned enough.

Three students prepare for an exam with a practice sheet. One memorises the answers word for word (**overfitting**): perfect on the practice sheet, lost on the real exam. One understands the method (**generalisation**): good on both. One barely studies (**underfitting**): poor on both.

With only 114 training photos and about 40 million weights, a network can easily memorise details that don't matter, like a particular shadow or a particular patch of pavement. You spot overfitting by watching two curves: **training loss keeps falling while validation performance stops improving or gets worse**.



An illustration, not real results. Training loss keeps dropping, but validation mAP peaks and then slides as the model starts memorising. Early stopping keeps the peak epoch.

Anything that discourages memorising is called **regularisation**. The project uses four kinds: pretrained weights (concept 3.8), augmentation (concept 6.8), weight decay (concept 1.7) and **early stopping**. Early stopping means measuring on the validation photos after every epoch and keeping the weights from the best epoch, whatever happens afterwards.

IN YOUR PROJECT

`train_model` saves the checkpoint only when validation mAP beats its previous best. The dots in `training_curves.png` mark those saved epochs. This is the answer to viva Q27 (p. 43).

1.10 Training, validation and test sets

IN THE GUIDE [D3 p. 7](#) [D6 p. 7](#) [§3.6 p. 11](#) [§6.5 p. 22](#)

In one line. Three separate groups of photos: one to learn from, one to make decisions with, and one to measure the final result honestly.

163 original photos, split by photo and saved in `split.json`

Training	Validation	Test
114 photos (70%)	24 photos	25 photos
the only photos the weights learn from; freshly augmented every epoch	chooses the best epoch and the threshold	locked away; used once, at the very end

Not used: 4,923 pre-made augmented copies of the same 163 photos.
Copies of test photos could end up in training (data leakage), so they are left out.

How the 163 original photos are divided, and what each group is allowed to do.

- **Training set (114 photos):** the only photos the weights ever learn from.
- **Validation set (24 photos):** used to *choose* things, namely the best epoch and the confidence threshold. Because you use it to choose, the final model is slightly tuned to these photos.
- **Test set (25 photos):** locked away until the very end, and used once. Since nothing was chosen with it, its score is an honest estimate of how the model will do on new photos.

Why not choose the best epoch on the test set? You would pick whichever epoch happened to suit those 25 photos, and the test score would flatter the model. The validation set absorbs that bias so the test set stays clean.

IN YOUR PROJECT

The split is made once in Step 4 and saved in `split.json`, so every run and every teammate uses the same three groups. Step 6 uses validation to keep the best epoch, and Step 7 uses validation to pick the threshold, then touches the test set once.

1.11 Data leakage

IN THE GUIDE [D3 p. 7](#) [§6.1 p. 21](#) [Q3 p. 41](#)

In one line. Data leakage is when information from the test photos sneaks into training, so the test score looks better than the model really is.

The classic form is near-duplicates: the same photo, or a slightly changed copy of it, sitting in both the training set and the test set. The model has effectively seen the exam paper in advance. Leakage is dangerous because nothing crashes and nothing looks wrong. The numbers just come out too good.

YOUR DATASET'S TRAP

Besides the 163 originals, the download contains 4,923 augmented images: flips, crops and colour changes of those same originals, about 30 copies each. If you shuffled all 5,086 images and split them, copies of almost every photo would end up on both sides of the split. The model would be tested largely on pictures it had already studied.

Prevention: split first, by original photo; then augment only the training photos, freshly each epoch.

IN YOUR PROJECT

This is decision D3 (p. 7) and the "Use it? No" row in the guide's table in §6.1 (p. 21). It's also viva question Q3 (p. 41). Examiners like this point because it shows you understand *why* the evaluation can be trusted.

1.12 Hyperparameters

IN THE GUIDE [§2.3 p. 8](#) [§7.1 p. 24](#) [Step 0 p. 31](#)

In one line. Hyperparameters are the settings you choose before training (learning rate, epochs, batch size...). Parameters are the weights the model learns.

Parameters (concept 1.2) are learned automatically from the data. Hyperparameters are decided by you and stay fixed during a run. In this project they include `EPOCHS`, `BATCH_SIZE`, `LR`, `WEIGHT_DECAY`, `WARMUP_EPOCHS`, the input size `MIN_SIZE / MAX_SIZE`, `SEED`, and the post-processing settings (score threshold 0.01, NMS threshold 0.5, at most 100 boxes).

Tuning hyperparameters means trying a few values and comparing results on the *validation* set, never the test set. In a comparison study, every model must get the same hyperparameters or the same tuning effort. Otherwise the comparison measures your effort, not the models.

IN YOUR PROJECT

They all sit in the settings cell at the top of the notebook (Step 0), identical for all three models. §2.3 (p. 8) lists what is kept equal, and the note in §7.1 (p. 24) describes the fair way to tune if you have time.

1.13 Randomness, seeds and reproducibility

IN THE GUIDE §3.4 p. 10 §6.5 p. 22 ch. 13 p. 45

In one line. Training uses random numbers. Fixing the **seed** makes the "random" choices the same every run, so results can be repeated.

Training uses randomness in several places: the order the photos are shuffled into batches, which augmentation each photo gets, and the random starting values of the new layers. Computers produce "random" numbers with a formula that starts from a number called the **seed**. The same seed gives the same sequence, so the run repeats exactly. That is **reproducibility**.

GPUs add a little unavoidable variation: they add up thousands of numbers in parallel, in slightly different orders each time, and rounding differs in the last digits. So two runs with the same seed can differ slightly in their final numbers. That's normal, and you mention it under limitations.

IN YOUR PROJECT

`SEED = 42 ; train_model` sets `random.seed`, `np.random.seed` and `torch.manual_seed` before building each model. The split has its own seed, found by the seed search (concept 6.7) and saved in `split.json`.

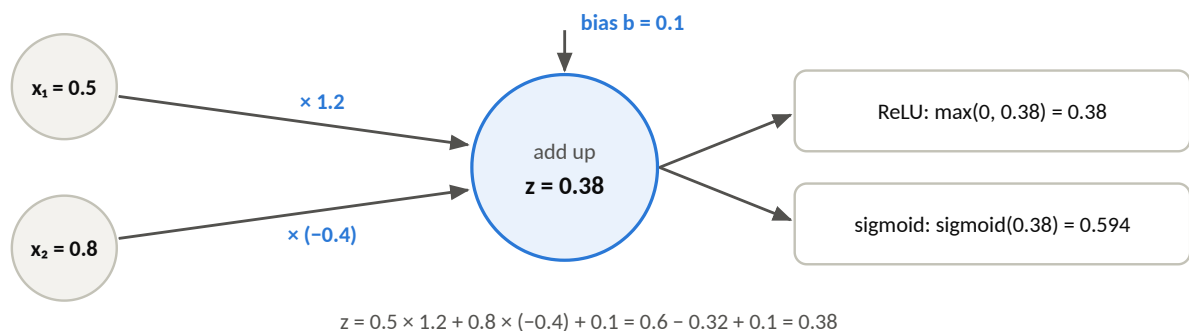
Neural networks

The models in this project are neural networks. The guide mentions layers, ReLU, sigmoid and softmax, BatchNorm, GPUs and mixed precision without stopping to define them. This part does, starting from a single artificial neuron.

2.1 The artificial neuron

IN THE GUIDE §3.1 p. 10

In one line. A neuron multiplies each input by a weight, adds the results together with a bias, and passes the total through an activation function.



One neuron with two inputs, worked through with numbers.

In symbols: $z = w_1x_1 + w_2x_2 + \dots + b$, then output = activation(z). The **weights** say how much each input matters and in which direction: a positive weight means that input pushes the output up, a negative one pushes it down. The **bias** b shifts the result, a bit like a default opinion before looking at the inputs.

WORKED EXAMPLE (THE FIGURE'S NUMBERS)

Inputs 0.5 and 0.8, weights 1.2 and -0.4, bias 0.1:

$$z = 0.5 \times 1.2 + 0.8 \times (-0.4) + 0.1 = 0.6 - 0.32 + 0.1 = \mathbf{0.38}.$$

Through ReLU (concept 2.2) the output is 0.38. Through a sigmoid it's 0.594.

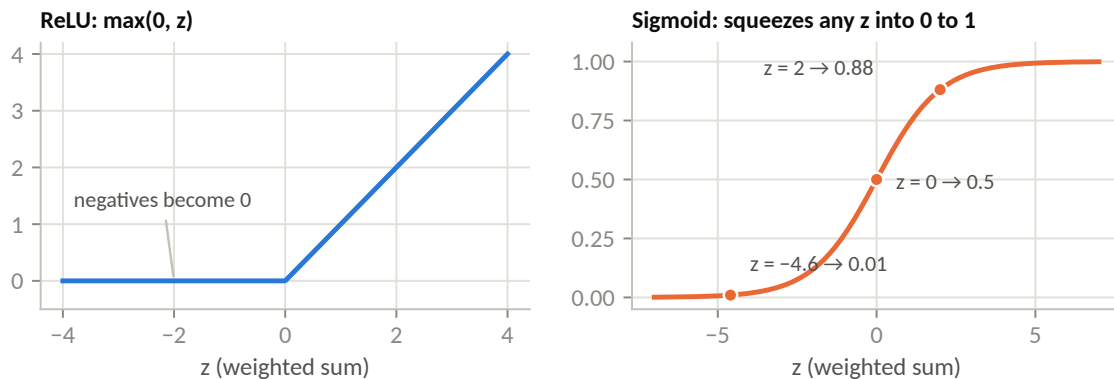
One neuron is very limited. The power comes from millions of them in layers, each learning its own weights. In an image network (Part 3), a "neuron" is one small filter looking at one spot of the image.

2.2 Activation functions: ReLU, sigmoid and softmax

IN THE GUIDE §5.1 p. 16 §5.2 p. 17 §5.3 p. 19 Q21 p. 42

In one line. Activation functions bend the straight-line maths of a neuron. They let networks learn complicated patterns and turn raw scores into probabilities.

Why they're needed. A layer without an activation just multiplies and adds. Stack ten such layers and the result can always be rewritten as a single multiply-and-add, no smarter than one layer. A bend after each layer is what lets depth add power.



- **ReLU** (rectified linear unit): output = $\max(0, z)$. Positive values pass through unchanged; negatives become 0. It's cheap and works well, and it's used after almost every convolution in all three models.
- **Sigmoid**: output = $1 \div (1 + e^{-z})$, which squeezes any number into the range 0 to 1, so it can be read as a probability for a **yes/no** question. $z = 0$ gives 0.5, $z = 2$ gives 0.88, $z = -4.6$ gives 0.01.
- **Softmax**: turns a *list* of scores into probabilities that add up to 1, for a **pick one of several** question. Each score z_i becomes $e^{z_i} \div (\text{sum of all } e^z)$.

WORKED EXAMPLE: SOFTMAX

Scores for three classes: 2.0, 1.0, 0.1. Exponentials: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$, which add up to 11.212. Probabilities: $7.389/11.212 = \mathbf{0.659}$, $2.718/11.212 = \mathbf{0.242}$, $1.105/11.212 = \mathbf{0.099}$. They add up to 1, and the biggest score gets the biggest share.

The raw numbers before sigmoid or softmax are often called **logits** or scores. That's why RetinaNet's final layer is named `cls_logits`.

IN YOUR PROJECT

Faster R-CNN's box head and SSD300 use **softmax over 9 classes** (8 waste classes + background), so exactly one class wins for each box. RetinaNet uses a **separate sigmoid for each class**: each class independently answers "is this me?", and "background" simply means every answer is low. Faster R-CNN's RPN uses a sigmoid for "object or not". This is viva Q21 (p. 42).

2.3 Layers, depth and fully connected layers

IN THE GUIDE §3.1 p. 10 §5.1 p. 16

In one line. A layer is many neurons working side by side on the same input. A deep network stacks many layers, so each can build on what the previous one found.

In a **fully connected** (also "dense" or "linear") layer, every output neuron looks at every input number. These layers have a lot of weights: *inputs* $\times$ *outputs* of them, plus one bias per output. Convolutional layers (Part 3) are the other main type. They look only at small neighbourhoods and reuse the same weights everywhere, which suits images far better.

Deep learning just means learning with networks that have many layers. ResNet-50 has 50. Depth matters because patterns build on patterns: edges combine into corners, corners into shapes, shapes into objects.

WORKED EXAMPLE: FASTER R-CNN'S BOX HEAD

Each proposal arrives as a $256 \times 7 \times 7$ block of features = 12,544 numbers. The head's fully connected layer maps these to 1,024 numbers: $12,544 \times 1,024 \approx$ **12.8 million weights** in that one layer. The final predictor then has two small fully connected layers: $1,024 \rightarrow 9$ class scores, and $1,024 \rightarrow 36$ box numbers (4 for each of the 9 classes).

IN YOUR PROJECT

Those final two small layers form the `FastRCNNPredictor` you replace in `build_faster_rcnn`. It is the only part of Faster R-CNN that starts from scratch.

2.4 Tensors: how PyTorch stores numbers

IN THE GUIDE §4.1 p. 12 §6.6 p. 22 Step 5 p. 33

In one line. A tensor is a grid of numbers with any number of dimensions. Images, boxes, labels and weights are all tensors.

Dimensions	Example in this project	Shape
0-D: a single number	One loss value	[]
1-D: a list	Labels of 2 objects: Bottle = 1, Cigarette Butt = 6	[2]
2-D: a table	2 boxes $\times$ (xmin, ymin, xmax, ymax)	[2, 4]
3-D: an image	colour channels $\times$ height $\times$ width	[3, 768, 1024]
4-D: a batch	4 same-size images	[4, 3, 768, 1024]

The **shape** lists the size of each dimension. The **dtype** says what kind of number is stored: `uint8` is whole numbers 0–255 (how photos are stored), `float32` is decimals used for the maths, `float16` is half-precision decimals (concept 2.8), and `int64` is whole numbers such as class labels. Each tensor also lives on a **device**, either CPU memory or GPU memory. `.to(DEVICE)` moves it, and two tensors must be on the same device to be combined.

WORKED EXAMPLE

A 1024×768 colour photo becomes a `uint8` tensor of shape `[3, 768, 1024]`: $3 \times 768 \times 1024 = 2,359,296$ numbers between 0 and 255. `ToDtype(float32, scale=True)` divides every one by 255, so they run from 0 to 1, which is what the detectors expect.

IN YOUR PROJECT

In `TrashDataset`, `TF.pil_to_tensor` makes the `[3, H, W]` image, `target["boxes"]` is a `float32` tensor of shape `[N, 4]`, and `target["labels"]` is `int64` of shape `[N]`. The detectors take a *list* of 3-D image tensors rather than one 4-D batch, because the photos can have different sizes (§7.2, p. 24).

2.5 Normalisation layers: BatchNorm and GroupNorm

IN THE GUIDE §5.1 p. 16 §5.2 p. 17 Q22 p. 42

In one line. Normalisation layers rescale the numbers flowing through the network to a standard range (average 0, spread 1), which keeps training stable and fast.

As training changes the early layers, the typical size of the numbers they pass on keeps drifting, and the later layers are always chasing a moving target. A normalisation layer fixes this: for each channel it subtracts the average and divides by the standard deviation. Then it applies a learned scale and shift, so the network can still choose the range it prefers.

WORKED EXAMPLE

Values 2, 4, 6, 8: the average is 5 and the standard deviation is 2.236. Normalised: $(2 - 5)/2.236 = -1.34$, then -0.45 , 0.45 , 1.34 . They are centred on 0 with spread 1, whatever the original scale was.

- **BatchNorm** computes the average and spread over the whole batch (all images and positions). During training it uses the current batch's numbers and keeps a running average. During evaluation it uses that stored running average instead. This is one of the reasons `model.train()` and `model.eval()` matter (concept 7.2).
- **GroupNorm** computes them inside each single image, over groups of channels. It doesn't depend on the batch size, so it behaves well with small batches like our 4.

IN YOUR PROJECT

The "v2" versions of our models add these layers: BatchNorm in Faster R-CNN v2's box head, and GroupNorm (32 groups) in RetinaNet v2's subnets (§5.1 (p. 16), §5.2 (p. 17), viva Q22, p. 42).

2.6 Initialisation: where the weights start

IN THE GUIDE §5.2 p. 17 Step 5 p. 33 Q15 p. 42

In one line. Before training, every weight needs a starting value: pretrained values where possible, and small random numbers for brand-new layers.

New layers start with small random numbers, for example drawn from a bell curve with standard deviation 0.01. They must be random, because if every weight started equal, every neuron would learn the same thing. **Pretrained layers** start from values learned on COCO (concept 3.8), a huge head start over random.

A clever bias trick (RetinaNet). RetinaNet scores about 160,000 anchors per photo, and almost all of them are background. If the new class layer started out guessing 50% "object" for each, the first iterations would produce an enormous loss and wild gradients. So its bias is set to make every starting probability 1%:

WORKED EXAMPLE

$b = -\log((1 - 0.01) / 0.01) = -\log(99) = -4.595$. Check: $\text{sigmoid}(-4.595) = 1 \div (1 + e^{4.595}) = 1 \div 100 = 0.01$. Every class starts at 1%, a sensible "probably not" for a background-heavy world.

IN YOUR PROJECT

`build_retinanet` does exactly this: `torch.nn.init.normal_(..., std=0.01)` for the weights and `constant_(bias, -math.log((1 - 0.01) / 0.01))` for the bias. SSD300's new head uses torchvision's built-in "Xavier" random start, and Faster R-CNN's new predictor uses PyTorch's default for linear layers. Viva Q15 (p. 42).

2.7 GPUs, CUDA and GPU memory

IN THE GUIDE §7.4 p. 25 Step 0 p. 31 ch. 13 p. 45

In one line. A GPU does thousands of simple calculations at the same time, which is exactly the kind of work neural networks need.

A **CPU** has a few very capable cores, good at varied tasks one after another. A **GPU** has thousands of small cores that do the same arithmetic on different numbers simultaneously. A detector's forward pass is billions of multiply-and-add operations (concept 8.8), all independent of each other. It's the perfect job for a GPU, typically tens of times faster than a laptop CPU.

CUDA is NVIDIA's software for programming its GPUs, and PyTorch calls the GPU device `"cuda"`. Two practical consequences:

- **GPU memory (VRAM) is limited.** Colab's T4 has 16 GB. During training it must hold the weights, their gradients, the optimizer's running averages, and every layer's outputs for every image in the batch (backpropagation needs them). Too many or too large images give "CUDA out of memory".
- **The GPU works asynchronously.** Python hands work to the GPU and moves on immediately, and the GPU finishes a moment later. This matters when timing (concept 8.8).

IN YOUR PROJECT

`DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")` picks the GPU when Colab provides one. The first check in Step 0 is that it prints `device: cuda`. The out-of-memory fixes in ch. 13 (p. 45) reduce what must fit: batch 2, or smaller input images.

2.8 Mixed precision (AMP)

IN THE GUIDE `$3.4 p. 10` `$7.1 p. 24` `Q28 p. 43` `ch. 13 p. 45`

In one line. Do most calculations with 16-bit numbers instead of 32-bit ones: roughly twice as fast on a T4 and half the memory, with the same results in practice.

Computers store decimals as "floating-point" numbers. **float32** keeps about 7 significant digits and can hold huge or tiny values. **float16** keeps about 3 digits and can't go above 65,504, and very small values (below about 0.00006) lose precision or become exactly zero. The T4 has special "tensor cores" that multiply float16 numbers much faster than float32.

Mixed precision uses each where it's safe. PyTorch's **autocast** runs the heavy convolutions and matrix multiplications in float16, and keeps sensitive steps (losses, sums, normalisation) in float32. One problem remains: many gradients are tiny, like 0.0000001, and would round to zero in float16 ("underflow"). The **gradient scaler** fixes this:

1. multiply the loss by a large factor (e.g. 65,536) before backpropagation, so every gradient is 65,536 times bigger and safely representable;
2. divide the gradients back down ("unscale") before the update;
3. if any gradient came out as infinity or `nan`, skip that update and try a smaller factor next time.

IN YOUR PROJECT

`USE_AMP` is switched on automatically when a GPU is present. You'll see `torch.autocast(DEVICE.type, enabled=USE_AMP)`, `scaler = torch.amp.GradScaler("cuda", ...)` and `scaler.unscale_(opt)` in Step 6. If you ever see `nan`, the first fix in ch. 13 (p. 45) is `USE_AMP = False`. Viva Q28 (p. 43).

2.9 Exploding gradients, clipping and `nan`

IN THE GUIDE `$3.4 p. 10` `$7.3 p. 25` `ch. 13 p. 45`

In one line. If the gradients suddenly become enormous, clipping shrinks them to a safe size before the weights are updated.

Now and then one batch produces huge gradients, for example an unusual augmented image. One giant step can throw the weights far from anything sensible (the "too big step" in concept 1.4). The overall size of all the gradients together is measured by their **norm**: square every gradient number, add them all, take the square root. **Gradient clipping** says: if the norm is above a limit, scale every gradient down by the same factor so the norm equals the limit. The direction is kept and only the size is capped.

WORKED EXAMPLE (LIMIT 10)

Gradients (6, 8): norm $\sqrt{36 + 64} = 10$, so they are left alone. Gradients (30, 40): norm 50, which is too big, so each is multiplied by $10/50$, giving (6, 8). Same direction, safe size.

`nan` means "not a number". It appears after impossible arithmetic such as infinity minus infinity, and once one weight is `nan` it spreads to everything it touches. The training loop checks the loss every iteration and stops with a clear message rather than silently training garbage.

IN YOUR PROJECT

```
torch.nn.utils.clip_grad_norm_(params, max_norm=10.0) and if not torch.isfinite(loss): raise
RuntimeError(...) in Step 6. The "Unstable" row in §7.3 (p. 25) and the "Loss becomes nan" row in ch. 13 (p. 45) are
what you'd see if clipping and the other safeguards weren't enough.
```

Seeing images: convolutional networks

Guide chapters 3 to 5 talk about backbones, feature maps, strides, ResNet-50, VGG-16, FPNs and transfer learning. This part starts from pixels and builds up to each of those ideas.

3.1 Digital images, pixels and coordinates

IN THE GUIDE §4.2 p. 12 §5.3 p. 19 §6.3 p. 21

In one line. A photo is a grid of pixels, and each pixel is three numbers (red, green, blue) from 0 to 255.

- **Resolution** is width $\times$ height in pixels. A 1600×1200 photo has 1.92 million pixels.
- **Channels:** every pixel mixes red, green and blue light. (255, 0, 0) is pure red, (0, 0, 0) is black and (255, 255, 255) is white. As a tensor, a photo is $3 \times \text{height} \times \text{width}$ (concept 2.4).
- **Coordinates:** the origin (0, 0) is the *top-left* corner. x grows to the right and y grows *downwards*, unlike school graphs. A box written as (xmin, ymin, xmax, ymax) is its top-left and bottom-right corners in these coordinates.
- **Aspect ratio** is width $\div$ height: $1600/1200 = 4/3$. Resizing can keep it ($1600 \times 1200 \rightarrow 1024 \times 768$) or ignore it and squash ($\rightarrow 300 \times 300$).

IN YOUR PROJECT

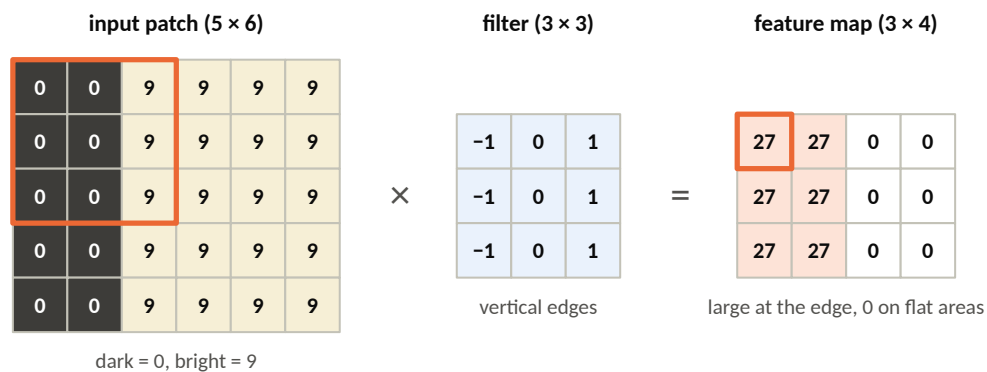
The clean copy keeps each photo's aspect ratio, with the long side at most 1,024 px (§6.3, p. 21). SSD300 always squashes to 300×300 , which is part of why it struggles with small objects (§5.3, p. 19).

3.2 Convolution: sliding pattern detectors

IN THE GUIDE §3.1 p. 10

In one line. A small grid of weights (a **filter**) slides across the image. At each position it multiplies and adds, producing a map of where its pattern appears.

Step by step: lay a 3×3 filter over the top-left 3×3 patch of pixels. Multiply the 9 pairs of numbers and add them up. That sum is one output number. Slide one pixel to the right and do it again, then carry on across and down the whole image. The grid of results is called a **feature map**.



Orange square: each row gives $(-1 \times 0) + (0 \times 0) + (1 \times 9) = 9$; three rows give 27.

A vertical-edge filter on a patch that is dark on the left and bright on the right. The output is large where brightness jumps and zero where the image is flat.

WORKED EXAMPLE (THE FIGURE)

Where the filter covers the two dark columns and one bright column, each row contributes $(-1 \times 0) + (0 \times 0) + (1 \times 9) = 9$, and three rows give **27**. Where it covers dark, bright, bright: $(-1 \times 0) + (0 \times 9) + (1 \times 9) = 9$ per row, so again **27**. On the all-bright part: $(-1 \times 9) + (0 \times 9) + (1 \times 9) = 0$. The filter "lights up" exactly at the edge.

Key facts:

- **The filters are learned.** Nobody writes the edge filter above. Training discovers useful filters on its own (concept 1.4).
- **Colour and depth.** On a colour image a filter covers all three channels: $3 \times 3 \times 3 = 27$ weights plus a bias. Deeper in the network it covers all the channels of the previous layer.
- **Many filters per layer.** A layer with 64 filters produces 64 feature maps, called **channels**, each a map of a different pattern.
- **Weight sharing.** The same filter is used at every position, so a bottle cap is recognised wherever it is in the photo. This also needs far fewer weights than a fully connected layer (concept 2.3).

A network built mainly from these layers is a **convolutional neural network (CNN)**. All three of our detectors are CNNs.

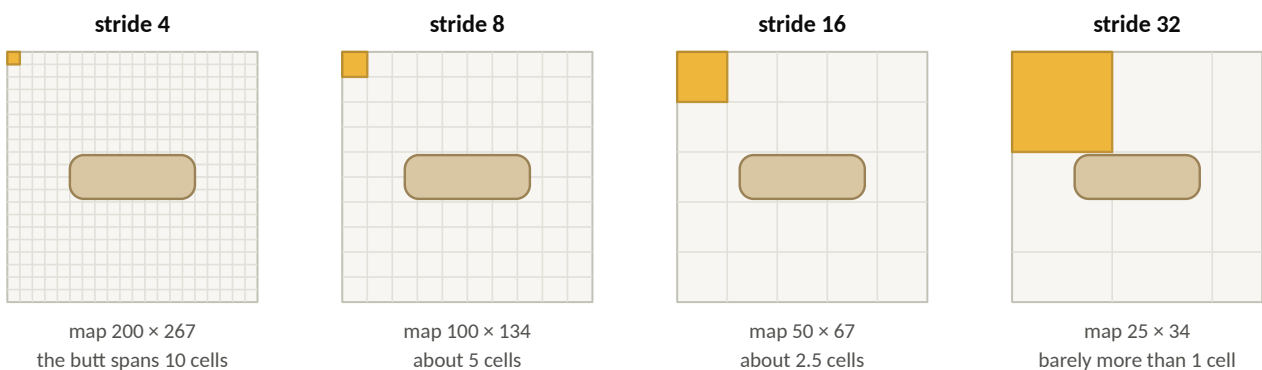
3.3 Stride, padding, pooling and downsampling

IN THE GUIDE §3.1 p. 10 §4.8 p. 14 §5.1 p. 16

In one line. Moving the filter several pixels at a time (**stride**) or **pooling** shrinks the feature map. **Padding** handles the border.

- **Stride 2:** the filter jumps 2 pixels at a time, so the output is half as wide and half as tall.
- **Padding:** a border of zeros around the input, so the filter can be centred on edge pixels too. With a 3×3 filter, padding 1 and stride 1, the output is exactly the same size as the input.
- **Max pooling** (2×2): keep only the largest value in each 2×2 block. This also halves the size but keeps the strongest responses.

Why shrink at all? Each cell of a smaller map summarises a bigger area of the photo, so the network can recognise large things, and the computation drops a lot. The price is detail: a small object may shrink to less than one cell.



Each panel shows the same 80×80 px corner of an 800×1067 input. Brown: a 40 px cigarette butt. Yellow: one cell of the map.

How an 800×1067 input shrinks at each stride. The yellow square shows the size of one cell on each map, relative to a 40-pixel cigarette butt.

WORKED EXAMPLE (800 × 1067 INPUT)

Stride 4 gives 200×267 , stride 8 gives 100×134 , stride 16 gives 50×67 , and stride 32 gives 25×34 : the four ResNet-50 stages. The FPN adds stride 64 (13×17), and RetinaNet adds stride 128 (7×9). (torchvision first pads the width to 1,088, a multiple of 32, so the real maps are 272, 136 ... cells wide. Same idea.)

IN YOUR PROJECT

This is the callout in §3.1 (p. 10): at stride 32 one cell covers 32×32 pixels, larger than a 20-px cigarette butt. `padding=1` in RetinaNet's `cls_logits` conv keeps the map size unchanged, so there is exactly one prediction per position.

3.4 Feature maps, feature vectors and receptive fields

IN THE GUIDE §3.1 p. 10 §4.8 p. 14

In one line. A feature map is the grid a layer outputs. Each cell holds a list of numbers describing one patch of the photo, and cells in deeper layers describe bigger patches.

A feature map has shape channels × height × width. For an 800×1067 photo, ResNet-50's last stage gives $2048 \times 25 \times 34$. The 2,048 numbers at one cell form that cell's **feature vector**: the network's description of the patch, something like "how much curved edge, metal shine, bottle-neck shape...", though the meanings are learned, not labelled.

The **receptive field** of a cell is the area of the original photo that can influence it. Every 3×3 filter and every stride makes it bigger. So early layers see a few pixels (edges, colour blobs), middle layers see textures and parts (a can's rim, a bottle's neck), and deep layers see whole objects. This build-up from simple to complex is the **feature hierarchy**.

EVERYDAY COMPARISON

A street map at different zoom levels. Zoomed in, you see individual houses but only a few streets. Zoomed out, each square covers a whole neighbourhood, but individual houses disappear. A detector needs both kinds of view, and that's what the FPN (concept 4.12) provides.

3.5 1×1 and 3×3 convolutions as building blocks

IN THE GUIDE §4.8 p. 14 §5.3 p. 19 Step 5 p. 33

In one line. A 1×1 conv mixes the channels at each position, which changes how many channels there are. A 3×3 conv also looks at the neighbours and is often used to make predictions.

- **1×1 convolution.** At each cell it takes a weighted combination of all input channels. It's like a tiny fully connected layer applied separately at every position. The FPN uses it to turn the backbone's 512-, 1,024- or 2,048-channel maps into a uniform 256 channels. These are the "lateral" connections of concept 4.12.
- **3×3 convolution as a predictor.** Give it as many output channels as there are numbers to predict at each position. Then at every cell it outputs exactly that list: for example, a score for every class of every anchor.

WORKED EXAMPLE: RETINANET'S NEW LAST LAYER

256 input channels → 9 anchors × 9 classes = 81 output channels. A 3×3 conv therefore has $3 \times 3 \times 256 \times 81 = 186,624$ weights plus 81 biases = **186,705 parameters**. At every position of every pyramid level, it outputs 81 scores.

IN YOUR PROJECT

`torch.nn.Conv2d(head.cls_logits.in_channels, head.num_anchors * num_classes, kernel_size=3, padding=1)` in `build_retinanet` is exactly this layer. SSD300's `SSDClassificationHead` is six such 3×3 predictors, one per feature map.

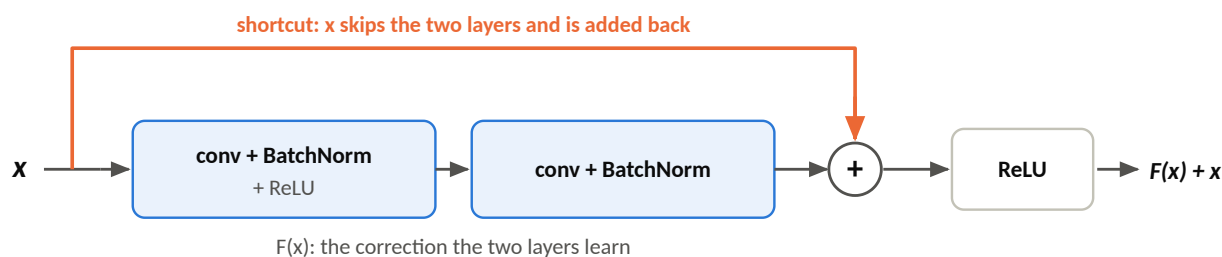
3.6 Backbones: VGG-16 and ResNet-50

IN THE GUIDE §3.2 p. 10 §3.3 p. 10 §5.1 p. 16 §5.3 p. 19

In one line. Backbones are well-known image networks, pretrained on millions of photos, that detectors use as their feature extractors.

VGG-16 (2014) is a plain stack of 3×3 convolutions with pooling in between: 13 conv layers plus 3 fully connected layers in its original classification form, so 16 layers with weights. It is simple and heavy. SSD300 uses its conv part, with the fully connected layers converted into convolutions, plus extra layers.

ResNet-50 (2015) has 50 layers built from **residual blocks**. Each block computes a correction $F(x)$ and *adds its own input back*: $\text{output} = F(x) + x$. That "skip connection" is a shortcut. If a block has nothing useful to add, it can simply pass its input through, and during backpropagation the gradient flows back through the shortcuts without fading. Before ResNet, making networks deeper often made them worse. Residual blocks fixed that.



A residual block: the input skips around two layers and is added back at the end.

ResNet-50 is organised as a "stem" (the first convolution) plus four stages, `layer1` to `layer4`, at strides 4, 8, 16 and 32. That's the "5 stages" counted in §3.3 (p. 10). Both backbones were first trained on **ImageNet**, 1.28 million photos in 1,000 categories, to learn general visual features.

IN YOUR PROJECT

Faster R-CNN and RetinaNet use ResNet-50 plus an FPN; SSD300 uses VGG-16 (§3.2, p. 10; §5.4, p. 19).

3.7 Backbone, neck and head

IN THE GUIDE §3.2 p. 10 §5.1 p. 16 §5.2 p. 17 §5.3 p. 19

In one line. A detector has three parts: a **backbone** that turns pixels into features, a **neck** that combines features of different sizes, and **heads** that make the final predictions.

An analogy: the backbone is the eyes and visual memory, recognising edges, textures and shapes. The neck is attention across distances, combining the close-up and zoomed-out views. The heads are the decision-makers: "there's a Can here, and its box is exactly there."

Part	In our models	Starts from...
Backbone	ResNet-50 or VGG-16	COCO-trained weights; earliest layers frozen (concept 3.8)
Neck	FPN (Faster R-CNN, RetinaNet); extra layers (SSD300)	COCO-trained weights, fine-tuned
Heads	RPN + box head; class/box subnets; per-map predictors	COCO-trained weights, <i>except</i> the final class layer, which is new and random

IN YOUR PROJECT

Only the very end of the head changes for our 8 classes. Everything else keeps its COCO knowledge and is gently fine-tuned. That's the whole idea behind the three `build_*` functions in Step 5.

3.8 Transfer learning, fine-tuning and freezing

IN THE GUIDE [D1 p. 6](#) [§3.3 p. 10](#) [Step 5 p. 33](#) [Q20 p. 42](#) [Q23 p. 42](#)

In one line. Start from a model already trained on a big dataset, replace its final layer for your own classes, and train it gently on your data.

A trained chef learning a new cuisine picks it up far faster than someone who has never cooked: knife skills, heat control and tasting all carry over. Networks are the same. Edges, textures, shapes, even "bottle-ness", are useful in any photo. Only the final decision ("which of *our* 8 classes?") is specific to this task. Learning general features from scratch needs millions of photos. We have 114, which is plenty for fine-tuning.

Our starting weights went through two rounds: ImageNet (classification) and then **COCO** (detection: 118,000 photos of 80 everyday object types, including *bottle* and *cup*). COCO's labels are numbered 1 to 90 with some unused numbers, plus 0 for background. That's why the original output layer has **91** slots.

1. **Load** the COCO-trained detector.
2. **Replace** the final class layer: 91 → 9 outputs (8 classes + background). The new layer starts random.
3. **Freeze** the earliest backbone layers by setting `requires_grad = False`, so their weights never change. They detect generic edges that don't need to change, and freezing saves memory. torchvision's default is to train the last 3 of ResNet-50's 5 stages and the last 4 of VGG-16's 5 blocks.
4. **Fine-tune** everything else with a small learning rate, so the pretrained knowledge adapts rather than being overwritten.

IN YOUR PROJECT

`weights=...` `DEFAULT` loads the COCO weights, the three builders do step 2, and `params = [p for p in model.parameters() if p.requires_grad]` hands only the trainable weights to the optimizer. These are decision D1 (p. 6) and viva Q20 (p. 42) and Q23 (p. 42).

3.9 Upsampling and bilinear interpolation

IN THE GUIDE [§4.8 p. 14](#) [§5.1 p. 16](#) [§6.3 p. 21](#)

In one line. These are ways to estimate values *between* grid points, used to enlarge feature maps, to read features at fractional positions and to resize photos.

- **Nearest neighbour:** use the value of the closest grid point. Fast, but blocky.

- **Bilinear interpolation:** blend the four surrounding values, giving more weight to the closer ones. Smooth and more accurate.

WORKED EXAMPLE: BILINEAR

Four grid values: 10 (top-left), 20 (top-right), 30 (bottom-left), 40 (bottom-right). We want the value a quarter of the way across and halfway down. Along the top edge, a quarter of the way from 10 to 20 is 12.5. Along the bottom edge, a quarter of the way from 30 to 40 is 32.5. Halfway between those two is **22.5**.

Where it appears: the FPN enlarges deep maps by 2× before adding them to shallower ones (torchvision uses nearest neighbour there). **RoIAlign** (concept [5.1](#)) reads features at exact fractional box positions using bilinear sampling, instead of rounding to the nearest cell. And the clean copy shrinks photos with `Image.BILINEAR`.

From classification to object detection

Guide chapter 4 introduces the building blocks that all three detectors share: boxes, IoU, anchors, box regression, the flood of background, NMS and the FPN. This part explains each one from the ground up, with every number worked through. After it, the three models in Part 5 are just different ways of putting the same blocks together.

4.1 What a detector outputs

IN THE GUIDE §4.1 p. 12 §2.1 p. 6 Q1 p. 41 Q49 p. 44

In one line. A detector finds every object in a photo and returns, for each one, a box, a class and a confidence score.

Three computer-vision tasks are easy to mix up:

- **Classification** gives one answer for the whole photo: "this photo contains a bottle". It can't say how many bottles there are, or where.
- **Object detection** gives a list of objects, each with a **bounding box** (the rectangle around it), a **class** (what it is) and a **confidence score** between 0 and 1 (how sure the model is). The length of the list changes from photo to photo: zero objects, three, or thirty.
- **Segmentation** labels every single pixel. It gives exact outlines, but it needs outline labels, which this dataset doesn't have and the brief doesn't ask for.

Detectors also need a name for "nothing here". In torchvision that is the **background** class, and it always has id 0. Our eight classes are numbered 1 to 8 in alphabetical order: 1 Bottle, 2 Bottle Cap, 3 Can, 4 Cardboard, 5 Cigarette Box, 6 Cigarette Butt, 7 Cup, 8 Plastic. So the models need **9** class slots, not 8.

WORKED EXAMPLE: READING THE OUTPUT IN GUIDE §4.1

The first detection has box [112.3, 40.8, 198.0, 260.1], label 1 and score 0.97: a Bottle, $198.0 - 112.3 = 85.7$ px wide and $260.1 - 40.8 = 219.3$ px tall (tall and thin, like a standing bottle), and the model is very sure. The second has box [530.2, 610.5, 571.9, 640.0], label 6 and score 0.64: a Cigarette Butt of about 42×30 px, detected with much less confidence. That's typical, because small objects are harder.

IN YOUR PROJECT

`CLASS_TO_ID = {c: i + 1 ...}` and `NUM_CLASSES = len(CLASSES) + 1` in Step 3 set up the numbering. `predict()` in Step 6 drops any box labelled 0 (`keep = p["labels"] > 0`), because "background" is never a real detection. This is viva Q1 (p. 41) and Q49 (p. 44).

4.2 Cross-entropy: the loss for choosing a class

IN THE GUIDE §5.1 p. 16 §5.3 p. 19 §2.3 p. 8 Q29 p. 43

In one line. Cross-entropy looks at the probability the model gave to the *correct* class and takes minus its logarithm. Confident and right costs almost nothing; confident and wrong costs a lot.

After a softmax (concept 2.2) the model has spread 100% of its belief over the 9 classes. Cross-entropy ignores everything except the share given to the right answer, p , and computes:

$$\text{CE} = -\log(p_{\text{correct class}})$$

Here "log" is the natural logarithm (the $\ln$ key on a calculator). The logarithm of 1 is 0, so a perfect answer costs nothing. As p gets close to 0, $-\log p$ shoots up, so a confident wrong answer is punished hard:

Probability given to the correct class	0.99	0.9	0.5	0.1	0.01
Cross-entropy, $-\log p$	0.010	0.105	0.693	2.303	4.605

WORKED EXAMPLE

Take the softmax from concept 2.2: scores 2.0, 1.0, 0.1 became probabilities 0.659, 0.242, 0.099. If the first class is correct, the loss is $-\log 0.659 = \mathbf{0.417}$. If the third class is correct, the model gave it only 0.099, and the loss is $-\log 0.099 \approx \mathbf{2.31}$, more than five times bigger. Gradient descent (concept 1.4) then pushes hardest on the examples where the model is most wrong.

In a detector, the "examples" are candidate boxes rather than photos. Each candidate's correct class is the class of the object it was matched to, or background if it matched nothing (concept 4.7).

IN YOUR PROJECT

`loss_classifier` in Faster R-CNN's log and `classification` in SSD300's log are cross-entropy over the 9 classes. Because the three models add up different losses on different candidates, their loss values can't be compared with each other: viva Q29 (p. 43).

4.3 Yes/no scores: sigmoid and binary cross-entropy

IN THE GUIDE §5.1 p. 16 §5.2 p. 17 §5.4 p. 19 Q21 p. 42

In one line. For a yes/no question the model outputs a single probability through a sigmoid, and binary cross-entropy scores it: $-\log p$ if the true answer is yes, $-\log(1 - p)$ if it is no.

Some questions inside a detector are yes/no questions. "Is there any object in this anchor?" is one, asked by Faster R-CNN's first stage. RetinaNet asks one yes/no question per class for every anchor: "is this a Bottle?", "is this a Can?", and so on. Each answer comes from its own sigmoid (concept 2.2), and the loss is **binary cross-entropy** (BCE), with $y = 1$ for yes and $y = 0$ for no:

$$\text{BCE} = -[y \log p + (1 - y) \log(1 - p)]$$

Only one of the two terms is ever switched on. It is really the cross-entropy of concept 4.2, applied to the two options "yes" and "no".

WORKED EXAMPLE

An anchor sits on a real object ($y = 1$). If the model says $p = 0.8$, the loss is $-\log 0.8 = \mathbf{0.223}$. If it says $p = 0.2$, the loss is $-\log 0.2 = \mathbf{1.609}$. An anchor on bare pavement ($y = 0$) with $p = 0.1$ costs $-\log 0.9 = 0.105$.

Softmax or sigmoid? With a softmax the classes compete: the probabilities add up to 1, exactly one class wins, and background is one of the classes. With a sigmoid per class, each class decides on its own, and background simply means that every class said "no". Both work. RetinaNet's focal loss (concept 5.2) is built on the sigmoid version.

IN YOUR PROJECT

`loss_objectness` in Faster R-CNN's log is BCE on "object or not". RetinaNet's `classification` loss is a re-weighted BCE for every class of every anchor. Faster R-CNN's second stage and SSD300 use a softmax instead. The guide's table in §5.4 (p. 19) has a "Class scores" row for exactly this, and it's viva Q21 (p. 42).

4.4 Bounding boxes and the three label formats

IN THE GUIDE §4.2 p. 12 §6.2 p. 21 §2.6 p. 9 Q6 p. 41 Q7 p. 41

In one line. A box is four numbers. VOC stores the two corners in pixels, COCO stores the top-left corner plus the width and height, and YOLO stores the centre and the size as fractions of the image.

A bounding box is an **axis-aligned** rectangle: its sides are parallel to the edges of the photo, so it can't tilt. A bottle lying diagonally gets a box with empty corners, and that's normal. Remember the coordinate system from concept 3.1: the origin is the top-left corner and *y* grows *downwards*. Four numbers fix such a rectangle, but there are several ways to choose them:

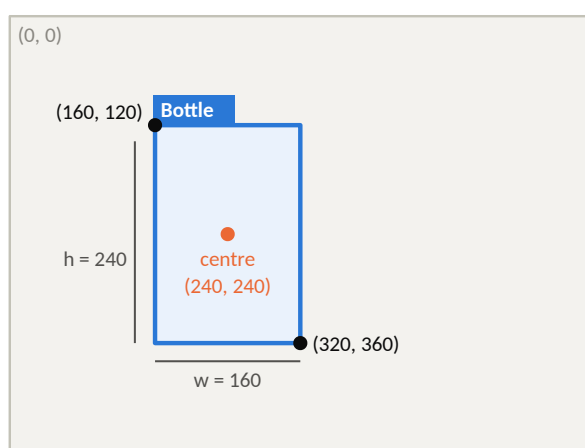


image: 640 × 480 pixels

Pascal VOC (.xml, one file per image)

xmin ymin xmax ymax, in pixels

160 120 320 360

COCO (.json, one file for all images)

x y width height, in pixels

160 120 160 240

YOLO (.txt, one file per image)

class cx cy w h, as fractions 0–1

0 0.375 0.500 0.250 0.500

The same figure as guide §4.2 (p. 12): one Bottle box in a 640 × 480 image, written in all three formats.

WORKED EXAMPLE: CONVERTING THE BOX IN THE FIGURE (VIVA Q7)

VOC (xmin, ymin, xmax, ymax) = (160, 120, 320, 360).

To COCO (x, y, width, height): x = 160, y = 120, width = 320 – 160 = 160, height = 360 – 120 = 240, giving (160, 120, 160, 240).

To YOLO (centre and size as fractions): cx = (160 + 320) ÷ 2 ÷ 640 = 240 ÷ 640 = 0.375; cy = (120 + 360) ÷ 2 ÷ 480 = 240 ÷ 480 = 0.5; w = 160 ÷ 640 = 0.25; h = 240 ÷ 480 = 0.5.

Back to VOC: xmin = (0.375 – 0.25 ÷ 2) × 640 = 0.25 × 640 = 160. The other three corners come out the same way.

Why do the formats exist? Corners are convenient for computing overlaps. Width and height are convenient for computing areas. Fractions don't change when the image is resized, which suits YOLO's training code. Pixel coordinates, on the other hand, must be scaled whenever the image is resized (concept 6.4).

Three classic bugs. Reading a COCO box as if it were VOC turns the width into "xmax", so boxes shrink towards the top-left, and some end up with zero or negative width. Forgetting to scale boxes when resizing puts them in the wrong place. And class ids can be off by one: YOLO files count classes from 0 (Bottle = 0), but torchvision counts from 1 because 0 means background. These are pitfalls 4 and 5 in §2.6 (p. 9).

IN YOUR PROJECT

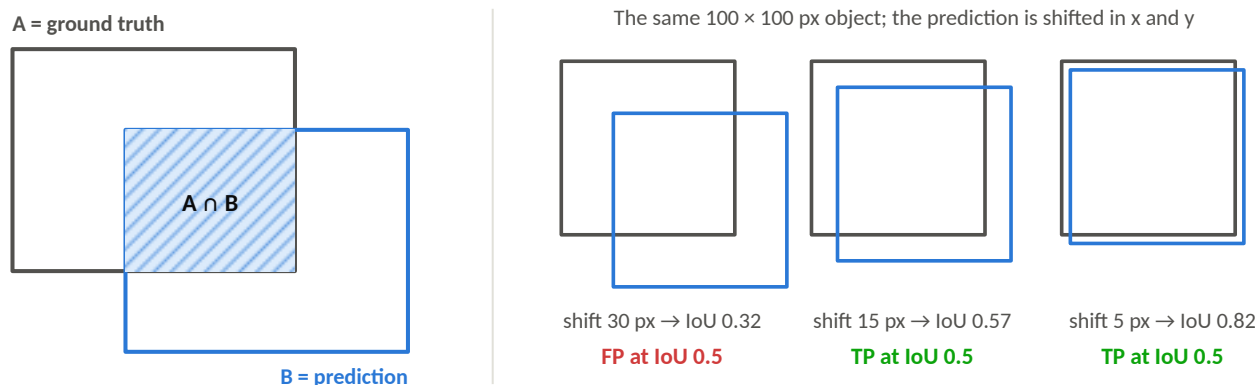
We read the VOC .xml files (`read_voc` , Step 2) because torchvision uses exactly the same layout. That's why the dataset code wraps the boxes as `tv_tensors.BoundingBoxes` (`boxes` , `format="XYXY"` , ...) : XYXY means xmin, ymin, xmax, ymax. Viva Q6 (p. 41) and Q7 (p. 41).

4.5 Intersection over Union (IoU)

IN THE GUIDE §4.3 p. 12 §8.1 p. 26 §9.1 p. 29 Q31 p. 43

In one line. IoU is the area two boxes share divided by the total area they cover: 1 means identical boxes, 0 means no overlap at all.

Detection needs a single number for "how well does this box fit that one?" IoU is that number. It divides the **intersection** (the area inside both boxes) by the **union** (the area inside at least one of them). The union is the two areas added together *minus* the overlap, because otherwise the shared part would be counted twice.



The same figure as guide §4.3 (p. 12). Left: the overlap (hatched) and the union. Right: the IoU of a 100 × 100 px object's prediction as it drifts by 30, 15 and 5 px.

How to compute the overlap of two boxes. The overlap is itself a rectangle. Its left edge is the larger of the two xmins, its right edge the smaller of the two xmaxs, its top edge the larger of the two ymins, and its bottom edge the smaller of the two ymaxs. If $\text{right} \leq \text{left}$ or $\text{bottom} \leq \text{top}$, the boxes don't overlap and IoU is 0.

WORKED EXAMPLE (GUIDE §4.3, VIVA Q31)

Ground truth $A = (100, 100, 300, 300)$, prediction $B = (150, 150, 350, 350)$.

Overlap: $\text{left} = \max(100, 150) = 150$, $\text{right} = \min(300, 350) = 300$, $\text{top} = 150$, $\text{bottom} = 300$. So the overlap is $150 \times 150 = 22,500 \text{ px}^2$.

Areas: $A = 200 \times 200 = 40,000 \text{ px}^2$, and B is the same. Union = $40,000 + 40,000 - 22,500 = 57,500 \text{ px}^2$.

$\text{IoU} = 22,500 \div 57,500 = 0.39$. Below the usual 0.5, so this prediction counts as wrong even though it clearly found the object.

Why small objects are harder. In the figure, a 5-px slip in both directions still leaves the 100 × 100 px object at IoU 0.82. On a 20 × 20 px cigarette butt the very same slip gives $(15 \times 15) \div (400 + 400 - 225) = 0.39$, a miss. Small objects need pixel-perfect boxes, which is why they suffer most at strict IoU thresholds (concept 8.5).

IoU turns up four times in this project: to decide which anchors are responsible for which object during training (concept 4.7), to remove duplicate boxes (concept 4.11), as a box loss (GIoU, concept 4.9), and to decide whether a prediction counts as correct (concept 8.1).

IN YOUR PROJECT

`box_iou` from `torchvision.ops` computes a whole table at once: the IoU of every prediction with every ground-truth box in a photo. `match_image` (Step 7) and `explain_errors` (Step 8) both start from that table.

4.6 Anchors: a fixed set of candidate boxes

IN THE GUIDE §4.4 p. 13 §5.1 p. 16 §5.2 p. 17 §5.3 p. 19 Q11 p. 42

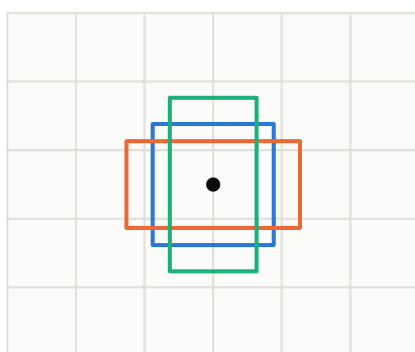
In one line. Anchors are reference boxes of a few sizes and shapes, placed at every position of a feature map. The network answers the same questions for every one of them.

The problem anchors solve. A network always outputs the same number of values, but a photo might contain no objects or twenty. So the detector is given a long, fixed list of "slots", and for each slot it predicts: *is there an object here, which class is it, and how should this box be moved and stretched to fit it?* Almost every slot answers "nothing here". These slots are the **anchors**. SSD calls them **default boxes**; it's the same idea.

An anchor is defined by three things:

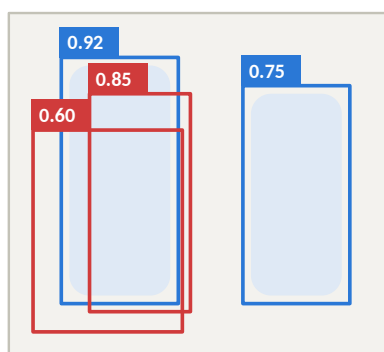
- a **position**: one for every cell of a feature map, so they are spaced one stride apart (every 8 px on a stride-8 map);
- a **size**: small on the fine, high-resolution maps and large on the coarse ones;
- a **shape** (aspect ratio): usually 1:1, 1:2 and 2:1. torchvision keeps the area the same across shapes, so a "size 64" anchor is 64×64 px square, 45×91 px tall or 91×45 px wide.

Anchors: 3 shapes at every grid position



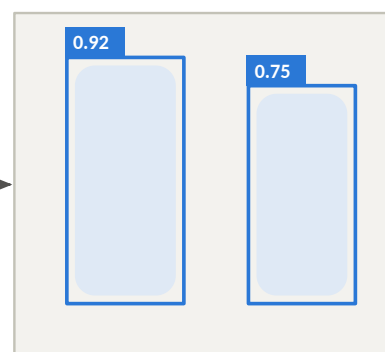
aspect ratios 1:1, 2:1, 1:2 ($\times$ several sizes)
each anchor $\rightarrow$ object? which class? how to shift it?

Before NMS



Sort by score; keep the best box; delete boxes overlapping it by IoU > 0.5 ; repeat.

After NMS (IoU threshold 0.5)



The same figure as guide §4.4 (p. 13). Left: three anchor shapes centred on one grid position. Right: non-maximum suppression, explained in concept 4.11.

WORKED EXAMPLE: HOW MANY ANCHORS?

A typical 4:3 photo reaches Faster R-CNN and RetinaNet as 800×1088 pixels (concept 5.1).

Faster R-CNN's first stage: five pyramid levels of 200×272 , 100×136 , 50×68 , 25×34 and 13×17 cells = $54,400 + 13,600 + 3,400 + 850 + 221 = 72,471$ positions, $\times 3$ shapes = **217,413 anchors**.

RetinaNet: levels of 100×136 down to $7 \times 9 = 18,134$ positions, $\times 9$ anchors (3 sizes $\times$ 3 shapes) = **163,206 anchors**, the "about 160,000" in the guide.

SSD300: 8,732 default boxes on its six small maps (concept 5.3).

A typical trash photo has fewer than 10 objects.

IN YOUR PROJECT

You never create anchors yourself: each torchvision model has an **anchor generator** built in, and the pretrained heads learned their predictions for exactly those anchors, so we keep them unchanged. They only show up in our code where a new layer must match them: `build_retinanet` sizes `cls_logits` with `head.num_anchors` (9 per position), and in `build_ssd`, `num_anchors_per_location()` returns `[4, 6, 6, 6, 4, 4]`, so the new classification head outputs the right number of scores at each position. Viva Q11 (p. 42).

4.7 Giving every anchor a target: positive, negative or ignored

IN THE GUIDE §4.4 p. 13 §4.6 p. 14 §5.1 p. 16

In one line. During training, each anchor is compared with the true boxes. A high IoU makes it a **positive** (learn that object's class and box), a low IoU makes it a **negative** (learn "background"), and anchors in between are left out.

You only give the model a few boxes per photo, yet it makes predictions for over 100,000 anchors. So before the loss can be computed, every anchor needs its own "correct answer". The model generates these automatically by computing the IoU (concept 4.5) of each anchor with every true box in the photo and applying two thresholds. The guide's table in §4.4 (p. 13) lists them:

Model	Positive if IoU ≥	Negative if IoU <	In between
Faster R-CNN, stage 1 (RPN)	0.7	0.3	ignored
Faster R-CNN, stage 2 (proposals)	0.5	0.5	—
RetinaNet	0.5	0.4	ignored
SSD300	0.5	0.5	—

Why ignore the in-between anchors? An anchor that covers half a can is neither clearly "the can" nor clearly "background". Forcing either answer would teach the model something confusing, so it's simply left out of the loss.

The best-match rule. A very small or oddly shaped object may have no anchor above the positive threshold. So RetinaNet, SSD300 and the RPN also make each object's single best-matching anchor positive, whatever its IoU. No object is left without an anchor responsible for it.

WORKED EXAMPLE (RETINANET'S RULE)

A photo contains one Can. Three nearby anchors have IoU 0.62, 0.45 and 0.12 with it. The first is **positive**: it must say "Can" and learn offsets onto the can. The second is **ignored**. The third is **negative**: it must say "no class here". Now a 20 × 15 px cigarette butt: RetinaNet's smallest anchor is 32 × 32 px, so even a perfectly centred one has $\text{IoU } 300 \div 1,024 = 0.29$. That's below 0.4, so it would be background, but the best-match rule makes it positive anyway.

IN YOUR PROJECT

All of this happens inside torchvision when Step 6 calls `model(imgs, tgts)` in training mode. That's why your targets only contain the true `boxes` and `labels`: the model builds the per-anchor answers itself. Tiny objects that only get positives through the best-match rule are one reason small objects are learned less well (concept 9.4).

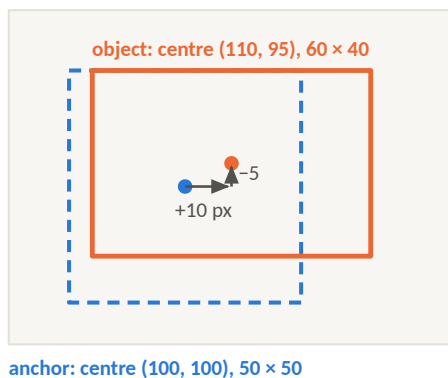
4.8 Box regression: learning corrections to anchors

IN THE GUIDE §4.5 p. 14 §5.1 p. 16 App. A p. 46

In one line. Instead of outputting pixel coordinates, the network predicts four small numbers (t_x, t_y, t_w, t_h) that say how to shift and stretch an anchor onto the object.

Predicting a number, rather than choosing a class, is called **regression**. For boxes, the numbers are corrections relative to the anchor, using the anchor's centre (x_a, y_a), width w_a and height h_a , and the object's centre (x, y), width w and height h :

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$



The four targets for this anchor

$$\begin{aligned} t_x &= (110 - 100) \div 50 &= 0.2 \\ &\text{centre shift, in anchor widths} \\ t_y &= (95 - 100) \div 50 &= -0.1 \\ &\text{centre shift, in anchor heights} \\ t_w &= \log(60 \div 50) &= 0.182 \\ &\text{wider by a factor 1.2} \\ t_h &= \log(40 \div 50) &= -0.223 \\ &\text{shorter by a factor 0.8} \end{aligned}$$

Decoding reverses it: the box (80, 75, 140, 115).

An anchor (dashed) and the object it should fit (solid). The four targets describe the shift of the centre and the change of size, both relative to the anchor.

WORKED EXAMPLE (THE FIGURE'S NUMBERS)

Anchor: centre (100, 100), 50 × 50 px. Object: centre (110, 95), 60 × 40 px.

$$t_x = (110 - 100) \div 50 = 0.2 \cdot t_y = (95 - 100) \div 50 = -0.1 \cdot t_w = \log(60 \div 50) = \log 1.2 = 0.182 \cdot t_h = \log(40 \div 50) = \log 0.8 = -0.223.$$

Decoding runs the formulas backwards: $x = 100 + 0.2 \times 50 = 110$, $y = 100 - 0.1 \times 50 = 95$, $w = 50 \times e^{0.182} = 60$, $h = 50 \times e^{-0.223} = 40$. As corners: (80, 75, 140, 115).

Why corrections and not coordinates?

- **Dividing by the anchor size** puts small and large objects on the same scale: a 10-px shift of a 50-px anchor and a 100-px shift of a 500-px anchor are both 0.2. The same weights can serve every size.
- **The logarithm** makes "twice as wide" ($\log 2 = +0.69$) and "half as wide" ($\log 0.5 = -0.69$) equally big corrections. When decoding, e to the power of anything is positive, so a predicted width can never come out negative.
- The corrections are usually small numbers near 0, which networks learn more easily than raw values in the hundreds.

During training the true box is **encoded** into targets for each positive anchor. At prediction time the network's outputs are **decoded** back into boxes. Negative anchors get no box loss: there is nothing for them to fit.

A detail you may meet in torchvision's code: Faster R-CNN's second stage and SSD300 multiply these targets by fixed weights (10, 10, 5, 5) to put them on a convenient scale. The idea is unchanged.

IN YOUR PROJECT

Faster R-CNN regresses twice: the RPN turns anchors into proposals, then the box head refines each proposal again, with a separate set of 4 numbers for each of the 9 classes (36 outputs of `bbox_pred`). RetinaNet's box subnet outputs 9 anchors $\times$ 4 = 36 numbers per position. The formula is the "Box encoding" row of App. A (p. 46).

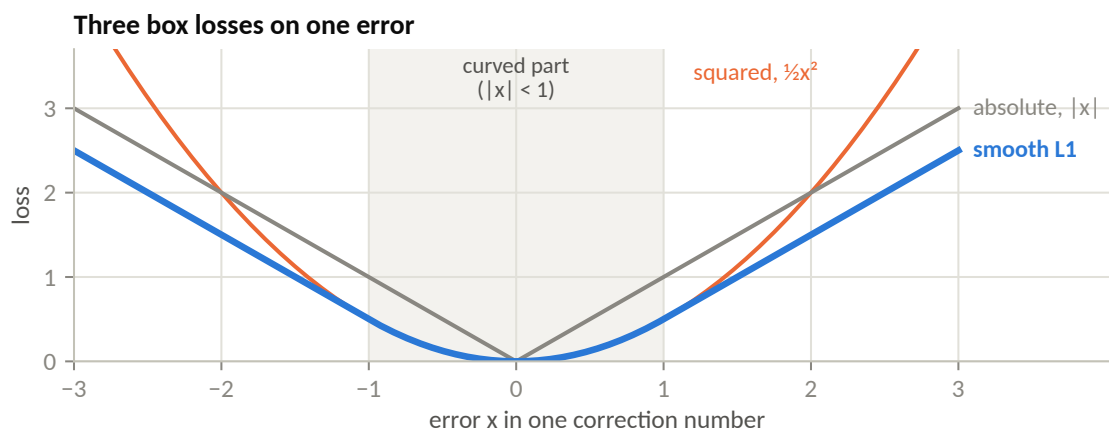
4.9 Box losses: L1, smooth L1 and GIoU

IN THE GUIDE §4.5 p. 14 §5.1 p. 16 §5.2 p. 17 §5.3 p. 19 Q22 p. 42

In one line. A box loss measures how far a predicted box is from the true one. Smooth L1 compares the four correction numbers; GIoU compares the boxes' overlap directly.

Losses on the four numbers. Let x be the error in one correction number (predicted minus target). Three options:

- **Squared (L2)**, $\frac{1}{2}x^2$: gentle near 0 but explosive for big errors. One badly labelled box, or a wild early prediction, would dominate the whole loss.
- **Absolute (L1)**, $|x|$: grows steadily, so it is robust to outliers. But it has a sharp corner at 0, and its slope jumps straight from -1 to $+1$, so the prediction keeps jittering around the answer instead of settling.
- **Smooth L1**: squared for small errors (a soft landing) and a straight line for big ones (robust). With the switch at 1 it is $\frac{1}{2}x^2$ when $|x| < 1$ and $|x| - \frac{1}{2}$ otherwise.



Smooth L1 follows the squared loss near zero and the absolute loss further out.

Error x	0.1	0.5	1	2	5
Squared, $\frac{1}{2}x^2$	0.005	0.125	0.5	2	12.5
Absolute, $ x $	0.1	0.5	1	2	5
Smooth L1	0.005	0.125	0.5	1.5	4.5

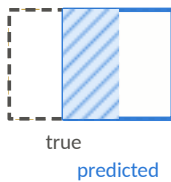
torchvision's Faster R-CNN switches from the squared part to the straight part at $1/9$ instead of 1. Same shape, just a narrower curved section.

Losses on the overlap. The four numbers are each judged separately, but what we really care about is overlap, which is what the metrics measure. An **IoU loss**, $1 - \text{IoU}$, targets overlap directly. It has one flaw: whenever the boxes don't touch, IoU is 0 however far apart they are, so the loss can't tell the model which way to move. **GIoU** (generalised IoU) fixes that. Let C be the smallest box that encloses both boxes, and U their union:

$$\text{GIoU} = \text{IoU} - \frac{\text{area}(C) - U}{\text{area}(C)}, \quad \text{GIoU loss} = 1 - \text{GIoU}$$

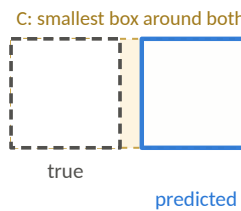
The penalty is the share of C that is empty space. It keeps growing as the boxes drift apart, so GloU runs from -1 to 1.

overlapping by half



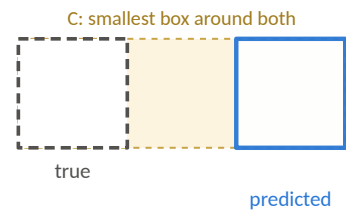
IoU 0.33 • GloU 0.33

4 px apart



IoU 0 • GloU -0.09

20 px apart



IoU 0 • GloU -0.33

Yellow: empty space inside C, which is GloU's penalty. Blue hatching: the overlap.

A 20 × 20 px true box (grey) and a prediction (blue) at three distances. IoU can't tell the last two apart; GloU can.

WORKED EXAMPLE (THE FIGURE)

True box (0, 0, 20, 20), area 400 px². Predictions are the same size, shifted to the right.

Overlapping by half (10, 0, 30, 20): overlap 200, union 600, IoU = 0.333. C = (0, 0, 30, 20) = 600 = U, so no penalty: GloU = 0.333.

4 px apart (24, 0, 44, 20): IoU = 0. C = 44 × 20 = 880, U = 800, penalty 80 ÷ 880 = 0.091: GloU = -0.091.

20 px apart (40, 0, 60, 20): IoU = 0. C = 60 × 20 = 1,200, penalty 400 ÷ 1,200: GloU = -0.333.

The GloU losses are 0.667, 1.091 and 1.333: the further away, the bigger the loss, so the model is always pulled towards the object.

IN YOUR PROJECT

`loss_rpn_box_reg` and `loss_box_reg` (Faster R-CNN) and SSD300's `bbox_regression` are smooth L1. RetinaNet v2's `bbox_regression` is the GloU loss, one of the "v2" changes in viva Q22 (p. 42). In every model, only positive anchors or proposals contribute to the box loss.

4.10 The imbalance between objects and background

IN THE GUIDE §4.6 p. 14 D2 p. 7 §5.4 p. 19

In one line. Almost every anchor is background, so a naive loss is dominated by easy background, and the model can score well while learning little about waste.

Picture a teacher marking a test with 10,000 questions, where 9,990 of them are "Is this blank patch of pavement a bottle? (no)". A student who answers "no" to everything scores 99.9% without learning anything. Anchors create exactly this situation.

WORKED EXAMPLE: HOW BADLY THE BACKGROUND CAN SWAMP THE LOSS

RetinaNet has 163,206 anchors per photo. Suppose the photo has 5 objects with about 10 positive anchors each: 50 positives, and 99.97% of anchors are background.

Imagine a lazy model that gives every anchor 99% background. Each negative then costs only $-\log 0.99 = 0.010$, but there are 163,156 of them: $163,156 \times 0.010 \approx 1,640$. Each positive costs $-\log 0.01 = 4.6$, but there are only 50 of them: $50 \times 4.6 = 230$.

So even though the model has learned *nothing* about waste, most of its loss (and most of the gradient) is about making easy background slightly more certain.

Two words help here. **Easy negatives** are obvious background such as plain pavement: there are huge numbers of them and they teach little. **Hard negatives** are background that looks like waste, such as a leaf, a stone or a paper scrap: they are rare and very informative. A good strategy stops the easy negatives from drowning everything else.

Model	Strategy	In short (details in Part 5)
Faster R-CNN	Filter, then sample	The RPN keeps only about 2,000 regions. Each training step then uses a fixed mix: 256 anchors with up to half positive, and 512 regions with up to a quarter positive (concept 5.1).
RetinaNet	Re-weight	Keeps every anchor but uses focal loss, which shrinks the loss of easy examples almost to zero (concept 5.2).
SSD300	Mine the hard ones	Uses only the hardest negatives, 3 per positive, and ignores the rest (concept 5.3).

Don't confuse this with the other imbalance in the project: some waste *classes* have more examples than others (concept 6.6).

IN YOUR PROJECT

Guide §4.6 (p. 14) calls this "the thread through all three models", and it's the second of the two questions in decision D2 (p. 7): how do you stop the background from swamping the learning? The "Handles background by" row of the table in §5.4 (p. 19) is the short answer for each model.

4.11 Post-processing: score threshold, top-k and NMS

IN THE GUIDE §4.7 p. 14 §2.3 p. 8 §9.2 p. 30 Q19 p. 42

In one line. After the network runs, weak boxes are dropped, and non-maximum suppression (NMS) deletes near-duplicate boxes so that each object is reported once.

Raw network output is messy. Every anchor produces a box and a score, and all the anchors near an object tend to fire at once, so each object is buried under a pile of near-identical boxes. **Post-processing** cleans this up in four steps:

1. **Score threshold:** drop boxes scoring below 0.01. This is deliberately low: it removes junk but keeps nearly every candidate that could matter for mAP.
2. **Top-k:** keep only the best few hundred or thousand candidates (RetinaNet keeps the top 1,000 per pyramid level), to save time.
3. **NMS**, for each class separately: sort the boxes by score, keep the best, delete every remaining box that overlaps it with IoU above 0.5, then repeat with the best box still standing.
4. **Cap:** keep at most 100 boxes per photo.

WORKED EXAMPLE: THE NMS PICTURE IN CONCEPT 4.6 (GUIDE §4.4)

Four Bottle boxes, in the pixel coordinates of that drawing:

0.92: (26, 62, 84, 184) · **0.85:** (40, 80, 90, 188) · **0.75:** (116, 76, 169, 184) · **0.60:** (12, 98, 86, 198).

Round 1. Keep 0.92 (area $58 \times 122 = 7,076$). Compare the rest with it:

- 0.85: overlap $44 \times 104 = 4,576$; union $7,076 + 5,400 - 4,576 = 7,900$; IoU = **0.58** > 0.5, so **delete**.
- 0.75: no overlap, IoU = **0**, so it stays.
- 0.60: overlap $58 \times 86 = 4,988$; union $7,076 + 7,400 - 4,988 = 9,488$; IoU = **0.53** > 0.5, so **delete**.

Round 2. The best box still standing is 0.75. Keep it. Nothing is left.

Result: two boxes for two bottles.

Per class. A Can box is never compared with a Bottle box, so a can lying against a bottle keeps its box even if the two overlap a lot.

The trade-off. The NMS threshold decides what counts as a duplicate. If two *real* bottles touch and their boxes overlap with IoU above 0.5, NMS deletes the second one, and it becomes a missed object ("merged with neighbour", concept 9.3). Raise the threshold and more genuine duplicates survive instead ("duplicate", concept 9.2). **Soft-NMS**, suggested as future work in the guide, doesn't delete overlapping boxes. It lowers their scores in proportion to the overlap, so a real neighbour can survive with a reduced score.

Two different thresholds. The 0.01 score threshold here keeps almost everything so that mAP can sweep over all scores. The *confidence threshold* used for precision and recall is a separate, much higher value, chosen later on the validation set (concept 8.3).

IN YOUR PROJECT

`SCORE_THRESH, NMS_THRESH, MAX_DETS = 0.01, 0.5, 100`, set at the end of each `build_*` function so all three models are post-processed identically (§2.3, p. 8). Faster R-CNN's RPN also runs its own NMS at 0.7 on its proposals. The error types that NMS causes are in §9.2 (p. 30). Viva Q19 (p. 42).

4.12 The Feature Pyramid Network (FPN)

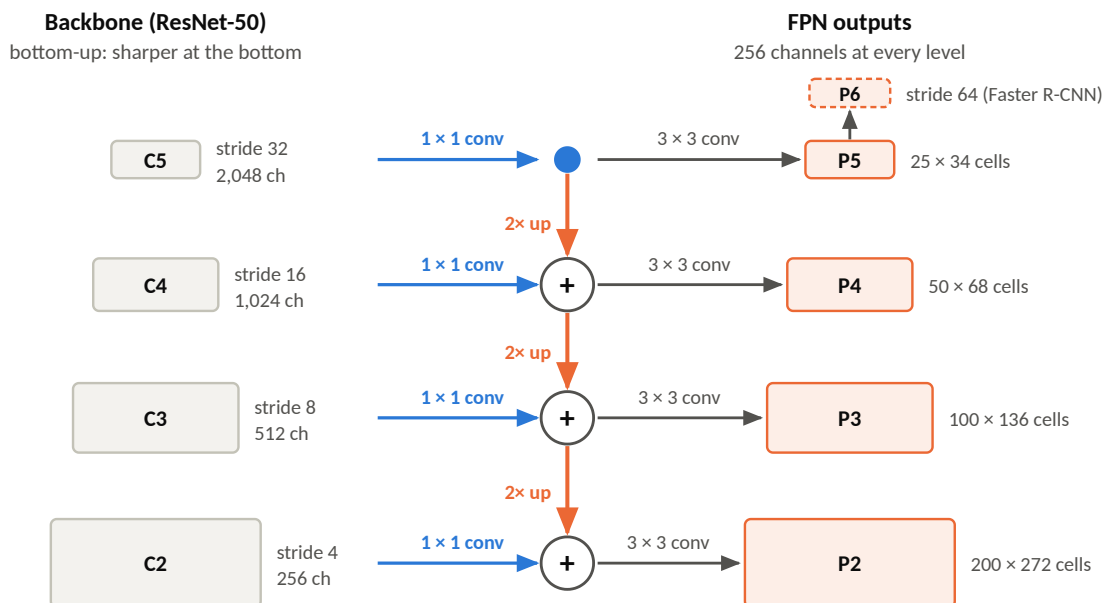
IN THE GUIDE §4.8 p. 14 §3.1 p. 10 §5.1 p. 16 §5.2 p. 17 Q18 p. 42

In one line. An FPN mixes the backbone's deep maps (coarse but smart) into its shallow maps (sharp but simple), so every level of the pyramid is both sharp and meaningful.

The dilemma. From concept 3.4: ResNet-50's stride-4 map (200×272 cells for our photos) is sharp enough to hold a 20-px cigarette butt, but it comes from early layers that only know edges and blobs. Its stride-32 map (25×34) "knows" bottles and cans, but each cell covers 32×32 px, bigger than the butt. A detector needs both kinds of knowledge in the same place.

How the FPN solves it, in three steps (the backbone's outputs are called C2 to C5, the FPN's outputs P2 to P5, and the number is the stride as a power of two: P2 = stride 4, P5 = stride 32):

1. **Lateral connections.** A 1×1 convolution (concept 3.5) turns each backbone map, with 256, 512, 1,024 or 2,048 channels, into a map with 256 channels.
2. **Top-down pathway.** Start at the deepest map. Enlarge it $2\times$ (nearest-neighbour upsampling, concept 3.9) and add it, cell by cell, to the next shallower map. Then enlarge that sum and add it to the next one, and so on down to stride 4. The deep knowledge flows down into the sharp maps.
3. **Smoothing.** A 3×3 convolution cleans up each sum, giving the output maps P2 to P5.



The FPN in Faster R-CNN. Deep features flow down (orange) and are added to the sharper maps coming in from the side (blue). Every output level has 256 channels.

Extra coarse levels are added for very large objects: Faster R-CNN adds P6 (stride 64) by down-sampling P5, and RetinaNet uses P3 to P7 (strides 8 to 128), with no stride-4 level.

Which level handles which object? In RetinaNet and the RPN, it's decided by the anchors: small anchors live on the fine levels, big anchors on the coarse ones. Faster R-CNN's second stage chooses a level for each proposal from its size, measured as $\sqrt{(\text{width} \times \text{height})}$: below 112 px it reads from P2, from 112 px from P3, from 224 px from P4, and from 448 px from P5.

WORKED EXAMPLE

A 40-px cigarette butt proposal is read from P2 (stride 4), where it covers 10×10 cells of rich features. A 250-px sheet of cardboard is read from P4 (stride 16), where it covers about 16×16 cells. Both get a detailed, meaningful description.

SSD300 has no FPN: it came out a year earlier. Its finest map is taken from a middle layer of VGG-16 and never receives the deeper features. Together with its 300×300 input, that is why the guide expects it to struggle with small items (§4.8, p. 14).

IN YOUR PROJECT

This is the "FPN, 5 levels" block in the guide's Faster R-CNN figure and "P3-P7" in the RetinaNet figure (§5.1, p. 16; §5.2, p. 17). Recall by object size (concept 9.4) is where you can check whether the FPN models really find small items better. Viva Q18 (p. 42).

4.13 Families of detectors: two-stage, one-stage and beyond

IN THE GUIDE §4.9 p. 15 D2 p. 7 Q9 p. 41 Q10 p. 41

In one line. Two-stage detectors first propose regions and then examine each one closely; one-stage detectors decide everything in a single pass; newer designs remove the anchors, or even NMS.

An analogy: a **two-stage** security guard scans a crowd, notes a few suspicious spots, then walks over and inspects each one: thorough but slow. A **one-stage** guard makes a snap decision for every square of a grid at once: fast, but most squares are empty and must not distract the guard (concept 4.10).

Family	How it works	Examples
Two-stage	Stage 1 proposes regions that probably contain something; stage 2 classifies each region and refines its box.	Faster R-CNN , Mask R-CNN, Cascade R-CNN
One-stage, anchor-based	Every anchor gets a class and a box correction in one pass.	RetinaNet , SSD ; classic YOLO versions too
One-stage, anchor-free	No anchors: each position predicts its distances to the four sides of the object it's inside (FCOS), or object centres are found as peaks in a heat-map (CenterNet).	FCOS, CenterNet
Transformer	Predicts a fixed set of boxes (say 100) directly; training matches each object to exactly one prediction, so there is no NMS.	DETR, Deformable DETR

Our three cover both classic families, the accuracy–speed trade-off and three ways of handling background, and they share one torchvision pipeline. YOLO is not allowed by the brief, and the original DETR needed very long training (300–500 epochs on COCO), a poor fit for 114 photos.

IN YOUR PROJECT

Decision D2 (p. 7); viva Q9 (p. 41) and Q10 (p. 41). Asked what else you could have tried? The guide names FCOS, Faster R-CNN with a MobileNet backbone and SSDlite, all in torchvision.

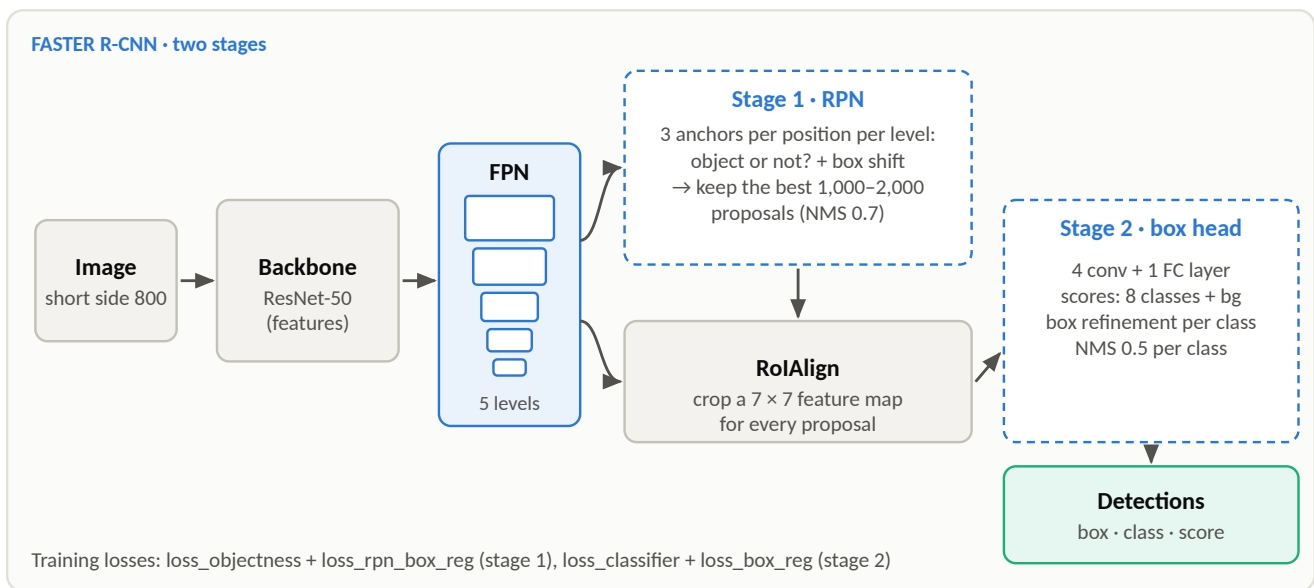
The three models, stage by stage

Guide chapter 5 describes each model in about a page. Here each one is walked through in order, following a single photo from pixels to final boxes, with the numbers at every stage. Everything is built from the Part 4 blocks, so the concept numbers in brackets tell you where to look if a step feels unfamiliar.

5.1 Faster R-CNN: propose, then inspect

IN THE GUIDE §5.1 p. 16 §4.6 p. 14 Q12 p. 42 Q13 p. 42 Q20 p. 42

In one line. Faster R-CNN looks twice: a Region Proposal Network suggests about a thousand regions that probably contain something, then a second network crops the features of each region and decides its class and exact box.



The same figure as guide §5.1 (p. 16). Follow the numbered steps below along it.

Step by step, for one 1024×768 photo

- 1. Resize and normalise.** The model's built-in transform scales the photo so its short side is 800 px (the long side may not exceed 1,333): about $1,067 \times 800$. It pads the width with zeros to 1,088, a multiple of 32, so the maps halve cleanly at every stride. It also **normalises** each colour channel with the ImageNet averages the backbone was trained with. For example, a red value of 0.6 becomes $(0.6 - 0.485) \div 0.229 = 0.50$.
- 2. Backbone and FPN.** ResNet-50 (concept 3.6) plus the FPN (concept 4.12) produce five maps, P2 to P6, at strides 4 to 64, each with 256 channels.
- 3. Stage 1: the Region Proposal Network (RPN).** A small convolutional head (two 3×3 layers in v2) slides over every level. At each position it has 3 anchors (concept 4.6), one size per level (32 px on P2 up to 512 px on P6) in three shapes. For each anchor it outputs one **objectness** score ("is there *any* object here?", concept 4.3) and 4 box corrections (concept 4.8). That's 217,413 anchors in total.
- 4. Proposals.** The anchors are decoded into boxes. On each level the 2,000 with the highest objectness are kept (1,000 at test time). Boxes are clipped to the image, tiny ones are removed, and NMS at IoU 0.7 (concept 4.11) removes duplicates. The best 2,000 overall survive during training, 1,000 at test time. These are the **proposals**, also called regions of interest (RoIs).

5. **RoIAlign.** For every proposal, a fixed 7×7 grid of features is read from the pyramid level that suits its size (concept 4.12): $7 \times 7 \times 256$ numbers per proposal, however big or small the region.
6. **Stage 2: the box head.** Four conv layers and one fully connected layer (the v2 design) turn each $7 \times 7 \times 256$ patch into 1,024 numbers. The final predictor turns those into 9 class scores (softmax, concept 2.2) and 36 box numbers, 4 corrections for each of the 9 classes (concept 4.8).
7. **Post-processing.** For each proposal and each class, decode that class's box and take the softmax probability as its score. Drop scores below 0.01, run NMS at 0.5 per class, keep the best 100 (concept 4.11).

WORKED EXAMPLE: THE FUNNEL

217,413 anchors $\rightarrow$ at most 1,000 proposals (at test time) $\rightarrow$ up to $1,000 \times 8$ class-specific boxes $\rightarrow$ at most **100 detections**. The expensive stage-2 head runs only on the 1,000 proposals, not on 217,413 anchors. That's the whole point of proposing first.

Why RoIAlign matters

Proposals have fractional positions on the feature map. A box edge at $x = 83.4$ px sits at cell $83.4 \div 16 = 5.21$ on a stride-16 map. The older **RoIPool** rounded this to cell 5, shifting the features by 0.21 cells, which is 3.4 px in the image. For a small object, a few pixels matter (concept 4.5). **RoIAlign** never rounds. It divides the region into 7×7 bins, reads 2×2 points in each bin at their exact fractional positions using bilinear interpolation (concept 3.9), and averages them. The features line up with the box, which gives more precise boxes. And because the backbone runs once for the whole photo and every region just reads from those shared maps, this is cheap.

How it learns: two sampled stages, four losses

Loss in the log	What it trains	Which candidates count
<code>loss_objectness</code>	RPN: object or not (binary cross-entropy, concept 4.3)	256 anchors per photo, up to half positive ($\text{IoU} \geq 0.7$), the rest negative ($\text{IoU} < 0.3$)
<code>loss_rpn_box_reg</code>	RPN: corrections from anchor to object (smooth L1, concept 4.9)	the positive anchors among those 256
<code>loss_classifier</code>	Box head: which of the 9 classes (cross-entropy, concept 4.2)	512 proposals per photo, up to a quarter positive ($\text{IoU} \geq 0.5$)
<code>loss_box_reg</code>	Box head: corrections from proposal to object (smooth L1)	the positive proposals, using only the 4 numbers of their true class

This **sampling** is Faster R-CNN's answer to the imbalance of concept 4.10. Of 217,413 anchors, only 256 are used per photo per step, and they are chosen so that up to half are positive. The easy background can't swamp the loss because most of it never enters the loss.

What "v2" changes (viva Q22): a two-layer RPN head, conv + BatchNorm layers in the box head and FPN, and a longer training recipe with stronger augmentation. It reaches 46.7 COCO mAP, against 37.0 for the original torchvision version.

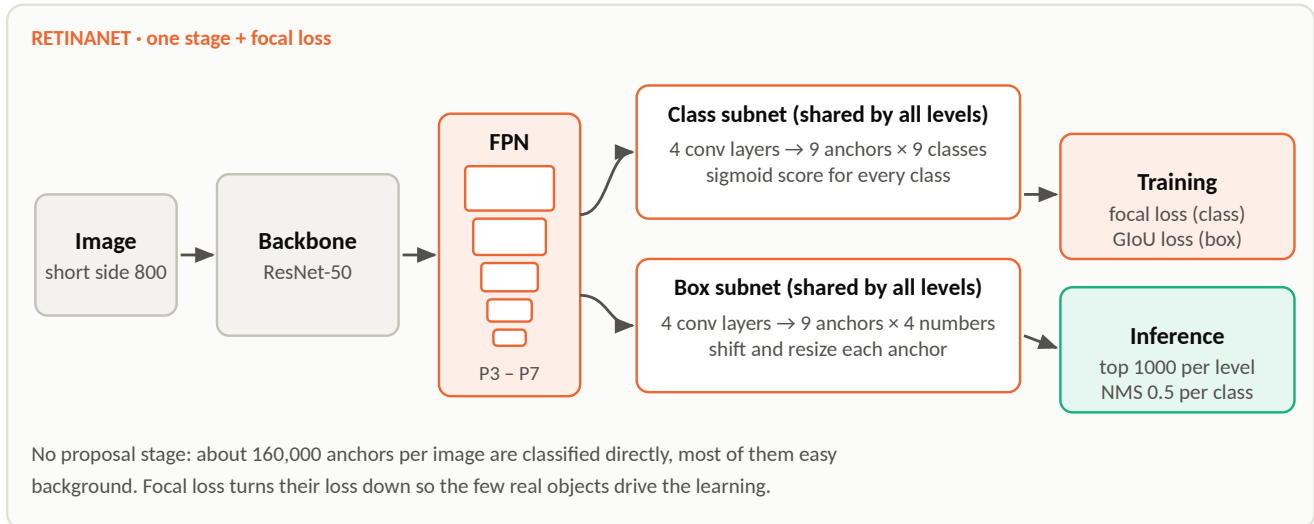
IN YOUR PROJECT

`build_faster_rcnn` replaces only `roi_heads.box_predictor` with `FastRCNNPredictor(1024, 9)`, the last two small layers (concept 2.3). The RPN only asks "object or not?", which doesn't depend on our classes, so it keeps its COCO weights and is fine-tuned. The four loss names are the four rows above. Viva Q12 (p. 42) (RPN), Q13 (p. 42) (RoIAlign) and Q20 (p. 42).

5.2 RetinaNet and focal loss

IN THE GUIDE §5.2 p. 17 §4.6 p. 14 App. A p. 46 Q14 p. 42 Q15 p. 42

In one line. RetinaNet classifies and adjusts every anchor in one pass, and uses **focal loss** so that the flood of easy background anchors doesn't drown out the few real objects.



The same figure as guide §5.2 (p. 17).

The architecture

- **Backbone and FPN:** ResNet-50 with pyramid levels P3 to P7, strides 8 to 128 (concept 4.12).
- **Anchors:** 9 per position on every level: 3 sizes (for P3: 32, 40 and 50 px) × 3 shapes. 163,206 anchors in total (concept 4.6).
- **Two subnets**, small stacks of four 3×3 conv layers (each with GroupNorm, concept 2.5, and ReLU). The **class subnet** ends in `cls_logits`, a 3×3 conv with $9 \text{ anchors} \times 9 \text{ classes} = 81$ outputs per position, one sigmoid score each (concept 4.3). The **box subnet** ends with $9 \times 4 = 36$ corrections per position.
- **Shared across levels:** the same subnet weights run on P3 and on P7. A can is a can at every scale, so there's no need for separate weights per level.

There is no proposal stage and no sampling. Every one of the 163,206 anchors is scored for every class, and the loss is computed over all of them, except the ignored ones (concept 4.7). So something must stop 163,000 easy background anchors from dominating the loss. That something is focal loss.

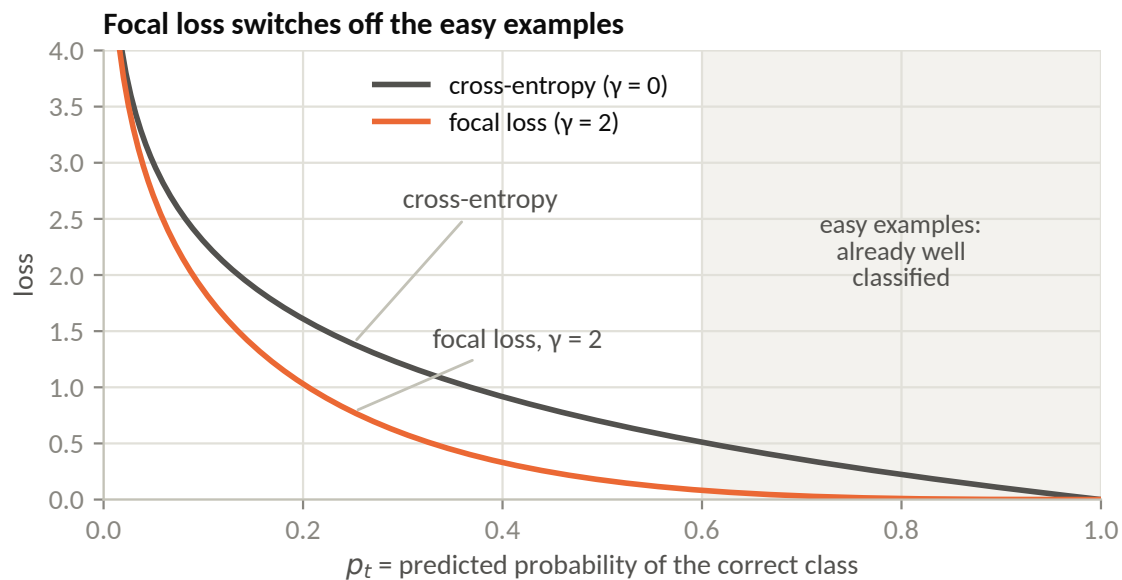
Focal loss, built up in three steps

Step 1: p_t , the probability of the correct answer. Each class output is a yes/no question. If the true answer is yes, $p_t = p$; if it's no, $p_t = 1 - p$. A Bottle anchor scoring 0.9 for Bottle has $p_t = 0.9$. A pavement anchor scoring 0.1 for Can also has $p_t = 0.9$, because the correct answer is "not a can" and the model gave that 0.9. Binary cross-entropy (concept 4.3) can then be written as a single formula: $CE = -\log p_t$.

Step 2: the modulating factor. Multiply by $(1 - p_t)^\gamma$ with $\gamma = 2$. When an example is already easy (p_t near 1), the factor is almost 0 and the loss nearly vanishes. When it's hard (p_t small), the factor is close to 1 and the loss stays.

Step 3: the balancing weight α . Multiply positives by $\alpha = 0.25$ and negatives by $1 - \alpha = 0.75$. That gives the full formula:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$



The same chart as guide §5.2 (p. 17). Above $p_t \approx 0.6$ the focal loss is nearly flat at zero: easy examples stop mattering.

WORKED EXAMPLE: THE GUIDE'S TABLE, NOW INCLUDING A

Easy background output, $p_t = 0.99$, $\alpha_t = 0.75$: $\text{CE} = -\log 0.99 = 0.0101$. $\text{FL} = 0.75 \times (0.01)^2 \times 0.0101 = \mathbf{0.00000075}$, about 13,000 times smaller than CE.

Hard, misclassified object, $p_t = 0.1$, $\alpha_t = 0.25$: $\text{CE} = -\log 0.1 = 2.303$. $\text{FL} = 0.25 \times (0.9)^2 \times 2.303 = \mathbf{0.466}$, only 5 times smaller.

Under plain cross-entropy the hard example counted 229 times as much as the easy one. Under focal loss it counts about 620,000 times as much. The 163,000 easy anchors fade out, and the few hard ones drive the learning.

Why is α only 0.25 for the rare positives? It seems backwards, since positives are the rare ones. But the $(1 - p_t)^\gamma$ factor has already removed most of the easy negatives' weight. With $\gamma = 2$, the RetinaNet paper found that giving positives slightly *less* weight worked best. The two settings were tuned together.

Normalisation. The focal losses of all anchors are summed and then divided by the number of positive anchors. A photo with ten objects and one with a single object then give losses on a similar scale.

The 1% prior: why the last layer starts biased

A freshly created layer outputs logits near 0, and $\text{sigmoid}(0) = 0.5$ (concept 2.2): every anchor would start out 50% sure it holds every class. RetinaNet instead sets the bias of `cls_logits` to

$$b = -\log \frac{1 - \pi}{\pi} = -\log 99 = -4.595 \quad \text{for } \pi = 0.01,$$

and $\text{sigmoid}(-4.595) = 0.01$. Every anchor now starts at a 1% belief in every class. That is honest, because nearly all anchors really are background.

WORKED EXAMPLE: THE FIRST STEP, WITH AND WITHOUT THE PRIOR

163,206 anchors $\times$ 9 class outputs = about 1.47 million yes/no outputs, nearly all of them "no". Starting at 50%, each "no" output costs $0.75 \times 0.5^2 \times \log 2 = 0.13$ even after focal weighting. Together that is about 191,000. Starting at 1%, each costs 0.00000075, about 1.1 in total. The 50% start would produce gigantic gradients in the very first steps, strong enough to damage the pretrained layers (concept 2.6).

Losses and inference

Loss in the log	What it measures
<code>classification</code>	Focal loss over all non-ignored anchors and classes, divided by the number of positive anchors
<code>bbox_regression</code>	GIoU loss (concept 4.9) between the decoded boxes and their objects, positive anchors only

At test time, each level keeps its 1,000 best-scoring (anchor, class) pairs above the score threshold. These are decoded, the levels are merged, NMS at 0.5 runs per class, and the best 100 are kept (concept 4.11). **What "v2" changes:** GroupNorm in the subnets, which is stable with small batches like our 4 (concept 2.5), the GIoU box loss, and a longer training recipe: 41.5 COCO mAP against 36.4 for v1.

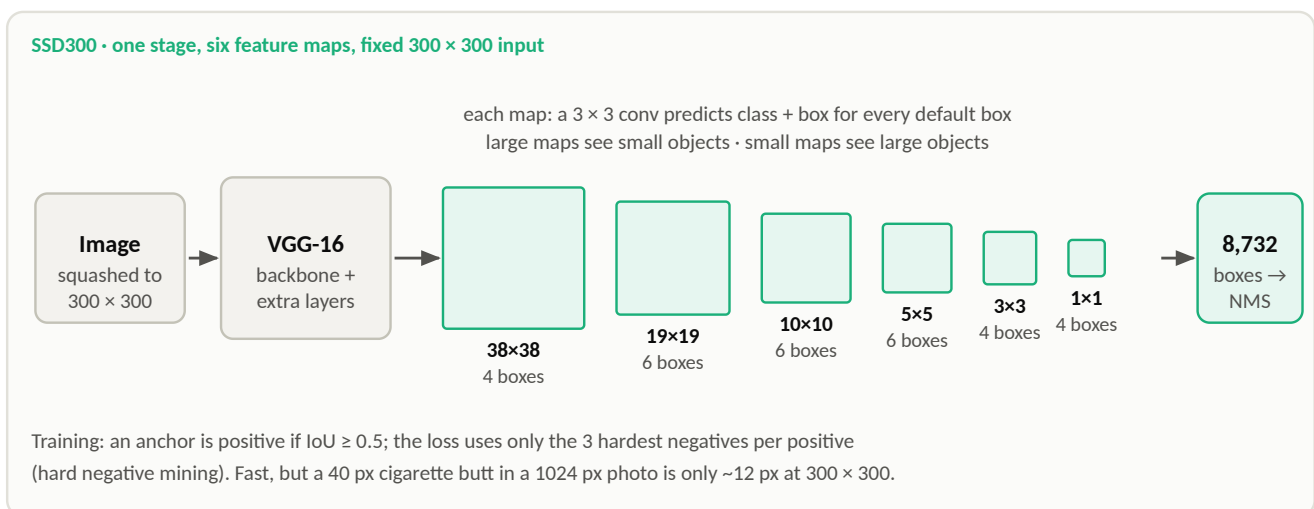
IN YOUR PROJECT

`build_retinanet` keeps the pretrained subnet layers and replaces only `cls_logits` (91 COCO classes $\rightarrow$ 9). It starts the new weights small (`normal_(std=0.01)`) and the bias at the prior: `constant_(head.cls_logits.bias, -math.log((1 - 0.01) / 0.01))`. It also sets `head.num_classes = 9`, because the head uses it to reshape its output. The new layer's 186,705 parameters are worked out in concept 3.5. Viva Q14 (p. 42) and Q15 (p. 42); the formulas are in App. A (p. 46).

5.3 SSD300: one quick look at six scales

IN THE GUIDE §5.3 p. 19 §4.8 p. 14 Q16 p. 42 Q17 p. 42

In one line. SSD squashes the photo to 300×300 and, in one pass, predicts a class and a box correction for 8,732 default boxes spread over six feature maps of decreasing size.



The same figure as guide §5.3 (p. 19).

Step by step

- Squash to 300×300 .** The aspect ratio is *not* kept. A 1024×768 photo shrinks 3.4 times horizontally but only 2.6 times vertically, so a round bottle cap becomes a slightly tall oval. Training and testing squash the same way, so the model learns squashed shapes. The real cost is size: everything becomes small.
- VGG-16 and extra layers** (concept 3.6). The first prediction map is taken from the middle of VGG-16 (38×38 cells). The rest of VGG-16 gives 19×19 , and extra conv layers keep halving: 10×10 , 5×5 , 3×3 and 1×1 .
- Default boxes** on each map. The sizes grow with the map: 21, 45, 99, 153, 207 and 261 px (7% to 87% of 300). Each position also gets an extra square of in-between size (31 px on the first map) and stretched shapes 2:1 and 1:2, plus 3:1 and 1:3 on the three middle maps. That gives 4 or 6 boxes per position.

4. **Predictors:** two 3×3 convs per map (concept 3.5), one for classes and one for boxes, output for every default box 9 class scores (softmax, including background) and 4 corrections.
5. **Post-processing:** per class, the best candidates above the score threshold go through NMS, and the best 100 overall are kept.

WORKED EXAMPLE: WHERE 8,732 COMES FROM

$38 \times 38 \times 4 = 5,776 \cdot 19 \times 19 \times 6 = 2,166 \cdot 10 \times 10 \times 6 = 600 \cdot 5 \times 5 \times 6 = 150 \cdot 3 \times 3 \times 4 = 36 \cdot 1 \times 1 \times 4 = 4$.
Total: **8,732**. Two-thirds of them sit on the 38×38 map, the only one fine enough for small objects.

Training: hard negative mining

A default box is positive if its IoU with an object is at least 0.5, plus each object's best box (concept 4.7). Every other box is a negative. Instead of re-weighting like RetinaNet, SSD **chooses** which negatives count. It computes the cross-entropy of every negative, sorts them, and keeps only the highest-loss ones: 3 for every positive. These **hard negatives** are exactly the background patches that look most like waste. All other negatives are left out.

WORKED EXAMPLE

A photo with 4 objects gives, say, 20 positive default boxes. SSD keeps the 60 hardest of the remaining 8,712 negatives. The classification loss uses 80 boxes, the other 8,652 are ignored for this photo, and the total is divided by the 20 positives. The box loss (smooth L1, concept 4.9) uses the 20 positives only.

Why small objects are SSD300's weak spot

WORKED EXAMPLE (GUIDE §5.3 AND VIVA Q17)

A cigarette butt 40 px wide in a 1,024-px-wide photo becomes $40 \times 300 \div 1,024 \approx 12$ px wide at 300×300 . The finest map has cells of $300 \div 38 \approx 7.9$ px, so the butt covers about 1.5 cells. The smallest default box is 21×21 px: even perfectly centred on a 12×12 px object, its IoU is $144 \div 441 = 0.33$, below the 0.5 needed to be positive. The butt only gets a positive box through the best-match rule. And the 38×38 map comes from the middle of VGG-16, with no FPN bringing deeper knowledge down to it (concept 4.12).

Its strengths: about 35 GFLOPs per image against 152 and 280 for the others (concept 8.8), a small checkpoint, and simple, fast inference.

IN YOUR PROJECT

`build_ssd` rebuilds the whole classification head with `SSDClassificationHead(in_channels, num_anchors, 9)`, where `in_channels` is `[512, 1024, 512, 256, 256, 256]` (the channels of the six maps) and `num_anchors` is `[4, 6, 6, 6, 4, 4]`. SSD300's own defaults were an NMS threshold of 0.45 and 200 boxes; our builder sets 0.5 and 100 like the others. Its log shows `bbox_regression` and `classification`. Viva Q16 (p. 42) and Q17 (p. 42).

5.4 Comparing the three designs

IN THE GUIDE §5.4 p. 19 §2.5 p. 8 D7 p. 7 Q9 p. 41 Q22 p. 42 Q41 p. 44 Q44 p. 44

In one line. The three models make different trade-offs between accuracy and speed. The guide's hypotheses H1 to H3 are predictions of how those trade-offs will show up on your data.

The guide's table in §5.4 (p. 19) lists the facts. This table lines up the *reasons* behind them, using the concepts of Part 4:

	Faster R-CNN v2	RetinaNet v2	SSD300
Candidates examined	217,413 anchors → 1,000 proposals, each inspected again	163,206 anchors, each once	8,732 default boxes, each once
Input resolution	800-px short side	800-px short side	300 × 300
Finest map, in original-photo pixels	stride 4: ≈ 4 px per cell	stride 8: ≈ 8 px per cell	38 × 38: ≈ 27 px per cell (1,024-px photo)
Deep features at fine levels	yes (FPN)	yes (FPN)	no
Background handled by	proposals + sampling	focal loss	hard negative mining
Compute per image	280 GFLOPs	152 GFLOPs	35 GFLOPs
Expected result	most accurate, slowest	close in accuracy, faster	fastest, weakest on small items

Parameters are not speed. The number of weights (43.3 M, 36.5 M, 24.7 M) sets the checkpoint size. The time per image depends on how much arithmetic is done, and that grows with the input size and the number of candidates. SSD300 has a bit more than half as many weights as Faster R-CNN, yet it needs about eight times less arithmetic per image, mostly because it works on $300 \times 300 = 90,000$ pixels instead of $800 \times 1,088 \approx 870,000$. Concept 8.8 explains how to measure speed properly.

COCO mAP won't be your mAP. The published 46.7 / 41.5 / 25.1 come from 80 classes, 118,000 training photos and COCO's 5,000-photo validation set. You have 8 classes, 114 training photos and 25 test photos, so your absolute numbers will differ, and even the ranking could change. That's exactly why the brief asks you to measure.

The hypotheses, and the concepts behind them (guide §2.5, p. 8):

- **H1**, Faster R-CNN most accurate, especially on small objects: a second look at every region (5.1), a stride-4 level with FPN (4.12), 800-px input.
- **H2**, SSD300 fastest but weakest on small items: 35 GFLOPs, but a 300×300 input with no FPN (5.3).
- **H3**, RetinaNet close to Faster R-CNN, and faster: focal loss (5.2) handles the background without a second stage.

IN YOUR PROJECT

The table in guide §5.4 (p. 19) goes into the Methods section of your report, and your results table tests H1 to H3. Decision D7 (p. 7) says why speed is measured at all: a sorting line runs in real time. "Why these three?" is viva Q9 (p. 41), and "what does v2 mean?" is Q22 (p. 42). "Which model is best?" (Q41, p. 44) and "did your hypotheses hold?" (Q44, p. 44) are answered from your results table, with the reasons in this table.

Data: from files to training examples

Guide chapter 6 and notebook Steps 1 to 5 turn a Kaggle download into clean, checked, split and augmented training examples. This part explains the ideas behind each step: what a label file is, why phone photos can silently break a detector, how the split protects the test set, and why augmentation for detection needs extra care.

6.1 Labelled datasets and what's in this one

IN THE GUIDE §6.1 p. 21 §11.4 p. 40 Q2 p. 41

In one line. A labelled detection dataset is a set of photos plus, for each photo, a list of boxes and class names drawn by a person, the **annotator**. The model can only ever be as good as those labels.

Annotation means drawing the labels. For detection, the annotator drags a rectangle around every object and picks its class. This is slow, careful work, which is why datasets are small and why every label matters. Labels are never perfect: one annotator draws boxes tightly, another loosely; a half-hidden can might be labelled or skipped. These inconsistencies are called **label noise**. The model learns them too, and the evaluation measures against them.

What the Kaggle download contains, according to the dataset page (§6.1, p. 21):

- 163 original photos of street-collected trash, with boxes in 8 classes;
- the same labels in three formats: Pascal VOC, COCO and YOLO (concept 4.4);
- derived copies: the 163 photos resized to 1024×1024 , 4,923 augmented images, and a ready-made 70 / 20 / 10 split;
- a licence: Apache 2.0, which allows use and modification as long as the licence notice is kept. Always check the licence before using a dataset, and cite it in the report.

Originals versus derived data. Everything except the 163 originals was *made from* them. The derived copies add no new information, and some of them carry risks: squashed shapes in the square copies, and leakage in the augmented copies (concept 1.11). That's why the project uses only the originals and makes its own clean copy, split and augmentation.

IN YOUR PROJECT

The guide's "Use it?" table in §6.1 (p. 21) is your answer to "why didn't you use the augmented data?" (viva Q3, p. 41). "Labels by a single annotator; box tightness varies" is one of the limitations listed in §11.4 (p. 40). Describing the dataset is viva Q2 (p. 41).

6.2 Label files: XML, JSON and TXT

IN THE GUIDE §6.2 p. 21 Step 2 p. 32

In one line. Label files are plain text in a fixed structure. Reading one means finding the numbers you need inside that structure, which is called **parsing**.

- **XML** (Pascal VOC) nests values inside named tags, like `<xmin>812</xmin>`. One `.xml` file per photo, with an `<object>` block for every box. It also records the photo's file name and its `<size>`, the width and height at the time of labelling. That recorded size turns out to be very useful (concept 6.3).

- **JSON (COCO)** is the notation of Python dictionaries and lists. One file holds everything: a list of images, a list of annotations (each pointing to an image by `image_id`), and a list of categories.
- **TXT (YOLO)** is the simplest: one line per object, five numbers separated by spaces, with the class as a number counting from 0.

The guide shows the same Bottle Cap in all three files side by side (§6.2, p. 21). In XML the box is `812, 455, 871, 509`; in COCO it's `[812, 455, 59, 54]`; in YOLO it's `1 0.5259 0.4017 0.0369 0.0450`. Those are the conversions of concept 4.4.

WORKED EXAMPLE: WHAT `READ_VOC` DOES WITH ONE FILE

`ET.parse(xml_path).getroot()` reads the XML into a tree of tags. `root.find("size")` gives the recorded width and height, here (1600, 1200). `root.iter("object")` visits every object block. For each one it cleans the class name (concept 6.4) and reads the four `bndbox` numbers. The result is `("img_012.jpg", (1600, 1200), ["Bottle Cap"], [[812.0, 455.0, 871.0, 509.0]])`.

Pairing labels with images. `load_records` matches each `.xml` to the image with the same file name, in any sub-folder. Label files with no image are counted and reported rather than silently dropped, so you notice if something is wrong.

IN YOUR PROJECT

Step 2 prints "163 labelled images, ... boxes, 0 label files without an image". If you see 4,000+ images, `ORIG_DIR` points at the augmented folder, and if you see 0, at the wrong folder. See Step 2 (p. 32) and the matching rows in ch. 13 (p. 45).

6.3 EXIF orientation: the silent rotation bug

IN THE GUIDE §6.3 p. 21 §2.6 p. 9 Q5 p. 41

In one line. Phones often save a photo's pixels sideways and add a note, "rotate this before showing it". If your code and the labelling tool disagree about that note, every box lands in the wrong place, and nothing reports an error.

A JPEG file holds the pixels plus **metadata** called **EXIF**: camera model, date, exposure and an **orientation flag**. A phone's sensor is fixed in the body, so when you hold the phone upright the sensor still records a landscape image. Rather than rotating millions of pixels, the phone often stores them as recorded and sets the flag to, say, "rotate 90° clockwise when displaying".

Photo viewers obey the flag, so the photo looks upright. But some libraries and labelling tools ignore it. Now there are two versions of the same photo: the **raw** pixels (sideways) and the **upright** view. If the boxes were drawn on one version and your code loads the other, every box is misplaced. Training still runs and the loss still goes down a little, but the model is learning to find cans in empty corners.

WORKED EXAMPLE: HOW THE NOTEBOOK DECIDES

A phone stores 4000×3000 raw pixels with a "rotate 90°" flag, so the upright view is 3000×4000 . The XML's `<size>` tells us which one the annotator saw:

- XML says $3000 \times 4000 \rightarrow$ the labels match the upright view $\rightarrow$ use `ImageOps.exif_transpose`, which applies the flag.
- XML says $4000 \times 3000 \rightarrow$ the labels were drawn on the raw pixels $\rightarrow$ use the raw pixels.
- XML says something else, such as $750 \times 1000 \rightarrow$ the labels were drawn on a smaller copy of the upright photo $\rightarrow$ scale the boxes (concept 6.4).

The final safety net is looking: `label_check.png` draws the boxes on eight photos. If they sit on the objects, the orientation is right.

IN YOUR PROJECT

This is the first branch of `make_clean_copy` in Step 2, and pitfall 1 in §2.6 (p. 9). The printed note counts ("labels match the raw (unrotated) pixels", "boxes rescaled") are worth one sentence in your report's Dataset section. Viva Q5 (p. 41).

6.4 Cleaning the labels

IN THE GUIDE §6.3 p. 21 Step 2 p. 32 ch. 13 p. 45 Q50 p. 44

In one line. Before training, every photo is made upright and resized, and its boxes are adjusted to match. Impossible boxes are removed, and class names are made consistent.

Four fixes, in the order `make_clean_copy` applies them after the orientation check:

1. **Rescale boxes drawn on a resized copy.** Multiply x-coordinates by (real width ÷ recorded width) and y-coordinates by (real height ÷ recorded height).
2. **Resize big photos** so the long side is at most 1,024 px, keeping the aspect ratio. Every box coordinate is multiplied by the same scale factor.
3. **Clip** each box to the image, so no coordinate is below 0 or beyond the edge.
4. **Drop** any box less than 2 px wide or tall after clipping. A zero-size box has no inside, and torchvision stops with "All bounding boxes should have positive height and width".

WORKED EXAMPLE: THE GUIDE'S BOTTLE CAP

The photo is 1600 × 1200, so the scale is $1024 \div 1600 = 0.64$ and the clean copy is 1024 × 768. The box (812, 455, 871, 509) becomes $(812 \times 0.64, 455 \times 0.64, 871 \times 0.64, 509 \times 0.64) = (519.7, 291.2, 557.4, 325.8)$, about 38 × 35 px.

Rescaling: had the XML recorded 800 × 600 for this 1600 × 1200 photo, every coordinate would first be doubled: a recorded (406, 228, 436, 255) becomes (812, 456, 872, 510).

Clipping: a box (-3, 40, 60, 90) becomes (0, 40, 60, 90).

Class names. Annotators type names by hand, so "bottle_cap", "Bottle-Cap " and "bottle cap" can all appear.

`clean_name` replaces underscores and hyphens with spaces, squeezes the spaces and capitalises each word, so all three become "Bottle Cap". Without it, a typo creates a ninth class with a handful of examples. It can't fix every variant: "Cigarettes" stays different from "Cigarette". That's why Step 3 prints the class list: check by eye that exactly 8 appear.

Why a clean copy at all? Every later step (training, evaluation, drawing) then reads upright, reasonably sized photos with boxes that are guaranteed to fit. The fixes happen once, in one place, where you can check them.

IN YOUR PROJECT

Step 2 prints how many photos needed each fix and how many invalid boxes were dropped. The "9 classes printed" and "positive height and width" rows of ch. 13 (p. 45) are the failures these fixes prevent. After augmentation, `SanitizeBoundingBoxes` does the same job for boxes that a crop has destroyed (concept 6.8, viva Q50, p. 44).

6.5 Exploring the data before training

IN THE GUIDE §6.4 p. 22 Step 3 p. 33 §9.3 p. 30 §8.5 p. 27

In one line. Before training anything, count and look: how many examples each class has, how big the objects are, how crowded the photos are, and whether the boxes really sit on the objects.

This is called **exploratory data analysis** (EDA). It takes minutes and prevents days of confusion, because it tells you what results to expect and catches broken labels before a training run is wasted. The guide lists four things to look at (§6.4, p. 22):

- **Boxes per class.** Rare classes get fewer lessons and have unreliable test scores (concepts 6.6 and 8.9).
- **Object size.** The notebook measures it as $\sqrt{(\text{box area} \div \text{image area})}$, roughly "the object's width as a fraction of the photo's width".
- **Boxes per image.** Crowded photos mean touching objects and NMS trouble (concept 4.11).
- **Boxes drawn on photos,** the visual check (concept 6.3).

WORKED EXAMPLE: TWO WAYS TO CALL AN OBJECT "SMALL"

The guide's Bottle Cap: $\sqrt{(59 \times 54 \div (1600 \times 1200))} = \sqrt{(3,186 \div 1,920,000)} = \mathbf{0.04}$. The notebook's rule calls anything under 0.1 small, so it is small (concept 9.4).

COCO's rule uses the area in pixels instead: under $32 \times 32 = 1,024 \text{ px}^2$ is small. In the 1024×768 clean copy the cap is $37.8 \times 34.6 \approx 1,305 \text{ px}^2$, so COCO calls it **medium**.

Both are fine, but they answer different questions: one is relative to the photo, the other is absolute pixels. Say which one you use (concept 8.5).

IN YOUR PROJECT

Step 3 prints the class table and the boxes per image, and saves `eda.png` (boxes per class and a histogram of object sizes) and `label_check.png`. The class table and the share of small objects go in your report's Dataset section (§11.1, p. 39).

6.6 Class imbalance between the waste classes

IN THE GUIDE §6.7 p. 23 Q8 p. 41 §11.4 p. 40

In one line. When some classes have many more examples than others, the model gets more practice on the common ones and usually does worse on the rare ones.

This is a different imbalance from the background flood in concept 4.10. That one is between objects and empty anchors; this one is between the eight waste classes. Suppose, just for illustration, that Cigarette Butt had 150 boxes and Cigarette Box 30. Every epoch the model would get five times more practice finding butts, and with only a handful of Cigarette Boxes in the test set, one extra hit or miss would swing that class's score wildly.

Common remedies, all suitable for the future-work section:

- **Oversampling:** show photos that contain a rare class more often, for example twice per epoch.
- **Class-weighted loss:** make a mistake on a rare class cost more, for example five times more in the example above.
- **More data:** photograph and label more examples of the rare classes. Usually the best fix.

This project deliberately does *none* of these, and reports results **per class** instead. In a comparison study, any rebalancing would be one more setting to tune fairly for all three models. Per-class tables keep the effect visible and honest.

IN YOUR PROJECT

Look at Step 3's class table next to your per-class results (`per_class.csv`): do the rarest classes have the lowest AP? That's a good observation for the report. Viva Q8 (p. 41) ("which classes are hardest?") usually comes back to size and rarity.

6.7 Splitting by photo, and the seed search

IN THE GUIDE §6.5 p. 22 D3 p. 7 Step 4 p. 33 Q4 p. 41

In one line. The 163 photos are divided 70 / 15 / 15 *by photo*, and the random seed is chosen so that every class appears in both the validation and the test set.

By photo, not by box. All the boxes of one photo go to the same set. If the Bottle in a photo were in the training set and the Can next to it in the test set, the model would already have seen the test photo during training: leakage (concept 1.11).

WORKED EXAMPLE: 114 / 24 / 25

70% of 163 = 114.1, rounded to **114**. 15% of 163 = 24.45, rounded to **24**. The test set gets the rest: $163 - 114 - 24 = 25$.

Why search over seeds? A purely random split can leave a rare class with no examples in the test set. Its AP would then be undefined, and its test score would say nothing. Making sure every class is represented in every part is a simple form of **stratifying** the split (in its full form, stratifying keeps the class proportions similar across the parts). With several classes per photo there's no simple formula for it, so the notebook uses a search:

1. Sort the photo ids (so every computer starts from the same order).
2. Shuffle them with seed 0, cut 114 / 24 / 25, and count each class's boxes in each of the three parts.
3. If every class has at least 2 boxes in every part (the small validation and test parts are the hard ones), keep this split. Otherwise try seed 1, 2, 3, ... (up to 2,000).
4. If no seed achieves 2, repeat the search asking for at least 1.

The chosen split and its seed are saved to `split.json` in Drive. From then on the notebook loads that file instead of splitting again, so everyone on the team uses exactly the same photos, and results from different people can be compared.

IN YOUR PROJECT

`make_split` in Step 4. The data lead shares `split.json` *before* anyone else runs Step 4 (§1.3, p. 5). The printout should show 114 / 24 / 25 images and every class in all three lines. Why 15% for testing instead of the dataset's 10%? A larger test set gives a steadier estimate (concept 8.9, decision D3, p. 7). Viva Q4 (p. 41).

6.8 Augmentation for detection

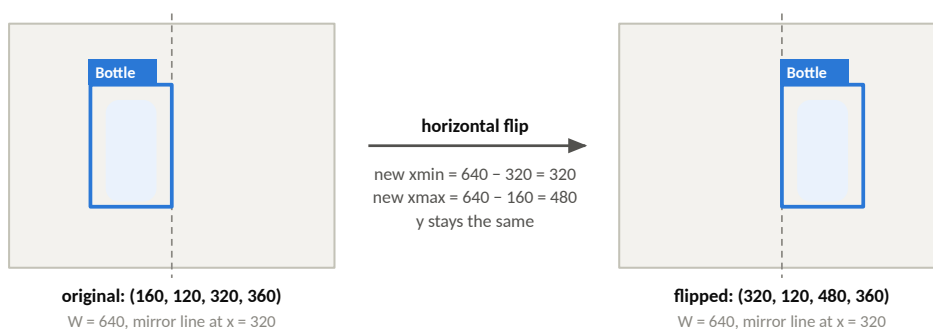
IN THE GUIDE §6.6 p. 22 D4 p. 7 Step 5 p. 33 Q50 p. 44

In one line. Augmentation shows the model a slightly different random version of each training photo every epoch. In detection, the boxes must be transformed together with the pixels.

With 114 photos, the model would soon memorise them (concept 1.9). **Augmentation** makes every epoch look a little different: brighter or duller, zoomed in or out, mirrored. The model can't memorise pixels that keep changing, so it has to learn what actually makes a can a can. The changes are made **on the fly**, freshly each time a photo is loaded. Over 30 epochs that's 30 variations of each training photo, and the validation and test photos are never touched.

There are two kinds of change:

- **Photometric** changes alter colours but not positions, so the boxes stay as they are.
- **Geometric** changes move pixels (flips, zooms, crops), so **every box must move in exactly the same way**, or the labels stop matching the objects.



A horizontal flip: new xmin = W - xmax, new xmax = W - xmin, and y stays the same.

WORKED EXAMPLE: FLIPPING THE BOTTLE FROM CONCEPT 4.4

Image width W = 640, box (160, 120, 320, 360). New xmin = 640 - 320 = 320, new xmax = 640 - 160 = 480, so the flipped box is (320, 120, 480, 360). The old *right* edge becomes the new left edge.

The pipeline in Step 5, one transform at a time (§6.6, p. 22):

Transform	What it actually does
<code>RandomPhotometricDistort(p=0.5)</code>	Five colour changes, each with a 50% chance: brightness ($\pm 12.5\%$), contrast and saturation ($\times 0.5$ to $\times 1.5$), a small hue shift, and reordering the red, green and blue channels. Boxes unchanged.
<code>RandomZoomOut</code> (30%)	Places the photo on a canvas up to 1.5 times bigger, filled with an average colour, so objects end up smaller, as if further away. Boxes shift with the photo.
<code>RandomIoUCrop</code> (30%)	Cuts out a region of 30–100% of the width and height that contains the centre of at least one object and overlaps it by a randomly drawn minimum IoU (0 to 0.9); one time in seven, no crop. Objects look bigger. Boxes are cut to the new edges; boxes whose centre falls outside are marked for removal.
<code>RandomHorizontalFlip</code> (50%)	Mirrors left and right (the example above).
<code>SanitizeBoundingBoxes</code>	Always. Removes boxes that the crop marked or shrank below 2 px, together with their labels.
<code>ToDtype(float32, scale=True)</code>	Always. Turns 0–255 whole numbers into 0–1 decimals (concept 2.4).

What we avoid. Large rotations (an upright box around a rotated object is bigger than the object, so labels get loose), vertical flips (photos have a clear "up", unless taken straight down), and any augmentation of validation or test photos, which must measure the model on real, unaltered photos.

IN YOUR PROJECT

`train_tf` is this pipeline; `eval_tf` only converts to 0–1. In Step 5's preview the boxes must stay on the objects. It replaces the 4,923 pre-made copies (decision D4, p. 7); concept 10.6 shows how boxes are recognised. Viva Q50 (p. 44).

Training in practice

Parts 1 and 2 explained the ingredients of training: loss, gradients, AdamW, the schedule, mixed precision and clipping. This part puts them together in the order the notebook runs them, following guide chapter 7 and Step 6: one iteration, the two modes of a model, checkpoints, the training curves, and what makes the comparison fair.

7.1 One training iteration, line by line

IN THE GUIDE §7.2 p. 24 §3.4 p. 10 Step 6 p. 35

In one line. Every iteration does the same seven things: take a batch, move it to the GPU, run the model to get its losses, add them up, backpropagate, update the weights, and advance the learning rate.

Here is the inner loop of `train_model`, with the guide's seven steps from §7.2 (p. 24) marked in the comments:

```
for imgs, tgts in loader:                # 1 a batch: 4 images, 4 target dicts
    imgs = [im.to(DEVICE) for im in imgs] # 2 move everything to the GPU
    tgts = [{"boxes": t["boxes"].to(DEVICE), "labels": t["labels"].to(DEVICE)}
            for t in tgts]
    with torch.autocast(DEVICE.type, enabled=USE_AMP):
        losses = model(imgs, tgts)        # 3 forward pass: returns a dict of losses
        loss = sum(losses.values())        # 4 one number to minimise ...
        if not torch.isfinite(loss):       # ... stop at once if it is inf or nan
            raise RuntimeError(...)
        opt.zero_grad(set_to_none=True)    # 5 forget the previous gradients
        scaler.scale(loss).backward()       # backpropagation, loss scaled for fp16
        scaler.unscale_(opt)               # 6 undo the scaling ...
        torch.nn.utils.clip_grad_norm_(params, max_norm=10.0) # ... cap the gradient size
        scaler.step(opt)                   # AdamW updates the weights
        scaler.update()                    # adjust the loss-scaling factor
        sched.step()                       # 7 move to the next learning rate
```

Where each line comes from: the batch (concept 1.6, and concept 10.4 for the loader), the GPU (2.7), the loss dictionary (4.2, 4.3, 4.9 and Part 5 for the names), autocast and the scaler (2.8), backpropagation (1.5), clipping (2.9), AdamW (1.7) and the schedule (1.8).

Around the inner loop, once per epoch: `model.train()` switches to training mode before the batches (concept 7.2). Each loss part is averaged over the epoch for the log. After the last batch, `predict` runs the model on the 24 validation photos and `coco_map` scores them. If validation mAP is the best so far, the weights are saved (concept 7.3). The history is written to `results/history_<model>.json` and one line is printed.

READING ONE LOG LINE (THE FORMAT, NOT A REAL RESULT)

[RetinaNet] epoch 12/30 loss 0.415 val mAP 0.352 mAP50 0.561 (38.2s) <- best so far, saved
`loss` is the epoch's average training loss (the sum of the model's loss parts). `val mAP` is validation mAP@0.5:0.95 and `mAP50` is validation mAP@0.5 (concept 8.5). The time covers training and validation for that epoch. The arrow means this epoch's weights replaced the saved checkpoint.

IN YOUR PROJECT

One `train_model` serves all three models; only the `build` function passed in differs. That is what makes the comparison fair (concept 7.5). Typical times on a T4 are in §7.4 (p. 25): roughly 15–25 minutes each for Faster R-CNN and RetinaNet, about 5 for SSD300.

7.2 Training mode and evaluation mode

IN THE GUIDE §7.2 p. 24 Step 6 p. 35 Step 7 p. 36 Q30 p. 43

In one line. A model has two modes. `model.train()` is for learning: torchvision detectors then return losses. `model.eval()` is for using the model: they return boxes.

Switching the mode changes several things at once:

	<code>model.train()</code>	<code>model.eval()</code>
You call	<code>model(images, targets)</code>	<code>model(images)</code>
It returns	a dict of losses, e.g. <code>{"classification": ..., "bbox_regression": ...}</code>	a list with one dict per image: <code>boxes, labels, scores</code>
Anchor matching and sampling	done, to build the targets for the loss (concept 4.7)	not done
Faster R-CNN proposals	2,000 per image	1,000 per image
BatchNorm (concept 2.5)	uses the current batch's statistics and updates its running averages	uses the stored running averages, so an image's result doesn't depend on the other images in its batch

`torch.no_grad()` is a separate switch. It tells PyTorch not to record operations for backpropagation (concept 1.5). When you're only predicting, there will be no `backward()`, so recording would just waste memory and time. `predict` and `time_model` are decorated with `@torch.no_grad()`, and `predict.py` uses `with torch.no_grad():`.

A classic mistake. Calling `model(images)` without targets while the model is still in training mode makes torchvision's detectors stop with "targets should not be none when in training mode". In an ordinary network, such as an image classifier, the same slip would be silent: BatchNorm would quietly use batch statistics and give slightly different, batch-dependent results. That's why `predict` starts with `model.eval()` and `train_model` calls `model.train()` at the start of every epoch.

IN YOUR PROJECT

Training mode in the inner loop of Step 6; evaluation mode in `predict` (validation after every epoch, and the test set in Step 7), in `time_model`, and in `load_trained`, which ends with `.eval()`. Viva Q30 (p. 43) asks exactly this.

7.3 Checkpoints and keeping the best epoch

IN THE GUIDE §7.1 p. 24 §7.2 p. 24 ch. 13 p. 45 Q26 p. 43

In one line. After every epoch the model is scored on the validation photos, and its weights are saved only if that score is the best so far. The file you end up with holds the best epoch, not the last one.

A **checkpoint** is a file holding the model's weights (concept 1.2; concept 10.8 covers the file itself). Saving only on improvement is early stopping without having to guess the number of epochs in advance (concept 1.9). Training simply runs all 30 epochs, and later epochs that overfit are never used.

WORKED EXAMPLE (ILLUSTRATION)

Validation mAP after epochs 1 to 6: 0.21, 0.30, 0.34, 0.33, 0.36, 0.35.

The file is written after epoch 1 (0.21 is the first score), epoch 2 (0.30 > 0.21), epoch 3 (0.34 > 0.30) and epoch 5 (0.36 > 0.34). Epochs 4 and 6 were worse, so they aren't saved. If nothing later beats 0.36, the final checkpoint holds **epoch 5**.

Why judge by validation mAP@0.5:0.95? It is the quantity you actually care about: finding objects with well-fitting boxes (concept 8.5). The training loss keeps falling even while the model overfits, and it isn't comparable between models.

The price of choosing. Because you picked the epoch that scored best on the validation photos, that validation score is slightly flattering: some of the peak is luck. That's why the honest final number comes from the test set, used once (concept 1.10).

Resuming. If Colab disconnects, re-running Step 6 skips any model whose history has all 30 epochs and whose checkpoint exists (the `finished` function). An interrupted model starts again from epoch 1. Continuing mid-way would also need the optimizer's and the scheduler's saved state, which the notebook doesn't keep, to keep things simple.

IN YOUR PROJECT

`torch.save(model.state_dict(), ckpt)` in `train_model`. The dots in `training_curves.png` mark each model's saved epoch. Note the best epochs in your decision log: viva Q26 (p. 43) asks how you chose the number of epochs. The "Colab disconnected" row of ch. 13 (p. 45) describes the resume behaviour.

7.4 Reading the training curves

IN THE GUIDE [§7.3 p. 25](#) [Step 6 p. 35](#) [Q27 p. 43](#) [Q29 p. 43](#)

In one line. Two curves tell you how training went: the training loss (is it learning?) and the validation mAP (is what it learns useful on new photos?). Their shapes tell you which of five situations you're in.

Why the loss is plotted relative to epoch 1. Each model's loss is a different sum of different parts (concept 1.3), so raw values can't share an axis. Dividing each model's losses by its own first-epoch value makes every curve start at 1. Then only the shapes are compared: 0.35 means "the loss has fallen to 35% of where it started".

WORKED EXAMPLE

A model's loss is 1.20 in epoch 1 and 0.42 in epoch 30, so the chart shows $1.20 \div 1.20 = 1.00$ at the start and $0.42 \div 1.20 = 0.35$ at the end. Another model going from 3.0 to 1.05 also ends at 0.35. The same relative progress, even though 1.05 and 0.42 look very different.

Pattern	What the two curves do	What it means
Healthy	Loss falls fast, then slowly. Val mAP rises, then levels off.	Learning and generalising.
Overfitting	Loss keeps falling. Val mAP peaks, then slides.	Memorising the training photos (concept 1.9). The saved best epoch protects you.
Under-trained	Val mAP still clearly rising at epoch 30.	More epochs would help, for all three models.
Unstable	Loss jumps up and down, or becomes <code>nan</code> .	Steps too big, 16-bit overflow or bad boxes (concepts 1.4, 2.8, 2.9).
Stuck	Val mAP near 0 after 5 or more epochs.	Almost always a data bug: misplaced boxes or wrong labels (concept 10.10).

Expect some wobble. With only 24 validation photos, val mAP can move by a couple of points between epochs by chance. Look at the trend over several epochs, not at single steps.

IN YOUR PROJECT

`training_curves.png` from Step 6 has exactly these two panels. The guide's table of patterns and actions is in §7.3 (p. 25). In the report, say which pattern each model showed and where its best epoch was: that's your answer to viva Q27 (p. 43) ("how do you know you didn't overfit?"), and the relative-loss point is Q29 (p. 43).

7.5 A fair experiment: change only the model

IN THE GUIDE [D5 p. 7](#) [§2.3 p. 8](#) [§2.5 p. 8](#) [§7.1 p. 24](#) [§11.4 p. 40](#)

In one line. To say that one model is better than another, everything except the model must be the same. Otherwise you can't tell whether the design or the extra care made the difference.

This is the logic of a **controlled experiment**, as in a school science lab. You change one thing on purpose (the **independent variable**, here the detector design), measure the outcome (mAP, precision, recall, speed), and hold everything else fixed (the **controlled variables**). Anything that differs by accident is a **confounder**: a rival explanation for your results.

EXAMPLE OF A CONFOUNDER

Suppose Faster R-CNN had been trained for 40 epochs and RetinaNet for 30, and Faster R-CNN wins by 0.02 mAP. Is it the two-stage design or the extra 10 epochs? You can't tell. The same goes for a different learning rate, a different split, or thresholds picked in different ways.

The guide's list in §2.3 (p. 8) is the full set of controlled variables: the same photos, split, cleaning, augmentation, starting point (COCO weights), optimizer, learning rate, schedule, epochs, batch size, seed, post-processing, threshold rule, metric code and GPU. What's allowed to differ is exactly what defines each model: its architecture, backbone, input size, anchors and loss functions.

Tuning budget. If you tune settings for one model, you must give the others the same effort. The guide's suggestion in §7.1 (p. 24): try the same three learning rates (5e-5, 1e-4, 2e-4) for each model, and keep each one's best by validation mAP.

One run each. Training involves randomness (concept 1.13), so a second run with another seed would give slightly different numbers. With one run per model you can't measure that variation, which is one more reason not to crown a winner on a tiny gap (concept 8.9). Say so under limitations (§11.4, p. 40).

IN YOUR PROJECT

This is decision D5 (p. 7). Write your hypotheses down *before* seeing results (§2.5, p. 8), then report a verdict on each. A "fairness statement" in the Methods section, listing what was kept identical, heads off the examiner's first question.

Measuring detection quality

The brief asks for precision, recall and mAP, and guide chapter 8 builds them from one matching rule. This part goes slower: every metric is computed by hand on a small example, and each one is tied to the function in Step 7 that computes it for real.

8.1 Correct, false and missed: matching predictions to objects

IN THE GUIDE §8.1 p. 26 Step 7 p. 36 Q32 p. 43 Q33 p. 43

In one line. Predictions are matched to true objects in order of confidence. A prediction is correct (TP) if it has the right class, $\text{IoU} \geq 0.5$ with a true box, and that box hasn't been claimed yet; every other prediction is false (FP), and every unclaimed object is missed (FN).

Before anything can be counted, each prediction must be declared right or wrong. The rule, used by PASCAL VOC, COCO and the notebook:

1. Sort the photo's predictions by score, highest first.
2. Take the next prediction. Among the true boxes **of the same class** that are **still unclaimed**, find the one it overlaps most.
3. If that IoU is at least 0.5, it's a **true positive (TP)** and it claims that box. Otherwise it's a **false positive (FP)**.
4. When all predictions are done, every true box nobody claimed is a **false negative (FN)**: a missed object.

WORKED EXAMPLE: ONE PHOTO

True objects: G1 Bottle, G2 Bottle, G3 Can. Four predictions, already sorted:

Prediction	Class, score	Best unclaimed same-class box	Result
P1	Bottle, 0.95	G1, IoU 0.82	TP, claims G1
P2	Bottle, 0.88	G1 is taken; G2 only IoU 0.05	FP
P3	Can, 0.74	G3, IoU 0.58	TP, claims G3
P4	Bottle, 0.52	G2, IoU 0.41 (below 0.5)	FP

G2 was never claimed, so it's an **FN**. Totals: TP = 2, FP = 2, FN = 1. (P2 is a second box on an object that was already found, and P4 found G2 but drew a poor box. Part 9 gives these errors names.)

Why highest score first? The most confident prediction gets the first claim, which is fair. It also has a useful side effect: raising the confidence threshold only removes predictions from the bottom of the list and never changes the matches above them. So the notebook matches each photo once and reuses the result for every threshold.

No true negatives, so no accuracy. A true negative would be "a place with no object, correctly left without a box". But how many such places does a photo have? Every possible rectangle is a candidate, so the count is endless and meaningless. Accuracy needs that count, so detection uses precision and recall, which don't (viva Q33, p. 43).

IN YOUR PROJECT

`match_image` (Step 7) is this rule in code. `cand[(gt["labels"] != pred["labels"][i]) | taken] = -1` removes other-class and already-claimed boxes from the candidates, then `cand.max() >= iou_thr` checks the 0.5. `match_all` sorts each photo's predictions and runs it once. Viva Q32 (p. 43).

8.2 Precision, recall and F1

IN THE GUIDE §8.2 p. 26 App. A p. 46 Q34 p. 43

In one line. Precision: of the boxes we drew, what share was right? Recall: of the objects that were there, what share did we find? F1 combines the two, and it is only high when both are high.

$$\text{Precision} = \frac{TP}{TP+FP} \quad \text{Recall} = \frac{TP}{TP+FN} \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Note the denominators. TP + FP is everything the model *drew*. TP + FN is everything that was really *there*. A fishing analogy: precision is the share of your catch that is the fish you wanted, and recall is the share of those fish in the lake that you caught. A huge net catches most of the fish (high recall) and a lot of junk (low precision).

WORKED EXAMPLE

On a set of test photos a model makes 18 boxes, 15 of them correct, and there are 20 objects in total. So TP = 15, FP = 3, FN = 20 – 15 = 5.

Precision = 15 ÷ 18 = **0.833**. Recall = 15 ÷ 20 = **0.750**. F1 = 2 × 0.833 × 0.750 ÷ (0.833 + 0.750) = **0.789**.

The photo in concept 8.1 gives P = 2 ÷ 4 = 0.50, R = 2 ÷ 3 = 0.67 and F1 = 0.57.

Why a harmonic mean? A model that draws one perfect box and misses everything else has P = 1.0 and R = 0.1. The ordinary average, 0.55, looks respectable. F1 = 2 × 1.0 × 0.1 ÷ 1.1 = 0.18 exposes it: the harmonic mean is dragged down by whichever of the two is worse.

IN YOUR PROJECT

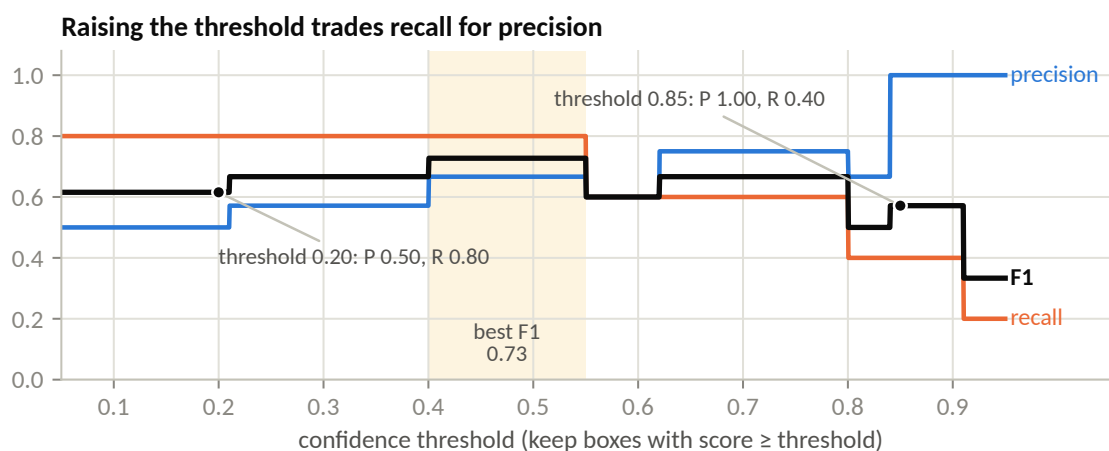
`prf_at(rows, thr)` counts TP, FP and FN over all test photos, overall and per class, at a given threshold, and computes P, R and F1. They're the Precision, Recall and F1 columns of your results table. What low recall and low precision mean on a sorting line is viva Q34 (p. 43) and the paragraph in §8.2 (p. 26).

8.3 The confidence threshold, and choosing it fairly

IN THE GUIDE §8.3 p. 26 D6 p. 7 §2.6 p. 9 Q37 p. 43

In one line. Precision and recall depend on which boxes you keep. Raising the confidence threshold keeps fewer, surer boxes: precision rises and recall falls. Each model's threshold is chosen where F1 is best on the validation set.

Every predicted box has a score. To count precision and recall you must decide which boxes to keep, and the **confidence threshold** is that cut-off. It's a dial, not a fixed property of the model. Turn it up and only very confident boxes remain: few false alarms (precision up), but more objects left without a box (recall down). Turn it down and the opposite happens.



Precision, recall and F1 for the guide's worked example (concept 8.4) as the threshold moves. F1 peaks for thresholds from just above 0.40 up to 0.55.

WORKED EXAMPLE: THE GUIDE'S NUMBERS (§8.3, P. 26)

Five objects, eight detections with scores 0.95, 0.91, 0.84, 0.80, 0.62, 0.55, 0.40 and 0.21 (the table in concept 8.4).

Threshold 0.85: keeps 0.95 and 0.91, both correct. $P = 2 \div 2 = 1.00$, $R = 2 \div 5 = 0.40$, $F1 = 0.57$.

Threshold 0.55: keeps six boxes, 4 correct. $P = 4 \div 6 = 0.67$, $R = 4 \div 5 = 0.80$, $F1 = 0.73$, the best.

Threshold 0.20: keeps all eight, still 4 correct. $P = 0.50$, $R = 0.80$, $F1 = 0.62$.

The notebook tries 0.05, 0.10, ..., 0.95. Here 0.45, 0.50 and 0.55 all give the best F1 (they keep the same six boxes), and `best_threshold` returns the first of them, 0.45.

Why not one threshold for all three models? Scores aren't on a common scale. A model is **calibrated** if, among all the boxes it scores about 0.7, about 70% are right. Detectors usually aren't, and each is off in its own way: focal-loss models such as RetinaNet tend to give lower scores. A fixed 0.5 might suit one model and cripple another. Choosing each model's best-F1 threshold puts all three at their own best balance.

Why on validation? The threshold is a choice, and choices made on the test set leak into the test score (concepts 1.10 and 1.11). So it is chosen on the 24 validation photos, fixed, and only then applied to the test photos.

IN YOUR PROJECT

`thr = best_threshold(val_rows)` in Step 7, then `prf_at(rows, thr)` on the test rows. The Threshold column of your results table shows the values, and they will differ between models. "Always state the threshold next to P and R" is pitfall 7 in §2.6 (p. 9). Decision D6 (p. 7), viva Q37 (p. 43).

8.4 The precision–recall curve and Average Precision (AP)

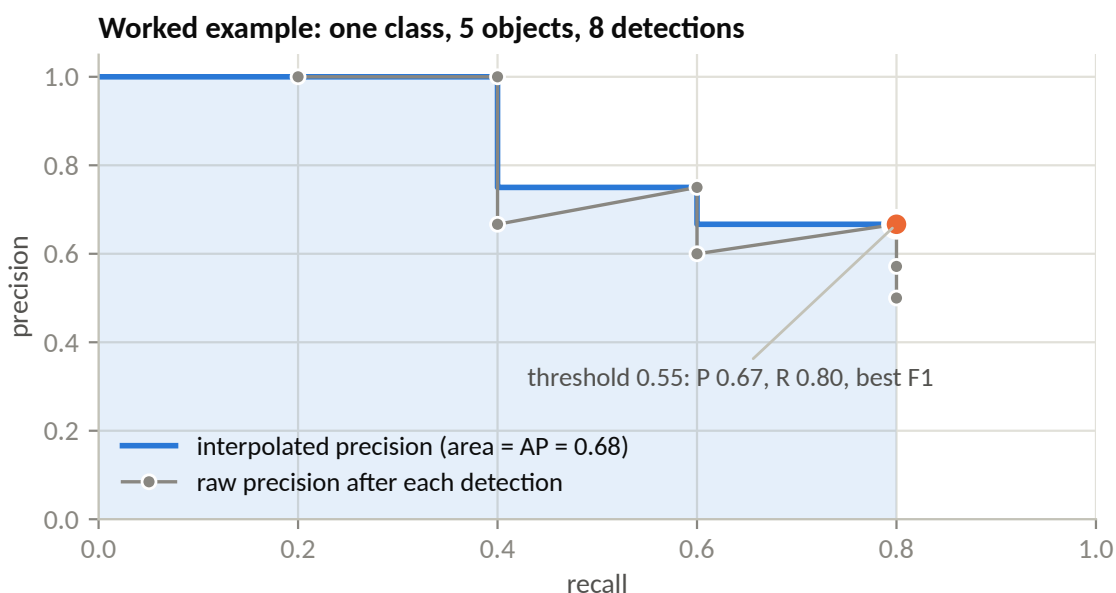
IN THE GUIDE §8.4 p. 26 App. A p. 46 Q35 p. 43

In one line. Instead of picking one threshold, sweep all of them. AP is the area under the resulting precision–recall curve: one number for how good the model's ranking of boxes is, from the most confident to the least.

Sort all of a class's detections (over all test photos) by score. Walk down the list; after each detection, work out the precision and recall of everything so far. Each row is what you'd get with the threshold set just at that detection's score. This is the guide's worked example: one class, 5 objects, 8 detections.

Rank	Score	Match	Cum. TP	Cum. FP	Precision	Recall
1	0.95	TP	1	0	1.00	0.20
2	0.91	TP	2	0	1.00	0.40
3	0.84	FP	2	1	0.67	0.40
4	0.80	TP	3	1	0.75	0.60
5	0.62	FP	3	2	0.60	0.60
6	0.55	TP	4	2	0.67	0.80
7	0.40	FP	4	3	0.57	0.80
8	0.21	FP	4	4	0.50	0.80

Reading down: rank 3 is a false positive, so precision drops from 1.00 to 0.67 while recall stays at 0.40. Rank 4 is correct again, so recall rises to 0.60 and precision recovers to 0.75. That's why the raw curve zig-zags.



The same chart as guide §8.4 (p. 26). Grey: raw precision after each detection. Blue: interpolated precision. The shaded area is the AP.

Interpolation. The zig-zags make "the area" awkward, so AP uses the **interpolated precision**: at each recall level, the best precision reached at that recall *or any higher recall*. The reasoning: if going further down the list could give you more recall *and* better precision, nobody would stop at the worse point. This turns the zig-zag into a staircase that only goes down.

WORKED EXAMPLE: AP FROM THE TABLE

Recall rises at ranks 1, 2, 4 and 6, each time by 0.2. The interpolated precision at those points is 1.00, 1.00, 0.75 and 0.67 (at recall 0.8, the best precision at recall 0.8 or more is 0.67, from rank 6).

$AP = 0.2 \times 1.00 + 0.2 \times 1.00 + 0.2 \times 0.75 + 0.2 \times 0.67 = 0.68$.

One object was never found, so recall never passes 0.8, and AP can't exceed 0.8. That's how misses lower AP.

COCO's version reads the staircase at 101 fixed recall levels, 0, 0.01, 0.02 ... 1, and averages them. For this example that gives 0.686 instead of 0.683: slightly different bookkeeping, the same idea.

What AP rewards. High AP needs correct boxes to have *higher scores* than wrong ones. A model whose true detections all score 0.9 and whose false alarms all score 0.3 has an excellent ranking, whatever threshold you later choose. AP judges the whole ranking, so it doesn't depend on picking a threshold at all.

IN YOUR PROJECT

`pr_curve` builds the curve and `average_precision` takes the area with exactly this interpolation. Step 7 draws `pr_curves.png`, with a dot at each model's chosen threshold; its legend's AP50 pools all classes into one curve (concept 8.7). The formula is in App. A (p. 46); viva Q35 (p. 43).

8.5 The mAP family

IN THE GUIDE §8.5 p. 27 §2.1 p. 6 App. A p. 46 Q36 p. 43

In one line. mAP is the average of AP over the classes. The number after the @ is the IoU a box needs to count as correct, and mAP@0.5:0.95 averages over ten such IoU levels, so it also rewards tight boxes.

From AP to mAP. AP is computed separately for each class, then averaged: with APs of 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3 and 0.2 for the eight classes, **mAP** = 0.55. Every class counts equally, however many objects it has, so a rare class with a poor AP pulls the mean down as much as a common one.

The IoU bar. In concept 8.1, a prediction needed $\text{IoU} \geq 0.5$ to count as correct. That bar can be moved:

- **mAP@0.5** (the classic PASCAL VOC metric): the box only has to overlap well. "Did it find the objects?"
- **mAP@0.75**: much stricter. "Are the boxes tight?"
- **mAP@0.5:0.95** (the main COCO metric): compute mAP at IoU 0.50, 0.55, 0.60, ... 0.95 (ten levels) and average them. It rewards finding objects *and* fitting the boxes tightly.

WORKED EXAMPLE: ONE BOX, TEN BARS

A single object and a single detection with IoU 0.72. It counts as correct at the bars 0.50, 0.55, 0.60, 0.65 and 0.70 (AP = 1 at each) and as wrong at 0.75 to 0.95 (AP = 0). Its mAP@0.5 is 1.0, but its mAP@0.5:0.95 is $5 \div 10 = 0.5$. With IoU 0.93 it would score $9 \div 10 = 0.9$. That's why mAP@0.5:0.95 is always the lower number (viva Q36, p. 43), and why the gap between the two tells you about box tightness.

Other members of the family, all produced by the same COCO evaluation:

- **AR@100** (average recall): the share of objects found if up to 100 boxes per photo are allowed, averaged over the ten IoU levels. It asks how many objects the model can find at all, ignoring false alarms.
- **mAP small / medium / large**: mAP computed only on objects of one size, by area in pixels: small under $32 \times 32 = 1,024 \text{ px}^2$, medium up to $96 \times 96 = 9,216 \text{ px}^2$, large above. In our clean copies a $40 \times 40 \text{ px}$ cigarette butt is $1,600 \text{ px}^2$, which is *medium* by COCO's rule. A value of -1 means there were no objects of that size, so the number doesn't exist.

What to headline (§8.5, p. 27): mAP@0.5:0.95, because the brief mentions the quality of the bounding boxes, together with mAP@0.5 and the precision, recall and F1 at the chosen threshold.

IN YOUR PROJECT

`coco_map` returns `map` (mAP@0.5:0.95), `map_50`, `map_75`, `map_small`, `map_medium`, `map_large` and `mar_100` (AR@100), plus AP per class at 0.5:0.95 and at 0.5. The first two are the headline columns of Table R1 (§11.2, p. 39).

8.6 The confusion matrix for detection

IN THE GUIDE §8.6 p. 28 Step 8 p. 37 Q43 p. 44

In one line. A table of *true* class (rows) against *predicted* class (columns), with an extra "background" row and column for missed objects and false alarms.

In classification, every photo has exactly one true and one predicted class, so the table has only classes. In detection, an object can get no box at all, and a box can sit on nothing. So a **background** row and column are added. To fill the table, each kept box is paired with the true object it overlaps most ($\text{IoU} \geq 0.5$), *whatever its class*. That's different from the matching in concept 8.1, and it's what lets wrong-class boxes show up. Pairs are made best-overlap first.

WORKED EXAMPLE: READING A SMALL MATRIX (MADE-UP COUNTS)

true ↓ / predicted →	background	Bottle	Cup	Plastic
background	-	3	1	2
Bottle	4	18	3	1
Cup	2	2	9	0
Plastic	3	1	0	7

Diagonal: 18 bottles, 9 cups and 7 plastics were found with the right class.

Bottle row: of 26 true bottles, 3 were called Cup and 1 Plastic (right place, wrong class), and 4 got no box at all (the background column: missed).

Background row: 6 boxes were drawn on nothing: 3 "Bottles", 1 "Cup" and 2 "Plastics". These are false alarms.

The background-background cell is empty: that would be the uncountable true negatives (concept 8.1).

Bottle and Cup are confused both ways (3 and 2), a look-alike pair, as hypothesis H4 in the guide predicts.

IN YOUR PROJECT

`confusion` in Step 8 fills the matrix at each model's threshold, and `confusion_matrices.png` shows one per model. The numbers are counts, and the shading is each row's share, so small classes stay readable next to the big background row (§8.6, p. 28). Which classes were confused, and why, is viva Q43 (p. 44).

8.7 COCO evaluation with torchmetrics, and why we also count by hand

IN THE GUIDE §8.4 p. 26 Step 7 p. 36 ch. 13 p. 45 Q40 p. 44

In one line. `torchmetrics` computes the standard COCO numbers exactly as published papers do. The notebook's own matching code adds what COCO doesn't give: counts at your chosen threshold, and a reason for every error.

torchmetrics is a library of ready-made, tested metrics. Its `MeanAveragePrecision` implements the whole COCO protocol: matching at ten IoU levels, the 101-point AP, the size groups and the 100-box limit per photo. Behind the scenes it calls **pycocotools**, the official COCO evaluation code. Using it means your mAP is computed the same way as the numbers in papers and in torchvision's model table.

How it's used, in two calls:

1. `metric.update(preds, gts)`: hand over two lists with one dict per photo. Predictions have `boxes`, `scores` and `labels`; ground truth has `boxes` and `labels`. Boxes in pixels, xmin-ymin-xmax-ymax (concept 4.4).
2. `metric.compute()`: returns a dict with `map`, `map_50`, `map_75`, the size-group values, `mar_100` and, with `class_metrics=True`, AP per class.

Why also count by hand? COCO's mAP sweeps over all thresholds, so it never says "at threshold 0.45 there were 38 TP's, 9 FP's and 12 FN's". It also never explains *why* a box was wrong. The notebook's own functions (`match_image` , `prf_at` , `explain_errors`) supply both. Two independent implementations have an extra benefit: they check each other. When the notebook's matching code was tested, the AP@0.5 it gave, averaged over the classes like COCO's, agreed with the COCO-style value to within 0.001 (§8.4, p. 26).

Don't be caught out by the PR-curve legend. The AP50 printed in the legend of `pr_curves.png` pools all classes into a single curve. COCO's mAP@0.5 in your results table is the average of eight per-class APs. The two are different quantities, so they won't be equal. If asked, that is the reason.

IN YOUR PROJECT

`coco_map` wraps `torchmetrics`; it runs every epoch on validation (Step 6) and once on test (Step 7). If `torchmetrics` complains about `pycocotools`, install it and restart the runtime (ch. 13, p. 45). "Why both?" is viva Q40 (p. 44).

8.8 Speed and size

IN THE GUIDE §8.7 p. 28 §5.4 p. 19 D7 p. 7 Q38 p. 43

In one line. Speed is the time to process one photo (latency), measured carefully on the GPU. Size is the number of weights and the checkpoint file. Compute (GFLOPs) explains the speed differences independently of the hardware.

- **Latency:** milliseconds for one photo, sent alone (batch size 1), including the model's own resizing, the network and NMS, but not reading the file from disk.
- **FPS** (frames per second) = $1,000 \div \text{latency in ms}$. 40 ms per photo is 25 FPS.
- **Parameters:** the number of weights (concept 1.2). **Size:** the checkpoint file. Each weight is a 32-bit float, 4 bytes, so 43.3 million parameters $\times$ 4 bytes $\approx$ 173 MB, the " $\approx$ 174 MB" in the guide.
- **FLOPs:** floating-point operations, the arithmetic needed per photo. **GFLOPs** are billions of them. They don't depend on the hardware, so they explain *why* one model is faster.

WORKED EXAMPLE: WHERE THE ARITHMETIC GOES

One 3×3 convolution with 256 input and 256 output channels, on Faster R-CNN's finest level ($200 \times 272 = 54,400$ positions), needs $3 \times 3 \times 256 \times 256 = 589,824$ multiply-adds per position, $\times 54,400$ positions $\approx$ **32 billion** multiply-adds. That's one layer on one level. The work grows with the number of pixels, which is why an 800-px input costs so much more than SSD300's 300×300 .

Measuring latency properly takes three precautions:

1. **Warm up.** The first few runs are slow while the GPU allocates memory and picks its fastest algorithms. The notebook runs 3 photos first and doesn't time them.
2. **Synchronise.** The GPU works *asynchronously* (concept 2.7): Python queues up the work and moves on at once. A stopwatch around `model(x)` would measure only the queuing. `torch.cuda.synchronize()` waits until the GPU has really finished, so the notebook calls it before starting and before stopping the timer.
3. **Take the median** of 20 runs. The median is the middle value, so one hiccup can't distort it: times of 44, 45, 45, 46 and 120 ms have a mean of 60 but a median of 45.

Without synchronize

CPU (Python)



the timer stops here: 3 ms, wrong

GPU



the model actually runs: 45 ms

With torch.cuda.synchronize()

CPU (Python)



the timer measures 47 ms: right

waits until the GPU is done

GPU



the model actually runs: 45 ms

Why the timer needs `torch.cuda.synchronize()`. The numbers are illustrative.

Parameters are not speed. Size decides how much memory and disk a model needs; FLOPs and the implementation decide how fast it runs (concept 5.4). A laptop CPU is many times slower than a T4, so say so if you demo live.

IN YOUR PROJECT

`time_model` in Step 7 does exactly the three steps above, with mixed precision as in training. The ms / image, Params (M) and Size (MB) columns of your table come from it, from `sum(p.numel() ...)` and from the file size. The speed-accuracy plot in `comparison.png` puts them together. Decision D7 (p. 7) explains why speed matters; viva Q38 (p. 43).

8.9 How much to trust the numbers

IN THE GUIDE §8.8 p. 28 §2.5 p. 8 §11.4 p. 40 Q39 p. 44

In one line. 25 test photos are a small sample, so every number has some luck in it. Trust clear gaps and patterns that repeat, not small differences.

Toss a coin 10 times and get 7 heads: you wouldn't call the coin biased, because with so few tosses that happens easily by chance. A test set is the same. A different 25 photos would give somewhat different numbers, and your results are one draw.

WORKED EXAMPLE: ONE OBJECT, A BIG SWING

Suppose a class has only 4 objects in the test set. Finding 3 of them gives recall 0.75; finding all 4 gives 1.00. A single object moves that class's recall by 25 points, and its AP by a similar amount. That's why the guide warns that one object can move a class's AP by 10 points or more (§8.8, p. 28).

How to be careful in writing: say "on our 25-photo test set"; don't declare a winner from a one- or two-point gap; look for results that agree across mAP@0.5, mAP@0.5:0.95, recall, several classes and several object sizes.

Two standard ways to measure the uncertainty, good future-work items:

- **Bootstrap confidence interval.** Draw 25 photos at random *with replacement* from the test set (some appear twice, some not at all) and recompute mAP. Repeat 1,000 times. The middle 95% of the 1,000 values is a 95% **confidence interval**, for example "mAP 0.41 (0.33 to 0.48)". If two models' intervals overlap heavily, the data can't separate them.
- **k-fold cross-validation.** Split all 163 photos into, say, 5 parts. Train 5 times, each time testing on a different part and training on the rest, then report the average and spread. Every photo gets tested once, at the cost of 5 times the training.

IN YOUR PROJECT

"Small dataset and a small test set" is the first limitation in §11.4 (p. 40); viva Q39 (p. 44) asks exactly this.

Error analysis

Metrics say *how much* a model gets wrong; error analysis says *what kind* of mistakes it makes and *why*. The brief asks for exactly this ("correct detections, missed detections and false detections"), and guide chapter 9 and notebook Step 8 do it. This part explains each error type with IoU numbers, then how to turn the counts into causes, remedies and well-written observations.

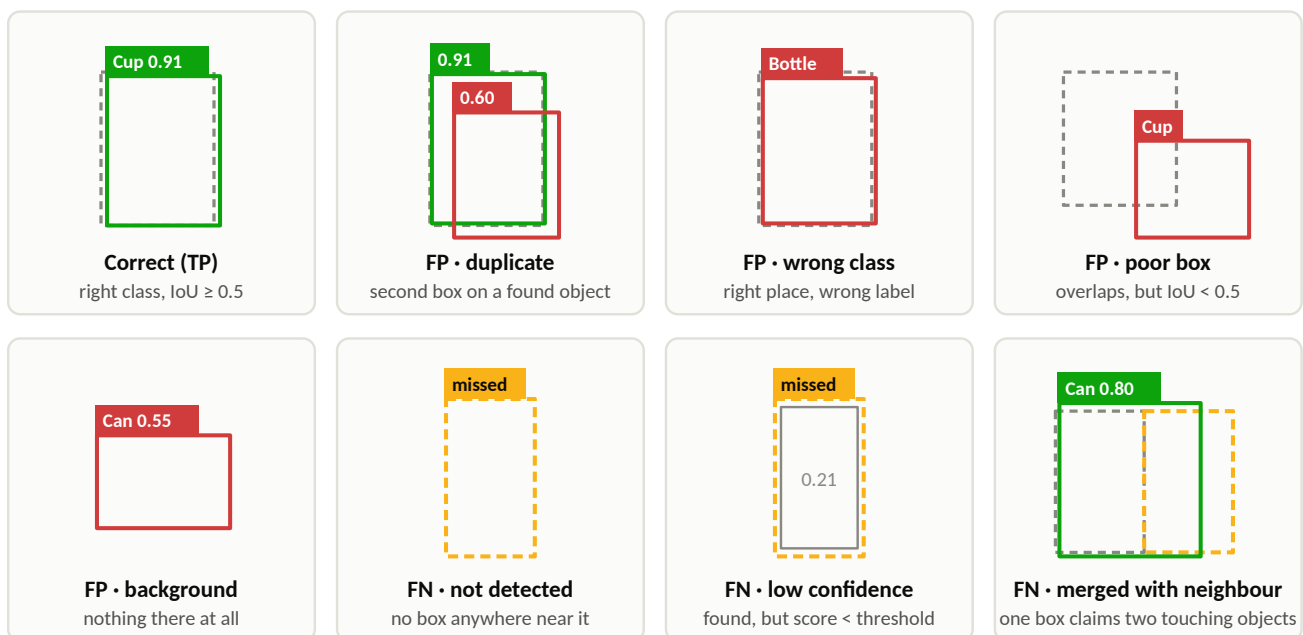
9.1 From a score to an explanation

IN THE GUIDE §9.1 p. 29 §2.1 p. 6 Step 8 p. 37

In one line. Error analysis gives every false detection and every missed object exactly one reason, so that "mAP 0.41" becomes something you can explain and act on.

Compare two sentences an examiner might read. "Model A has mAP 0.41 and model B has 0.37." And: "Most of model B's misses are cigarette butts and bottle caps under 20 px, which its 300×300 input shrinks to a few pixels." The second shows understanding. Error analysis is how you get from the first to the second.

The notebook sorts mistakes like a machine with a row of bins: each mistake is checked against the rules from top to bottom and dropped into the **first** bin whose rule fits. So every mistake gets exactly one reason, and the counts add up. The rules only use IoU (concept 4.5) and two cut-offs: $\text{IoU} \geq 0.5$ means "matches", and $\text{IoU} \geq 0.1$ means "near".



The same figure as guide §9.1 (p. 29). Grey dashed: the true object. Green: correct. Red: false detection. Yellow dashed: missed object.

The false-detection types follow the TIDE toolbox (Bolya et al., 2020), in simplified form. Concepts 9.2 and 9.3 go through each type; concept 9.4 connects them to causes and remedies; concept 9.5 is about writing them up.

IN YOUR PROJECT

`explain_errors` in Step 8 applies the rules at each model's threshold. The next cell counts the reasons into `errors.csv` and draws `error_breakdown.png`. Your report's Error analysis section (Table E1 in §11.2, p. 39) is built from these counts.

9.2 False detections: four reasons

IN THE GUIDE §9.1 p. 29 §9.2 p. 30 Step 8 p. 37

In one line. A kept box that isn't a TP is explained, in this order, as a duplicate, a wrong class, a poor box or pure background.

For each false box, the code computes two numbers: its best IoU with any object of the **same** class (`iou_same`) and with any object of **another** class (`iou_other`). Then:

Reason	Rule (first match wins)	What happened
duplicate	<code>iou_same</code> ≥ 0.5	A good box of the right class, but its object was already claimed by a higher-scoring box (concept 8.1). A second box on the same object that survived NMS (concept 4.11).
wrong class	<code>iou_other</code> ≥ 0.5	Right place, wrong name: a Cup called Bottle.
poor box	the larger of the two ≥ 0.1	It found something, but the box is badly placed or sized: IoU between 0.1 and 0.5.
background	otherwise	Nothing there: IoU under 0.1 with every object. A leaf, a stone or a shadow mistaken for waste.

WORKED EXAMPLE: THE PHOTO FROM CONCEPT 8.1

P2 (Bottle, 0.88) overlaps G1, a Bottle, with IoU 0.66, but P1 already claimed G1. `iou_same` = 0.66 ≥ 0.5 , so: **duplicate**.

P4 (Bottle, 0.52) overlaps G2, a Bottle, with IoU 0.41. Not ≥ 0.5 with anything, but 0.41 ≥ 0.1 , so: **poor box**.

Two more for practice. A Cup box sitting on a true Bottle with IoU 0.70 and near no Cup: **wrong class**. A Can box on a patch of leaves, with IoU below 0.1 against every object: **background**.

Why the order matters. A box can fit more than one rule. A Bottle box with IoU 0.60 against an already-claimed Bottle and 0.55 against a Cup fits both "duplicate" and "wrong class". The fixed order decides: it is counted once, as a duplicate. The rules run from the most specific explanation to the least, so each box gets the most informative label that fits.

IN YOUR PROJECT

The first loop of `explain_errors`. The likely causes in this dataset, from §9.2 (p. 30): duplicates on long or crumpled objects; wrong classes between look-alikes (Cup / Bottle / Plastic, Cigarette Box / Cardboard); poor boxes on small, hidden or crushed items; background on leaves, stones and paper scraps.

9.3 Missed objects: five reasons

IN THE GUIDE §9.1 p. 29 §9.2 p. 30 Step 8 p. 37 Q42 p. 44

In one line. An object nobody claimed is explained, in this order, as merged with a neighbour, low confidence, wrong class, poor box or not detected at all.

For each missed object the code looks at all predictions, including those *below* the threshold, and checks:

Reason	Rule (first match wins)	What happened
merged with neighbour	a <i>kept</i> box of the same class has IoU ≥ 0.5 with it	A good box existed, but it was claimed by a touching object of the same class. Often NMS also deleted this object's own box (concept 4.11).

Reason	Rule (first match wins)	What happened
low confidence	a same-class box with IoU ≥ 0.5 exists, but <i>below</i> the threshold	The model found it but wasn't sure enough.
wrong class	a kept box of another class has IoU ≥ 0.5	Found in the right place, but given the wrong name.
poor box	a kept box has IoU ≥ 0.1	Something was drawn near it, but badly placed.
not detected	otherwise	Nothing anywhere near it.

WORKED EXAMPLES

G2 in concept 8.1: no same-class box reaches IoU 0.5, no other-class box either, but P4 (kept) has IoU 0.41 with it, so: **poor box**.

Two touching cans A and B. One kept Can box overlaps A with IoU 0.62 and B with IoU 0.55. It claims A, its better match, so B is missed, although a box of its class matches it: **merged with neighbour**.

A cigarette butt with a Cigarette Butt box at IoU 0.70 but score 0.30, below a threshold of 0.45: **low confidence**. Lowering the threshold would recover it, at the cost of more false alarms (concept 8.3).

One mistake, two entries. A Cup labelled Bottle is a *false Bottle* (FP, wrong class) and a *missed Cup* (FN, wrong class). That's standard counting, not a bug: the model produced a box that wasn't right, and it failed to produce the box that was. Mention it when you present the table.

A check that the counts are right. Every object is either found or missed, so $TP + (\text{all FN rows}) = \text{the number of test objects}$, for every model. Every kept box is either correct or false, so $TP + (\text{all FP rows}) = \text{the number of kept boxes}$.

IN YOUR PROJECT

The second loop of `explain_errors`, and the check in Step 8. "Merged with neighbour" is the brief's "objects may overlap" in numbers. Your most common error types per model, with their causes, are viva Q42 (p. 44).

9.4 Recall by object size, and from cause to remedy

IN THE GUIDE §9.3 p. 30 §9.2 p. 30 §2.5 p. 8 §11.4 p. 40 Q46 p. 44

In one line. Splitting the test objects by size and measuring recall in each group shows whether small objects are the problem, and whether it's the data's fault or one model's design.

The notebook uses the relative size of concept 6.5, $\sqrt{(\text{box area} \div \text{image area})}$, with three groups: **small** under 0.1, **medium** 0.1 to 0.3, **large** above 0.3. For each model it reports the share of objects found in each group.

WORKED EXAMPLE: SIZING OBJECTS IN A 1024 × 768 PHOTO

A 40 × 40 px cigarette butt: $\sqrt{(1,600 \div 786,432)} = 0.045$, so **small**. A 300 × 200 px piece of cardboard: $\sqrt{(60,000 \div 786,432)} = 0.28$, so **medium**. A 200 × 500 px bottle: $\sqrt{(100,000 \div 786,432)} = 0.36$, so **large**.

Reading the result (§9.3, p. 30):

- If **all three** models find few small objects, it's the data: too few small examples, or too little detail in the photos.
- If **only SSD300** drops sharply on small objects, it's the architecture: a 300 × 300 input and no FPN (concept 5.3). That tests hypothesis H2 directly.

From cause to remedy. Each error type points to a cause, and each cause to a fix you can propose as future work. The guide's table is in §9.2 (p. 30); here is what the less obvious remedies mean:

- **Higher input resolution:** e.g. `MIN_SIZE, MAX_SIZE = 1024, 1707` instead of 800, 1333 makes every object 28% bigger in pixels, at about $1.28^2 \approx 1.6$ times the computation.
- **Tiling:** cut a large photo into overlapping tiles, run the detector on each tile at full size, shift each tile's boxes back by the tile's position, and merge everything with NMS. Small objects get far more pixels, at the cost of several runs per photo.
- **Soft-NMS** for touching objects (concept 4.11): lowers the scores of overlapping boxes instead of deleting them, which reduces "merged with neighbour" misses.
- **Photos of empty ground** for "background" false alarms: they give the model more examples of what is *not* waste.
- **More examples of the confused pairs** for "wrong class" errors, and **clearer class definitions** (when is a crushed bottle "Plastic"?).

IN YOUR PROJECT

Step 8 prints recall by size with the number of objects in each group; the numbers belong in Table E1. The remedies feed the future-work list in §11.4 (p. 40) and your answer to viva Q46 (p. 44) ("what would you do with two more weeks?"). The SSD300 comparison is your verdict on H2 (§2.5, p. 8).

9.5 Your own photos, domain shift, and writing observations

IN THE GUIDE §9.4 p. 30 §9.5 p. 30 Step 9 p. 38 Q45 p. 44

In one line. New photos from your own phone test whether the models work outside the dataset's "world", and every observation you write should run evidence → cause → implication.

Domain shift. The test set was drawn from the same collection as the training photos: same collector, similar camera, streets and light. Your own photos differ in camera, lighting, background, distance and even the brands of the waste. This change of conditions is called **domain shift**. Models usually do worse under it, and how much worse says a lot about how general their learning is. The test set measures "works on photos like these"; your own photos probe "works in the real world".

Judging photos without labels. There are no true boxes, so no mAP. Describe instead, per model: how many items you can see, how many were found, what was missed, what was "hallucinated" (boxes on nothing), and in what conditions things went wrong (glare, clutter, unusual angles). Comparing the three models on the same photo is the most telling part.

Writing an observation (§9.5, p. 30). Every observation has three parts:

1. **Evidence:** a number or a figure from your results, with its context. "On the 25 test photos, SSD300 found ___ of ___ small objects..."
2. **Cause:** the mechanism, from the concepts in this companion. "...consistent with its 300×300 input, where a 40-px butt shrinks to about 12 px, and its lack of an FPN."
3. **Implication:** what it means for the use case. "For a sorting line that must catch small litter, SSD300 is unsuitable despite being ___ times faster."

If a result contradicts one of your hypotheses, say so plainly and suggest why. An honest "H3 did not hold: RetinaNet was less accurate than SSD300 on large objects, possibly because..." reads as understanding, not failure.

IN YOUR PROJECT

Step 9 saves `results/unseen_*.png` with the three models side by side. The guide's template is in §9.5 (p. 30), and the example images to choose are described in §9.4 (p. 30). How the models did on your own photos is viva Q45 (p. 44), where "domain shift" is the key phrase.

The tools: Colab, PyTorch and friends

You know Python, so this part skips the language itself. It explains the tools the notebook relies on, as they appear in guide chapter 10 and the notebook steps: Colab and Drive, the PyTorch libraries, the dataset and loader classes, the detection and transform APIs, the result files, checkpoints, running on a laptop, and how to debug when something goes wrong.

10.1 Colab, notebooks and runtimes

IN THE GUIDE [ch. 10 p. 31](#) [Step 0 p. 31](#) [§1.3 p. 5](#) [ch. 13 p. 45](#)

In one line. Colab runs your notebook on a temporary Google computer with a GPU. The notebook is a list of cells; the **runtime** behind it holds all variables in memory until it is restarted or shut down.

- **Cells** are either code or text. Running a code cell executes it in the runtime, and every variable it creates stays in memory for the next cells. Cells can be run in any order, which is convenient but risky: if you edit an early cell, re-run everything after it, or later cells will use stale values. When in doubt, **Run all** from the top.
- The **runtime** is a virtual machine with a GPU (choose *Runtime* → *Change runtime type* → *T4 GPU*). It is temporary: after a period of inactivity or a usage limit it is taken away, together with its memory and local disk. Only files saved to Google Drive survive (concept 10.2).
- **Restarting the runtime** clears memory but keeps the machine and its disk. Do it after installing some packages, and after a "CUDA out of memory" error, which can leave the GPU memory in a bad state (ch. 13, p. 45).
- Two notebook-only commands appear in the notebook. A line starting with `!` runs a shell command, as in `!pip install -q torchmetrics pycocotools`. A cell starting with `%%writefile path` saves the rest of the cell as a file instead of running it: that's how `predict.py` is written to Drive.
- **GPU quota.** Free GPU time is limited per Google account. That's one reason the guide suggests each model owner trains their own model in their own Colab (§1.3, p. 5).

IN YOUR PROJECT

The key box at the start of ch. 10 (p. 31) is the checklist: open the notebook in Colab, pick the T4, run from the top, edit only `ORIG_DIR`. Step 0 prints `device: cuda` and the GPU name when the GPU is really there (Step 0, p. 31).

10.2 Files, paths and Google Drive

IN THE GUIDE [Step 0 p. 31](#) [Step 1 p. 31](#) [§1.3 p. 5](#) [§2.6 p. 9](#)

In one line. Anything that must survive a disconnect lives in your Google Drive folder; bulky working files live on Colab's fast but temporary local disk.

Mounting Drive (`drive.mount("/content/drive")`) makes your Drive appear as an ordinary folder, `/content/drive/MyDrive`, after you approve access. The notebook uses two places:

Folder	Where	What's there, and why
<code>PROJECT_DIR</code>	Drive: <code>MyDrive/trash_detection</code>	Permanent: the <code>zip</code> , <code>split.json</code> , <code>checkpoints/</code> , <code>results/</code> , <code>unseen/</code> , <code>model_info.json</code> , <code>predict.py</code> . Survives disconnects and can be shared.

Folder	Where	What's there, and why
<code>WORK_DIR</code>	Colab disk: <code>/content/work</code>	Temporary: the unzipped dataset and the clean copy. Reading thousands of small files is much faster here than from Drive. Rebuilt each session in a minute or two.

Paths in code. The notebook uses `pathlib.Path` objects. The `/` operator joins path parts, so `PROJECT_DIR / "checkpoints"` is the checkpoints folder on any operating system. `mkdir(parents=True, exist_ok=True)` creates a folder and any missing parents, and does nothing if it already exists.

The download (Step 1): `curl` saves the zip into Drive once, `zipfile.is_zipfile` checks that a real zip arrived (Kaggle might return an error page instead), and the zip is unpacked to the local disk. Later sessions skip the download.

IN YOUR PROJECT

Pitfall 9 in §2.6 (p. 9) ("Colab disconnects and work is lost") is solved by this split. Sharing between teammates also goes through these Drive files (§1.3, p. 5). If the download fails, the browser route is in Step 1 (p. 31).

10.3 PyTorch, torchvision and the other libraries

IN THE GUIDE [Step 0 p. 31](#) [Step 5 p. 33](#) [§11.1 p. 39](#)

In one line. PyTorch does the maths, torchvision supplies the detectors and image tools, torchmetrics the COCO metrics, and Pillow, NumPy and Matplotlib handle images, numbers and charts.

Library	What it provides	Where you see it
torch	Tensors (2.4), autograd (1.5), layers (<code>torch.nn</code>), optimizers (1.7), mixed precision (2.8), GPU access	everywhere; <code>train_model</code>
torchvision	Detection models (Part 5), <code>transforms.v2</code> and <code>tv_tensors</code> (10.6), box operations such as <code>box_iou</code> and NMS	Steps 5–8
torchmetrics (+ pycocotools)	The COCO mAP computation (8.7)	<code>coco_map</code>
Pillow (PIL)	Opening JPEGs, EXIF rotation (6.3), resizing	Step 2, <code>predict.py</code>
NumPy	Fast arrays for the calculations outside the model: medians, cumulative sums, argmax	Steps 2, 7, 8
Matplotlib	All charts and example images	every <code>.png</code> in <code>results/</code>

nn.Module, the base class of every PyTorch model and layer, gives the handful of methods the notebook uses: calling `model(x)` runs its `forward` computation; `model.parameters()` lists its weights; `model.to(DEVICE)` moves them to the GPU; `model.train()` and `model.eval()` switch modes (concept 7.2); `model.state_dict()` returns every weight by name (concept 10.8). Layers are just attributes, which is why `model.roi_heads.box_predictor = ...` can swap one out.

IN YOUR PROJECT

Step 0 prints the versions: `torch ... | torchvision ... | device: cuda`. Copy them into your report's appendix (§11.1, p. 39): results can change slightly between library versions.

10.4 Dataset, DataLoader and collate

IN THE GUIDE [Step 5 p. 33](#) [§7.2 p. 24](#) [§3.4 p. 10](#)

In one line. A **Dataset** knows how to produce one training example; a **DataLoader** groups examples into shuffled batches, loading them in the background; **collate** decides how a batch is packed.

A **Dataset** is any class with two methods: `__len__` (how many examples) and `__getitem__(i)` (give me example number *i*). `TrashDataset.__getitem__` opens the *i*-th clean photo, turns it into a $3 \times H \times W$ tensor, wraps its boxes as `BoundingBoxes`, converts class names to ids 1–8, and applies the transforms (augmentation for training, only the 0–1 conversion otherwise). It returns `(image, {"boxes": ..., "labels": ...})`.

A **DataLoader** calls `__getitem__` for you, again and again:

- `batch_size=4`: four examples per batch;
- `shuffle=True` for training (a new order every epoch, concept 1.6), `False` for evaluation;
- `num_workers=2`: two helper processes load and augment the next batches while the GPU works on the current one, so the GPU isn't kept waiting.

Why a custom collate? By default a `DataLoader` stacks the four images into one 4-D tensor (concept 2.4), which only works if they are all the same size. Our photos differ in size and in number of boxes, so `collate` simply regroups the batch: `tuple(zip(*batch))`.

WORKED EXAMPLE: WHAT `ZIP(*BATCH)` DOES

The loader hands `collate` a list of four pairs: `[(img1, t1), (img2, t2), (img3, t3), (img4, t4)]`. `zip(*batch)` pairs up the first items and the second items, giving `((img1, img2, img3, img4), (t1, t2, t3, t4))`: a tuple of images and a tuple of targets. That's exactly the two lists the detectors expect. Each model then resizes and pads the images into one tensor internally (concept 5.1).

IN YOUR PROJECT

`TrashDataset`, `collate` and `make_loader` in Step 5; used by `train_model` (Step 6) and `predict`. Step 1 of the iteration in §7.2 (p. 24) is one pass through this machinery.

10.5 Using torchvision's detection models

IN THE GUIDE [Step 5 p. 33](#) [§4.1 p. 12](#) [§7.2 p. 24](#) [Q30 p. 43](#)

In one line. One function call builds each detector, optionally with COCO weights. You give it a list of 0–1 image tensors (plus targets when training), and it handles resizing, normalising, anchors, losses and NMS itself.

Building. For example,

```
D.fasterrcnn_resnet50_fpn_v2(weights=D.FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT,
weights_backbone=None, min_size=800, max_size=1333).
```

- `weights=...DEFAULT` means "the best available COCO weights for this model". The first time, they are downloaded from PyTorch's servers and cached. `weights=None` builds the same architecture with random weights: that's what `load_trained` and `predict.py` do before loading your own checkpoint (concept 10.8).
- `weights_backbone=None`: the COCO detector weights already include a trained backbone, so there's no need for separate ImageNet backbone weights.
- `min_size`, `max_size`: the internal resize (concept 5.1). SSD300 has no such options: it always uses 300×300 .

Inputs. A list of $3 \times H \times W$ float tensors with values 0–1, on the model's device. Don't normalise or resize them yourself: the model's built-in transform does that, and doing it twice would feed it wrong values. In training mode, also a list of target dicts with `boxes` (float, [N, 4], XYXY pixels) and `labels` (int64, [N], 1–8).

Outputs. Training mode: a dict of losses. Evaluation mode: a list of dicts with `boxes`, `labels` and `scores`, one per image, with boxes in the input image's own pixel coordinates (concept 7.2).

Changing a model after building it. Settings and layers are ordinary attributes. The builders set the post-processing (`roi_heads.score_thresh` and friends for Faster R-CNN; `model.score_thresh`, `nms_thresh` and `detections_per_img` for the others) and replace the last class layer (concept 3.8).

IN YOUR PROJECT

The three `build_*` functions in Step 5 (Step 5, p. 33). The output format is shown in §4.1 (p. 12); the two modes are viva Q30 (p. 43).

10.6 transforms.v2 and tv_tensors: how boxes move with pixels

IN THE GUIDE §6.6 p. 22 Step 5 p. 33 Q50 p. 44

In one line. `tv_tensors` are tensors with a label saying what they are (an image, a set of boxes). `transforms.v2` reads those labels and applies each change correctly to every part of a sample at once.

Augmentation for detection needs boxes to move with the pixels (concept 6.8). `transforms.v2` makes this automatic. A transform receives the whole sample, `(image, target)`, and treats each piece according to its type:

- a `tv_tensors.Image` has its pixels flipped, cropped or recoloured;
- a `tv_tensors.BoundingBoxes` has its coordinates changed to match. It carries its `format` ("XYXY") and its `canvas_size` (the image height and width), which the transform needs: mirroring x means computing $W - x$, so it must know W ;
- anything else, such as the `labels` tensor, passes through unchanged.

`T.Compose([...])` runs transforms one after another. Most random transforms take a probability `p`; for those that don't, `T.RandomApply([t], p=0.3)` adds one. `SanitizeBoundingBoxes` finds the labels through the "labels" key of the target dict, so it can delete a label together with its box.

Why the wrapping matters. If the boxes were a plain tensor instead of `BoundingBoxes`, `transforms.v2` couldn't tell they are boxes. A flip on its own would then move the pixels and silently leave the numbers alone, so half the training photos would have their boxes on the wrong side. (In our pipeline you'd at least get an error, because `RandomIoUCrop` and `SanitizeBoundingBoxes` refuse to run without boxes.) That's why `TrashDataset` wraps them, and why the Step 5 augmentation preview exists: look at it.

IN YOUR PROJECT

`tv_tensors.BoundingBoxes(boxes, format="XYXY", canvas_size=(H, W))` in `TrashDataset`, and `train_tf / eval_tf` in Step 5 (§6.6, p. 22). Viva Q50 (p. 44) is about `SanitizeBoundingBoxes`.

10.7 The result files, and where each one goes

IN THE GUIDE §11.1 p. 39 §11.2 p. 39 §11.5 p. 40 §1.2 p. 4 Q47 p. 44

In one line. The notebook writes tables as CSV, structured data as JSON and charts as PNG. Together they hold every number and figure your report needs.

CSV (comma-separated values) is a plain-text table that opens in Excel or Google Sheets. **JSON** stores Python-style dictionaries and lists as text, readable by people and programs. **PNG** images are the charts, saved at print quality.

File (in <code>results/</code> unless noted)	Made in	Use it for
<code>eda.png</code> , <code>label_check.png</code>	Step 3	Dataset section: class counts, object sizes, "labels verified visually"
<code>split.json</code> (project folder)	Step 4	Dataset section: the split and its seed
<code>history_<model>.json</code> , <code>training_curves.png</code>	Step 6	Results: training behaviour and best epochs
<code>summary.csv</code> , <code>per_class.csv</code> , <code>metrics.json</code>	Step 7	Tables R1 and R2; <code>metrics.json</code> keeps every number
<code>comparison.png</code> , <code>per_class_ap.png</code> , <code>pr_curves.png</code>	Step 7	Results figures
<code>errors.csv</code> , <code>error_breakdown.png</code> , <code>confusion_matrices.png</code> , <code>examples_<id>.png</code>	Step 8	Table E1 and the Error analysis section
<code>unseen_*.png</code>	Step 9	Your own photos: Error analysis or Discussion
<code>model_info.json</code> , <code>predict.py</code> (project folder)	Step 9	Running the models on every laptop (concept 10.9)

Turn CSVs into proper report tables with 3 decimals for mAP, P, R and F1 and whole numbers for milliseconds, and never paste raw notebook printouts (§11.5, p. 40).

IN YOUR PROJECT

The structure in §11.1 (p. 39) names the source of every report section, and the empty tables in §11.2 (p. 39) are filled from `summary.csv`, `per_class.csv` and `errors.csv`. The "definition of done" in §1.2 (p. 4) lists these files. Read top to bottom, the table is also a walk through your pipeline, which is viva Q47 (p. 44).

10.8 Saving and loading weights

IN THE GUIDE Step 6 p. 35 Step 7 p. 36 Step 9 p. 38 §5.4 p. 19

In one line. A checkpoint is the model's `state_dict`, a dictionary from each weight's name to its tensor. To use it, rebuild exactly the same architecture and load the dictionary into it.

What's inside. Every weight has a name built from the attribute path to its layer, for example

`backbone.body.conv1.weight` (shape [64, 3, 7, 7]) or `roi_heads.box_predictor.cls_score.weight` (shape [9, 1024]). `torch.save(model.state_dict(), path)` writes this dictionary to a `.pt` file. It stores numbers only, not code, which is why the architecture has to be rebuilt first.

Loading, as `load_trained` and `predict.py` do it:

1. Build the same architecture with `weights=None` and the same 9-class head.

2. `torch.load(path, map_location="cpu", weights_only=True)`. `map_location="cpu"` lets a checkpoint saved from GPU tensors open on a computer without a GPU. `weights_only=True` accepts only tensors and plain data, never arbitrary Python objects, which makes loading a file safe.
3. `model.load_state_dict(state)`. By default this is **strict**: every name must exist in the model and every shape must match.

WORKED EXAMPLE: THE ERROR YOU'D GET FROM A MISMATCHED HEAD

Rebuild Faster R-CNN but forget to replace its head, so it still has 91 COCO classes. Loading your checkpoint fails with a message like: `size mismatch for roi_heads.box_predictor.cls_score.weight: copying a param with shape torch.Size([9, 1024]) from checkpoint, the shape in current model is torch.Size([91, 1024])`. Read it as: "the file has 9 classes, the model you built has 91". The fix is to build with the same function and `NUM_CLASSES` used in training.

File size follows from the parameter count: 4 bytes per 32-bit weight (concept 8.8), so about 174, 146 and 99 MB for our three models (§5.4, p. 19).

IN YOUR PROJECT

Saving in `train_model` (Step 6, p. 35); loading in `load_trained` (Step 7, p. 36) and in `predict.py`'s `build` and `main` (Step 9, p. 38).

10.9 Running the models on a laptop

IN THE GUIDE [Step 9 p. 38](#) [ch. 13 p. 45](#) [Q48 p. 44](#)

In one line. `predict.py` is a small stand-alone program: given a model name and a folder of photos, it rebuilds the model, loads your checkpoint, draws the detections and saves annotated copies, on an ordinary CPU.

The command line. You run it from a terminal (Command Prompt or PowerShell on Windows, Terminal on macOS) opened in the project folder. `pip install torch torchvision pillow` installs the three libraries it needs, once. Optionally, a **virtual environment** (`python -m venv .venv`) keeps the project's packages separate from everything else on the laptop.

Arguments. `argparse` reads the words after the script name: `--model` (`faster_rcnn`, `retinanet` or `ssd300`), `--images` (a photo or a folder), `--out` (where to save), and `--threshold` to override the default.

```
python predict.py --model ssd300 --images unseen --out predictions
python predict.py --model faster_rcnn --images unseen --threshold 0.3
```

What it does, in order: read `model_info.json` (class names, input size, each model's threshold chosen on validation); rebuild the architecture with `weights=None` (concept 10.8); load `checkpoints/<model>.pt`; then for each photo: make it upright with its EXIF flag (concept 6.3), shrink it to at most 1,024 px like the clean copy, convert it to a 0–1 tensor, run the model under `torch.no_grad()` (concept 7.2), keep boxes at or above the threshold, draw and save them, and print what was found.

Speed. A CPU has a few cores instead of thousands (concept 2.7), so expect seconds per photo for Faster R-CNN and RetinaNet and less for SSD300. That's why SSD300 is the safest choice for a live demo.

IN YOUR PROJECT

The setup steps are in Step 9 (p. 38), and the two `predict.py` rows of ch. 13 (p. 45) cover "No module named torch" and "file not found" (run it from inside the project folder). "Show it running" is viva Q48 (p. 44).

10.10 Debugging: reading errors and the sanity check

IN THE GUIDE [ch. 13 p. 45](#) [§13.1 p. 45](#) [§2.5 p. 8](#) [§7.3 p. 25](#)

In one line. Read the last line of an error first, change one thing at a time, and when training "works" but learns nothing, check whether the model can memorise four photos.

Reading a traceback. When Python stops with an error, it prints a **traceback**: the chain of function calls that led to it. The **last line** is the actual error ("RuntimeError: CUDA out of memory", "KeyError: 'img_012'"). The lines just above it show where it happened. Start at the bottom, find the first line that points into *your* notebook code, and look it up in the table in ch. 13 (p. 45).

Two kinds of bug. Loud bugs crash, with a message you can search for. Silent bugs are worse: everything runs, but the numbers are wrong. Misplaced boxes (concepts [6.3](#) and [6.8](#)), class ids off by one (4.4) and leakage (1.11) are all silent. The guide lists the signs in §2.5 (p. 8): validation mAP near 0 after several epochs, boxes shifted in the example images, a class never predicted, a `nan` loss. The notebook's visual checks exist to catch them: `label_check.png`, the augmentation preview and the example panels. Always look at them.

The sanity check (§13.1, p. 45): train Faster R-CNN for 10 short epochs on just 4 training photos (repeated 5 times per epoch, so 50 steps), then measure mAP@0.5 on those same 4 photos. A network with 43 million weights can easily memorise 4 photos, so a working pipeline climbs clearly, usually past 0.5. If it stays at 0.00, the model isn't being given correct boxes and labels, whatever the loss is doing, and the fault is in the data steps (2 and 3), not the model.

Change one thing at a time. If you change the learning rate, the batch size and the augmentation together and the problem disappears, you won't know which one fixed it, and for a comparison study you must then apply the same change to all three models (concept [7.5](#)).

IN YOUR PROJECT

The troubleshooting table in ch. 13 (p. 45) maps symptoms to causes and fixes. The sanity check saves under its own name, so it doesn't touch your real checkpoints. The "Stuck" pattern in §7.3 (p. 25) is when to use it.

A Viva questions and where they're explained

All 50 questions from the guide's ch. 12 (p. 41), each with the concepts that explain the answer. To revise, cover the right-hand column, answer the question aloud, then check yourself against the concepts and the guide's model answer.

12.1 The problem and the data guide p. 41

- | | | |
|---|--|-----------------------|
| 1 | Is this classification or detection? Why? | 4.1 p. 25 |
| 2 | Describe the dataset. | 6.1 p. 45 |
| 3 | Why didn't you use the 4,923 augmented images? | 1.11 p. 11, 6.1 p. 45 |
| 4 | How did you split the data, and why like that? | 6.7 p. 49 |
| 5 | What is EXIF orientation, and why did you handle it? | 6.3 p. 46 |
| 6 | Why Pascal VOC labels? | 4.4 p. 27 |
| 7 | Convert the VOC box (160, 120, 320, 360) in a 640 × 480 image to YOLO. | 4.4 p. 27 |
| 8 | Which classes are hardest, and why? | 6.6 p. 48 |

12.2 The models guide p. 41

- | | | |
|----|--|---------------------------------|
| 9 | Why these three models? | 4.13 p. 36, 5.4 p. 43 |
| 10 | One-stage vs two-stage? | 4.13 p. 36 |
| 11 | What is an anchor? | 4.6 p. 29 |
| 12 | What does the RPN do? | 5.1 p. 38 |
| 13 | What is RoIAlign? | 5.1 p. 38 |
| 14 | Explain focal loss. | 5.2 p. 40 |
| 15 | Why does RetinaNet's last layer start at a 1% prior? | 2.6 p. 16, 5.2 p. 40 |
| 16 | How does SSD handle the background imbalance? | 5.3 p. 42 |
| 17 | Why does SSD300 struggle with small objects? | 5.3 p. 42 |
| 18 | What is an FPN? | 4.12 p. 35 |
| 19 | What is NMS, and which threshold did you use? | 4.11 p. 34 |
| 20 | What did you change in each model for 8 classes? | 3.8 p. 23, 5.1 p. 38 |
| 21 | Softmax or sigmoid? | 2.2 p. 13, 4.3 p. 26 |
| 22 | What does "v2" mean? | 2.5 p. 15, 4.9 p. 32, 5.4 p. 43 |

12.3 Training guide p. 42

- | | | |
|----|---|-----------------------|
| 23 | What is transfer learning, and why use it here? | 3.8 p. 23 |
| 24 | Which optimizer and learning rate, and why the same for all three? | 1.7 p. 8 |
| 25 | Why warm-up? | 1.8 p. 9 |
| 26 | How did you choose the number of epochs? | 7.3 p. 52 |
| 27 | How do you know you didn't overfit? | 1.9 p. 9, 7.4 p. 53 |
| 28 | What is mixed precision? | 2.8 p. 17 |
| 29 | Why not compare loss values between models? | 4.2 p. 25, 7.4 p. 53 |
| 30 | What does a torchvision detector return in training vs evaluation mode? | 7.2 p. 52, 10.5 p. 69 |

12.4 Metrics guide p. 43

- | | | |
|----|------------------------------------|-----------|
| 31 | Define IoU and compute an example. | 4.5 p. 28 |
|----|------------------------------------|-----------|

32	When is a detection a true positive?	8.1 p. 55
33	Why is there no accuracy?	8.1 p. 55
34	Precision vs recall for a waste sorter?	8.2 p. 56
35	What is AP, and what is mAP?	8.4 p. 57
36	mAP@0.5 vs mAP@0.5:0.95?	8.5 p. 59
37	How did you choose the confidence threshold?	8.3 p. 56
38	How did you measure speed?	8.8 p. 61
39	How reliable are results on 25 test photos?	8.9 p. 62
40	Why use torchmetrics <i>and</i> your own matching code?	8.7 p. 60
12.5 Results and analysis guide p. 44		
41	Which model is best?	5.4 p. 43
42	What were the most common errors?	9.3 p. 64
43	Which classes were confused with each other?	8.6 p. 60
44	Did your hypotheses hold?	5.4 p. 43
45	How did the models do on your own photos?	9.5 p. 66
46	What would you do with two more weeks?	9.4 p. 65
12.6 Practical guide p. 44		
47	Walk me through your pipeline.	10.7 p. 71
48	Show it running.	10.9 p. 72
49	Why is <code>num_classes</code> 9 and not 8?	4.1 p. 25
50	What does <code>SanitizeBoundingBoxes</code> do?	6.4 p. 47, 6.8 p. 49, 10.6 p. 70

B Index

Every term explained in this companion, with its concept number and page. Terms used only in passing are listed under the concept that explains them.

Numbers

101-point interpolation [8.4](#) p. 57
1x1 convolution [3.5](#) p. 21
3x3 convolution [3.5](#) p. 21
91 classes [3.8](#) p. 23

A

accuracy [8.1](#) p. 55
accuracy-speed trade-off [5.4](#) p. 43
activation function [2.2](#) p. 13
Adam [1.7](#) p. 8
AdamW [1.7](#) p. 8
agreement check [8.7](#) p. 60
all-point interpolation [8.4](#) p. 57
alpha [5.2](#) p. 40
AMP [2.8](#) p. 17
anchor [4.6](#) p. 29
anchor assignment [4.7](#) p. 30
anchor box [4.6](#) p. 29
anchor generator [4.6](#) p. 29
anchor size [4.6](#) p. 29
anchor-free [4.13](#) p. 36
anchors per position [4.6](#) p. 29
annotation [6.1](#) p. 45
annotator [6.1](#) p. 45
AP [8.4](#) p. 57
Apache 2.0 [6.1](#) p. 45
AR@100 [8.5](#) p. 59
architecture [1.2](#) p. 5
area under the curve [8.4](#) p. 57
argparse [10.9](#) p. 72
artificial neuron [2.1](#) p. 13
aspect ratio [3.1](#) p. 19, [4.6](#) p. 29
aspect ratio distortion [5.3](#) p. 42
asynchronous [2.7](#) p. 16
asynchronous GPU [8.8](#) p. 61
augmentation [6.8](#) p. 49
autocast [2.8](#) p. 17
autograd [1.5](#) p. 7
average precision [8.4](#) p. 57
average recall [8.5](#) p. 59
axis-aligned box [4.4](#) p. 27

B

backbone [3.6](#) p. 22, [3.7](#) p. 22
background anchors [4.10](#) p. 33
background class [4.1](#) p. 25
background column [8.6](#) p. 60
background false positive [9.2](#) p. 64
background row [8.6](#) p. 60
backprop [1.5](#) p. 7
backpropagation [1.5](#) p. 7
backward pass [1.5](#) p. 7

batch [1.6](#) p. 8
batch normalization [2.5](#) p. 15
batch size [1.6](#) p. 8
batching [10.4](#) p. 69
BatchNorm [2.5](#) p. 15
bbox_regression [4.9](#) p. 32, [5.2](#) p. 40
BCE [4.3](#) p. 26
best validation mAP [7.3](#) p. 52
best-match rule [4.7](#) p. 30
best_threshold [8.3](#) p. 56
bias [2.1](#) p. 13
bias initialisation [2.6](#) p. 16, [5.2](#) p. 40
bilinear interpolation [3.9](#) p. 23
binary cross-entropy [4.3](#) p. 26
bootstrap [8.9](#) p. 62
bounding box [4.1](#) p. 25, [4.4](#) p. 27
BoundingBoxes [10.6](#) p. 70
box decoding [4.8](#) p. 31
box encoding [4.8](#) p. 31
box format [4.4](#) p. 27
box head [5.1](#) p. 38
box loss [4.9](#) p. 32
box refinement [4.8](#) p. 31
box regression [4.8](#) p. 31
box subnet [5.2](#) p. 40
box_iou [4.5](#) p. 28
BoxCoder [4.8](#) p. 31
boxes per image [6.5](#) p. 48

C

calibration [8.3](#) p. 56
canvas_size [10.6](#) p. 70
cause and remedy [9.4](#) p. 65
cell [10.1](#) p. 67
CenterNet [4.13](#) p. 36
chain rule [1.5](#) p. 7
change one thing at a time [10.10](#) p. 73
channel [3.1](#) p. 19
channels [3.4](#) p. 21
checkpoint [1.2](#) p. 5, [7.3](#) p. 52
checkpoint file [10.8](#) p. 71
checkpoint size [8.8](#) p. 61
class confusion [8.6](#) p. 60
class counts [6.5](#) p. 48
class id [4.1](#) p. 25
class imbalance [4.10](#) p. 33, [6.6](#) p. 48
class label [4.1](#) p. 25
class subnet [5.2](#) p. 40
class weights [6.6](#) p. 48
class-name normalisation [6.4](#) p. 47
class-weighted loss [6.6](#) p. 48
class_metrics [8.7](#) p. 60
classification [4.1](#) p. 25

classification loss [4.2](#) p. 25, [5.2](#) p. 40
clean copy [6.4](#) p. 47
clean_name [6.4](#) p. 47
clip_grad_norm [2.9](#) p. 17
clipping [6.4](#) p. 47
cls_logits [5.2](#) p. 40
CNN [3.2](#) p. 19
COCO [3.8](#) p. 23
COCO evaluation [8.7](#) p. 60
COCO format [4.4](#) p. 27
COCO mAP [5.4](#) p. 43
COCO metrics [8.5](#) p. 59
COCO size categories [6.5](#) p. 48
collate [10.4](#) p. 69
collate_fn [10.4](#) p. 69
command line [10.9](#) p. 72
command-line arguments [10.9](#) p. 72
Compose [10.6](#) p. 70
compute [8.7](#) p. 60
confidence interval [8.9](#) p. 62
confidence score [4.1](#) p. 25
confidence threshold [8.3](#) p. 56
confounding [7.5](#) p. 54
confusion matrix [8.6](#) p. 60
controlled experiment [7.5](#) p. 54
controlled variables [7.5](#) p. 54
conv head [3.5](#) p. 21
convolution [3.2](#) p. 19
convolutional layer [3.2](#) p. 19
coordinates [3.1](#) p. 19
cosine decay [1.8](#) p. 9
CPU [2.7](#) p. 16
CPU inference [10.9](#) p. 72
cross entropy [4.2](#) p. 25
cross-entropy [4.2](#) p. 25
CSV [10.7](#) p. 71
CUDA [2.7](#) p. 16
cuda.synchronize [8.8](#) p. 61

D

data augmentation [6.8](#) p. 49
data leakage [1.11](#) p. 11
data split [6.7](#) p. 49
DataLoader [10.4](#) p. 69
DataLoader loop [7.1](#) p. 51
dataset [1.1](#) p. 5, [6.1](#) p. 45, [10.4](#) p. 69
debugging [10.10](#) p. 73
deep learning [2.3](#) p. 14
default box [4.6](#) p. 29, [5.3](#) p. 42
DEFAULT weights [10.5](#) p. 69
degenerate box [6.4](#) p. 47
dense layer [2.3](#) p. 14
depth [2.3](#) p. 14

derived data [6.1](#) p. 45
detection confusion matrix [8.6](#) p. 60
detection output [4.1](#) p. 25
detections per image [4.11](#) p. 34
detections_per_img [10.5](#) p. 69
detector families [4.13](#) p. 36
detector parts [3.7](#) p. 22
DETR [4.13](#) p. 36
device [2.4](#) p. 15
diverge [1.4](#) p. 6
domain shift [9.5](#) p. 66
double counting [9.3](#) p. 64
downsampling [3.3](#) p. 20
dtype [2.4](#) p. 15
duplicate [9.2](#) p. 64
duplicate boxes [4.11](#) p. 34

E

early stopping [1.9](#) p. 9
easy negative [4.10](#) p. 33
EDA [6.5](#) p. 48
ElementTree [6.2](#) p. 45
enclosing box [4.9](#) p. 32
epoch [1.6](#) p. 8
epoch loop [7.1](#) p. 51
error [1.3](#) p. 6
error analysis [9.1](#) p. 63
error message [10.10](#) p. 73
error taxonomy [9.1](#) p. 63
errors.csv [10.7](#) p. 71
evaluation mode [7.2](#) p. 52
evidence cause implication [9.5](#) p. 66
example [1.1](#) p. 5
EXIF [6.3](#) p. 46
EXIF orientation [6.3](#) p. 46
exif_transpose [6.3](#) p. 46
exploding gradient [2.9](#) p. 17
exploratory data analysis [6.5](#) p. 48
extra layers [5.3](#) p. 42

F

F1 [8.2](#) p. 56
F1 score [8.2](#) p. 56
fair comparison [7.5](#) p. 54
false negative [8.1](#) p. 55
false negative types [9.3](#) p. 64
false positive [8.1](#) p. 55
false positive types [9.2](#) p. 64
Faster R-CNN [5.1](#) p. 38
Faster R-CNN v2 [5.1](#) p. 38
FastRCNNPredictor [5.1](#) p. 38
FC [2.3](#) p. 14
FCOS [4.13](#) p. 36
feature detector [3.2](#) p. 19
feature map [3.4](#) p. 21
feature pyramid network [4.12](#) p. 35
feature vector [3.4](#) p. 21
figures [10.7](#) p. 71
file pairing [6.2](#) p. 45

filter [3.2](#) p. 19
fine-tuning [3.8](#) p. 23
fixed input size [5.3](#) p. 42
float16 [2.4](#) p. 15, [2.8](#) p. 17
float32 [2.4](#) p. 15
FLOPs [8.8](#) p. 61
FN [8.1](#) p. 55
focal loss [5.2](#) p. 40
foreground [4.7](#) p. 30
foreground-background imbalance [4.10](#) p. 33
forward [10.3](#) p. 68
forward pass [1.5](#) p. 7
FP [8.1](#) p. 55
FP and FN together [9.3](#) p. 64
FPN [4.12](#) p. 35
FPS [8.8](#) p. 61
frames per second [8.8](#) p. 61
freezing [3.8](#) p. 23
frozen layers [3.8](#) p. 23
fully connected layer [2.3](#) p. 14

G

gamma [5.2](#) p. 40
generalised IoU [4.9](#) p. 32
generalisation [1.9](#) p. 9
generalisation to new photos [9.5](#) p. 66
GeneralizedRCNNTransform [5.1](#) p. 38
geometric transform [6.8](#) p. 49
__getitem__ [10.4](#) p. 69
GFLOPs [8.8](#) p. 61
GloU [4.9](#) p. 32
Google Colab [10.1](#) p. 67
Google Drive [10.2](#) p. 67
GPU [2.7](#) p. 16
GPU memory [2.7](#) p. 16
GPU quota [10.1](#) p. 67
gradient [1.4](#) p. 6
gradient clipping [2.9](#) p. 17
gradient descent [1.4](#) p. 6
gradient scaling [2.8](#) p. 17
GradScaler [2.8](#) p. 17
greedy matching [8.1](#) p. 55
ground truth [1.1](#) p. 5
group normalization [2.5](#) p. 15
group split [6.7](#) p. 49
GroupNorm [2.5](#) p. 15, [5.2](#) p. 40

H

H1 [5.4](#) p. 43
H2 [5.4](#) p. 43
H3 [5.4](#) p. 43
half precision [2.8](#) p. 17
hard negative [4.10](#) p. 33
hard negative mining [5.3](#) p. 42
harmonic mean [8.2](#) p. 56
head [3.7](#) p. 22
healthy training [7.4](#) p. 53
held-out [1.10](#) p. 10
hierarchy of features [3.4](#) p. 21

higher resolution [9.4](#) p. 65
history file [7.3](#) p. 52
history json [10.7](#) p. 71
horizontal flip [6.8](#) p. 49
Huber loss [4.9](#) p. 32
hyperparameter [1.12](#) p. 11
hypotheses [5.4](#) p. 43
hypothesis testing [7.5](#) p. 54

I

ignored anchor [4.7](#) p. 30
image [3.1](#) p. 19
image normalisation [5.1](#) p. 38
Image tv_tensor [10.6](#) p. 70
ImageNet [3.6](#) p. 22
imbalance problem [4.10](#) p. 33
independent classes [4.3](#) p. 26
independent variable [7.5](#) p. 54
inference [7.2](#) p. 52
infinity [2.9](#) p. 17
initialisation [2.6](#) p. 16
input [1.1](#) p. 5
int64 [2.4](#) p. 15
interpolated precision [8.4](#) p. 57
interpolation [3.9](#) p. 23
intersection over union [4.5](#) p. 28
invalid box [6.4](#) p. 47
IoU [4.5](#) p. 28
IoU loss [4.9](#) p. 32
IoU threshold [4.7](#) p. 30
iou_other [9.2](#) p. 64
iou_same [9.2](#) p. 64
iteration [1.6](#) p. 8
iteration in code [7.1](#) p. 51

J

Jaccard index [4.5](#) p. 28
joint transforms [10.6](#) p. 70
JSON [6.2](#) p. 45, [10.7](#) p. 71

K

k-fold cross-validation [8.9](#) p. 62
Kaggle [6.1](#) p. 45
kernel [3.2](#) p. 19, [10.1](#) p. 67

L

L1 loss [4.9](#) p. 32
L2 loss [4.9](#) p. 32
label [1.1](#) p. 5
label check [6.5](#) p. 48
label cleaning [6.4](#) p. 47
label file [6.2](#) p. 45
label noise [6.1](#) p. 45
label quality [6.1](#) p. 45
labels key [10.6](#) p. 70
LambdaLR [1.8](#) p. 9
laptop [10.9](#) p. 72
latency [8.8](#) p. 61
lateral connection [3.5](#) p. 21, [4.12](#) p. 35
layer [2.3](#) p. 14

- leakage [1.11](#) p. 11
- learning curve [7.4](#) p. 53
- learning rate [1.4](#) p. 6
- learning-rate schedule [1.8](#) p. 9
- __len__ [10.4](#) p. 69
- LevelMapper [4.12](#) p. 35
- library versions [10.3](#) p. 68
- licence [6.1](#) p. 45
- linear layer [2.3](#) p. 14
- load_records [6.2](#) p. 45
- load_state_dict [10.8](#) p. 71
- load_trained [10.8](#) p. 71
- local disk [10.2](#) p. 67
- localisation error [9.2](#) p. 64
- log loss [4.2](#) p. 25
- logit [2.2](#) p. 13
- look-alike classes [8.6](#) p. 60
- loss [1.3](#) p. 6
- loss dict vs detections [7.2](#) p. 52
- loss dictionary [7.1](#) p. 51
- loss function [1.3](#) p. 6
- loss_box_reg [4.9](#) p. 32, [5.1](#) p. 38
- loss_classifier [4.2](#) p. 25, [5.1](#) p. 38
- loss_objectness [4.3](#) p. 26, [5.1](#) p. 38
- loss_rpn_box_reg [5.1](#) p. 38
- low confidence [9.3](#) p. 64

M

- machine learning [1.1](#) p. 5
- majority class [6.6](#) p. 48
- make_loader [10.4](#) p. 69
- make_split [6.7](#) p. 49
- mAP [8.5](#) p. 59
- mAP small medium large [8.5](#) p. 59
- mAP@0.5 [8.5](#) p. 59
- mAP@0.5:0.95 [8.5](#) p. 59
- mAP@0.75 [8.5](#) p. 59
- map_location [10.8](#) p. 71
- match_image [8.1](#) p. 55
- Matcher [4.7](#) p. 30
- matching [8.1](#) p. 55
- matching anchors [4.7](#) p. 30
- Matplotlib [10.3](#) p. 68
- max pooling [3.3](#) p. 20
- MAX_SIDE [6.4](#) p. 47
- max_size [10.5](#) p. 69
- mean average precision [8.5](#) p. 59
- MeanAveragePrecision [8.7](#) p. 60
- median [8.8](#) p. 61
- memorisation [1.9](#) p. 9
- merged with neighbour [9.3](#) p. 64
- metadata [6.3](#) p. 46
- metrics.json [10.7](#) p. 71
- million parameters [1.2](#) p. 5
- min_size [10.5](#) p. 69
- mini-batch [1.4](#) p. 6
- missed object [9.3](#) p. 64
- mixed precision [2.8](#) p. 17
- model [1.2](#) p. 5

- model comparison [5.4](#) p. 43
- model inputs [10.5](#) p. 69
- model outputs [10.5](#) p. 69
- model selection [7.3](#) p. 52
- model size [8.8](#) p. 61
- model.eval [7.2](#) p. 52
- model.train [7.2](#) p. 52
- model_info.json [10.7](#) p. 71, [10.9](#) p. 72
- modulating factor [5.2](#) p. 40
- momentum [1.7](#) p. 8
- more data [9.4](#) p. 65
- mount [10.2](#) p. 67
- multi-scale [4.12](#) p. 35
- multi-scale feature maps [5.3](#) p. 42

N

- nan [2.9](#) p. 17
- natural logarithm [4.2](#) p. 25
- near [9.1](#) p. 63
- near-duplicate [1.11](#) p. 11
- nearest neighbour [3.9](#) p. 23
- neck [3.7](#) p. 22
- negative anchor [4.7](#) p. 30
- negative log-likelihood [4.2](#) p. 25
- negative-to-positive ratio [5.3](#) p. 42
- neuron [2.1](#) p. 13
- NMS [4.11](#) p. 34
- NMS threshold [4.11](#) p. 34
- nms_thresh [10.5](#) p. 69
- nn.Module [10.3](#) p. 68
- noisy validation [7.4](#) p. 53
- non-determinism [1.13](#) p. 12
- non-linearity [2.2](#) p. 13
- non-maximum suppression [4.11](#) p. 34
- norm [2.9](#) p. 17
- normalised coordinates [4.4](#) p. 27
- normalisation [2.5](#) p. 15
- not detected [9.3](#) p. 64
- notebook [10.1](#) p. 67
- num_classes [4.1](#) p. 25
- num_workers [10.4](#) p. 69
- NumPy [10.3](#) p. 68

O

- object detection [4.1](#) p. 25
- object size [6.5](#) p. 48
- objective [1.3](#) p. 6
- objectness [4.3](#) p. 26
- observation [9.5](#) p. 66
- offsets [4.8](#) p. 31
- on-the-fly augmentation [6.8](#) p. 49
- one reason per error [9.1](#) p. 63
- one-stage detector [4.13](#) p. 36
- operating point [8.3](#) p. 56
- optimizer [1.7](#) p. 8
- orientation flag [6.3](#) p. 46
- origin [3.1](#) p. 19
- original images [6.1](#) p. 45
- out of memory [2.7](#) p. 16

- overfit a tiny dataset [10.10](#) p. 73
- overfitting [1.9](#) p. 9
- overflow [2.8](#) p. 17
- overlap [4.5](#) p. 28
- oversampling [6.6](#) p. 48
- overshoot [1.4](#) p. 6
- own matching code [8.7](#) p. 60

P

- P2 [4.12](#) p. 35
- P3 [4.12](#) p. 35
- P7 [4.12](#) p. 35
- p_t [5.2](#) p. 40
- padding [3.3](#) p. 20
- parameter [1.2](#) p. 5
- parameters [8.8](#) p. 61, [10.3](#) p. 68
- parameters vs compute [5.4](#) p. 43
- parsing [6.2](#) p. 45
- Pascal VOC format [4.4](#) p. 27
- path [10.2](#) p. 67
- pathlib [10.2](#) p. 67
- per-class NMS [4.11](#) p. 34
- per-class results [6.6](#) p. 48
- per-class sigmoid [4.3](#) p. 26
- per_class.csv [10.7](#) p. 71
- permanent storage [10.2](#) p. 67
- photometric transform [6.8](#) p. 49
- PIL [10.3](#) p. 68
- Pillow [10.3](#) p. 68
- pip [10.9](#) p. 72
- pip install [10.1](#) p. 67
- pixel [3.1](#) p. 19
- pooling [3.3](#) p. 20
- poor box [9.2](#) p. 64
- positive anchor [4.7](#) p. 30
- post-processing [4.11](#) p. 34
- PR curve [8.4](#) p. 57
- precision [8.2](#) p. 56
- precision envelope [8.4](#) p. 57
- precision-recall curve [8.4](#) p. 57
- precision-recall trade-off [8.3](#) p. 56
- predict.py [10.9](#) p. 72
- prediction [1.1](#) p. 5
- predictor [3.5](#) p. 21
- pretrained [3.8](#) p. 23
- prf_at [8.2](#) p. 56
- prior [2.6](#) p. 16
- prior box [4.6](#) p. 29
- prior probability [5.2](#) p. 40
- probability [2.2](#) p. 13
- project folder [10.2](#) p. 67
- PROJECT_DIR [10.2](#) p. 67
- proposal [5.1](#) p. 38
- pycocotools [8.7](#) p. 60, [10.3](#) p. 68
- pyramid level [4.12](#) p. 35
- PyTorch [10.3](#) p. 68

Q

- qualitative analysis [9.1](#) p. 63

qualitative evaluation [9.5](#) p. 66

R

random init [2.6](#) p. 16
random seed [1.13](#) p. 12
RandomApply [10.6](#) p. 70
RandomHorizontalFlip [6.8](#) p. 49
RandomIoUCrop [6.8](#) p. 49
RandomPhotometricDistort [6.8](#) p. 49
RandomZoomOut [6.8](#) p. 49
rare class [6.6](#) p. 48
raw pixels [6.3](#) p. 46
read_voc [6.2](#) p. 45
recall [8.2](#) p. 56
recall by object size [9.4](#) p. 65
receptive field [3.4](#) p. 21
region of interest [5.1](#) p. 38
region proposal [4.13](#) p. 36
region proposal network [5.1](#) p. 38
regression [4.8](#) p. 31
regularisation [1.7](#) p. 8, [1.9](#) p. 9
relative loss [7.4](#) p. 53
relative object size [9.4](#) p. 65
relative size [6.5](#) p. 48
ReLU [2.2](#) p. 13
replacing a head [10.5](#) p. 69
report tables [10.7](#) p. 71
reproducibility [1.13](#) p. 12
requires_grad [1.5](#) p. 7
rescaling boxes [6.4](#) p. 47
residual connection [3.6](#) p. 22
resize [3.9](#) p. 23
resizing [6.4](#) p. 47
ResNet-50 [3.6](#) p. 22
resolution [3.1](#) p. 19, [3.3](#) p. 20
restart runtime [10.1](#) p. 67
result files [10.7](#) p. 71
resuming training [7.3](#) p. 52
RetinaNet [5.2](#) p. 40
RetinaNet v2 [5.2](#) p. 40
RGB [3.1](#) p. 19
RoI [5.1](#) p. 38
RoIAlign [5.1](#) p. 38
RoIPool [5.1](#) p. 38
rotated photo [6.3](#) p. 46
rounding [6.7](#) p. 49
RPN [5.1](#) p. 38
rule order [9.1](#) p. 63
run all [10.1](#) p. 67
running statistics [2.5](#) p. 15
runtime [10.1](#) p. 67

S

sample size [8.9](#) p. 62
sampling [4.10](#) p. 33
SanitizeBoundingBoxes [6.8](#) p. 49
sanity check [10.10](#) p. 73
saving the best epoch [7.3](#) p. 52
scheduler [1.8](#) p. 9

score [2.2](#) p. 13
score threshold [4.11](#) p. 34
score_thresh [10.5](#) p. 69
seed [1.13](#) p. 12
seed search [6.7](#) p. 49
segmentation [4.1](#) p. 25
session [10.1](#) p. 67
settings [1.12](#) p. 11
SGD [1.7](#) p. 8
shape [2.4](#) p. 15
shell command [10.1](#) p. 67
shuffle [1.6](#) p. 8
sigmoid [2.2](#) p. 13
sigmoid score [4.3](#) p. 26
single run [7.5](#) p. 54
single shot detector [5.3](#) p. 42
single-shot [4.13](#) p. 36
size bins [9.4](#) p. 65
size mismatch [10.8](#) p. 71
skip connection [3.6](#) p. 22
small objects [6.5](#) p. 48, [9.4](#) p. 65
small test set [8.9](#) p. 62
smooth L1 [4.9](#) p. 32
Soft-NMS [4.11](#) p. 34, [9.4](#) p. 65
softmax [2.2](#) p. 13
speed [8.8](#) p. 61
split [1.10](#) p. 10
split by image [6.7](#) p. 49
split.json [6.7](#) p. 49
squared error [1.3](#) p. 6
SSD [5.3](#) p. 42
SSD300 [5.3](#) p. 42
SSDClassificationHead [5.3](#) p. 42
state_dict [10.8](#) p. 71
statistical reliability [8.9](#) p. 62
step [1.6](#) p. 8
step size [1.4](#) p. 6
stochastic gradient descent [1.4](#) p. 6
stratified split [6.7](#) p. 49
strict loading [10.8](#) p. 71
stride [3.3](#) p. 20
stuck training [7.4](#) p. 53
summary.csv [10.7](#) p. 71
supervised learning [1.1](#) p. 5

T

T4 [2.7](#) p. 16
tag [6.2](#) p. 45
target [1.1](#) p. 5
targets should not be none [7.2](#) p. 52
tensor [2.4](#) p. 15
terminal [10.9](#) p. 72
test set [1.10](#) p. 10
text file [6.2](#) p. 45
threshold chosen on validation [8.3](#) p. 56
TIDE [9.1](#) p. 63
tiling [9.4](#) p. 65
time_model [8.8](#) p. 61
ToDtype [6.8](#) p. 49

top-down pathway [4.12](#) p. 35
top-k [4.11](#) p. 34
torch [10.3](#) p. 68
torch.load [10.8](#) p. 71
torch.no_grad [7.2](#) p. 52
torch.save [7.3](#) p. 52, [10.8](#) p. 71
torchmetrics [8.7](#) p. 60, [10.3](#) p. 68
torchvision [10.3](#) p. 68
torchvision detection API [10.5](#) p. 69
total loss [1.3](#) p. 6
TP [8.1](#) p. 55
traceback [10.10](#) p. 73
train_model [7.1](#) p. 51
trainable layers [3.8](#) p. 23
training curve [7.4](#) p. 53
training log [7.1](#) p. 51
training loop [7.1](#) p. 51
training mode [7.2](#) p. 52
training set [1.10](#) p. 10
transfer learning [3.8](#) p. 23
transformer detector [4.13](#) p. 36
transforms must move boxes [6.8](#) p. 49
transforms.v2 [10.6](#) p. 70
TrashDataset [10.4](#) p. 69
troubleshooting [10.10](#) p. 73
true negative [8.1](#) p. 55
true positive [8.1](#) p. 55
tuning [1.12](#) p. 11
tuning budget [7.5](#) p. 54
tv_tensors [10.6](#) p. 70
two-stage [5.1](#) p. 38
two-stage detector [4.13](#) p. 36
tx ty tw th [4.8](#) p. 31

U

uint8 [2.4](#) p. 15
uncertainty [8.9](#) p. 62
under-training [7.4](#) p. 53
underfitting [1.9](#) p. 9
underflow [2.8](#) p. 17
union [4.5](#) p. 28
unseen photos [9.5](#) p. 66
unstable training [7.4](#) p. 53
update [8.7](#) p. 60
upsampling [3.9](#) p. 23

V

v2 [5.4](#) p. 43
validation curve [7.4](#) p. 53
validation set [1.10](#) p. 10
variance [8.9](#) p. 62
VGG-16 [3.6](#) p. 22, [5.3](#) p. 42
virtual environment [10.9](#) p. 72
visual checks [10.10](#) p. 73
VRAM [2.7](#) p. 16

W

warm-up [1.8](#) p. 9, [8.8](#) p. 61
weight [1.2](#) p. 5
weight decay [1.7](#) p. 8

weight sharing [3.2](#) p. 19
weighted sum [2.1](#) p. 13
weights enum [10.5](#) p. 69
weights_backbone [10.5](#) p. 69
weights_only [10.8](#) p. 71
WORK_DIR [10.2](#) p. 67
writefile [10.1](#) p. 67
writing observations [9.5](#) p. 66

wrong class [9.2](#) p. 64

X

Xavier [2.6](#) p. 16
XML [6.2](#) p. 45
xywh [4.4](#) p. 27
xyxy [4.4](#) p. 27

Y

yes/no question [4.3](#) p. 26
YOLO [4.13](#) p. 36
YOLO format [4.4](#) p. 27

Z

zero-size box [6.4](#) p. 47
zero_grad [1.5](#) p. 7
zip file [10.2](#) p. 67